

Aula 09

DataPrev (Perfil 3: Desenvolvimento de Software) Desenvolvimento de Software - 2026 (Pós-Edital)

Autor:
Vinicius Borges

09 de Julho de 2026

Índice

1) SPA - Teoria	3
2) SPA - Questões Comentadas	7
3) SPA - Lista de Questões	10
4) Angular - Teoria	13
5) Angular - Questões Comentadas	35
6) Angular - Lista de Questões	49
7) AngularJS - Teoria	58
8) AngularJS - Questões Comentadas	80
9) AngularJS - Lista de Questões	90
10) ReactJS - Teoria	97
11) ReactJS - Questões Comentadas	126
12) ReactJS - Lista de Questões	145
13) VueJS - Teoria	157
14) VueJS - Questões Comentadas	175
15) VueJS - Lista de Questões	181
16) AJAX - Teoria	185
17) AJAX - Questões Comentadas	201
18) AJAX - Lista de Questões	212
19) PWA - Teoria	219
20) PWA - Questões Comentadas	222
21) PWA - Lista de Questões	226

SINGLE PAGE APPLICATION

Conceitos Gerais

O **desenvolvimento web** passou por muitas evoluções desde sua concepção inicial ao público em geral. Quem é um pouco mais “antigo” (não que eu seja velho, mas navego na web desde os anos 90, já que cresci com um pai programador e fissurado na internet) lembra de como eram as páginas antigamente: feias, lerdas, usando ou Flashplayer, ou Java (aquele do cafézinho).

Cada página na web que você acessava era um “programa” separado, necessitava de um carregamento específico e deixava a experiência muito pouco fluída. Não tínhamos nenhuma interatividade e dinamicidade. Eram basicamente páginas de texto que você acessava através de um navegador.



Evoluções precisavam ser feitas. O código estático, feito em HTML ou HTML 2, passou a incorporar dinamicidade. A Netscape introduziu o JavaScript, permitindo adicionar dinamicidade às páginas através de *scripts* interativos: surge o Dynamic HTML (DHTML). Além disso, o HTML 4 trouxe uma integração de 3 elementos que estruturam as páginas web: HTML, trazendo o conteúdo e a estruturação; CSS, trazendo os estilos e a parte visual; JavaScript trazendo a dinamicidade.

Além disso, passamos a ter páginas que executam *sripts* do lado do servidor, ao contrário do JavaScript, que os executa no lado do cliente. Temos linguagens como PHP, ASP e JSP para a criação das páginas.

Porém, apesar de termos interatividade, ainda era necessário recarregar a página inteira para qualquer interação. *Logou no sistema? Atualiza a página. Adicionou algo ao carrinho de compras? Atualiza a página novamente.* Isso tornava a interação ainda não tão fluida quanto era desejado pelos usuários.

Com isso temos a ascensão da Web 2.0, nos anos 2000. Tivemos a introdução do AJAX, ou Asynchronous JavaScript and XML. O AJAX permitiu a criação de páginas em que apenas a parte que fosse necessária do site seja atualizada, ao contrário do site inteiro. Isso trouxe interatividade e velocidade - uma grande ferramenta possibilitada por essa introdução foram as ferramentas SaaS, principalmente aplicações que suportam múltiplos usuários, como o Google Docs.

Porém, a internet se popularizou e uma coisa que sempre é levada como prioridade, quase que acima de todo o resto, é a User Experience (também chamada de UX). Precisávamos avançar ainda mais, de forma a termos verdadeiros aplicativos, como os programas que você acessa no seu computador, mas acessados através do navegador. É nesse contexto que surgem as **Single Page Applications**, ou **SPAs**.

As principais finalidades e objetivos das SPAs são:

- Elevar a **experiência do usuário**, fornecendo uma interação fluida e rápida, sem necessidade de recarregar a página tanto para interações, quanto para mudanças de páginas dentro de um mesmo site
- Aumentar o **desempenho**, já que apenas os recursos e dados necessários para o que já é exibido são carregados. Além disso, a renderização da página é feita diretamente no lado do cliente.
- Aumento da **facilidade de manutenção** e da **capacidade de desenvolvimento** de novas *features* para o site
- Garantir uma maior **interatividade**, incluindo atualizações em tempo real, animações e abordagens mais avançadas e complexas

Assim, antigamente, em MPA (Multi-Page Applications), quando você clicava em algum link que redirecionava você para outra página do "programa" (aqui por programa, entenda o site), você precisaria recarregar todo o site. Agora, no SPA, você vai estar sempre dentro do aplicativo, não precisando recarregar todo o site.

Você deve conviver com esse exemplo no dia a dia. Pense no Gmail ou Outlook, sempre que você acessa seus e-mails você pode abrir uma mensagem, ir para a caixa de *spam* ou outras localidades sem recarregar toda a página. Isso é uma SPA.

Claro que nem sempre elas são indicadas. Quando temos uma grande mudança de temas, ou de contexto, entre páginas, pode ser interessante usar as aplicações em múltiplas páginas. Por

exemplo, no Estratégia, ao acessarmos o portal inicial temos diversas verticais: Medicina, Concursos, Vestibulares. Podemos ter uma SPA para cada uma dessas verticais, mas não é interessante colocar todas numa mesma aplicação.

Para implementarmos as SPAs, utilizamos, usualmente, diferentes **bibliotecas**, ou **frameworks**, de **JavaScript**. Essas bibliotecas implementam diferentes formas de lidarmos com o HTML, CSS e JavaScript de forma a criarmos os SPAs.

(CEBRASPE/TCE PA/2016) Julgue o item que se segue, relativamente a desenvolvimento de sistemas web.

O Single Page Application, modelo de desenvolvimento de aplicações para Web e dispositivos móveis, utiliza templates que rodam basicamente no lado do cliente.

Comentários:

Exatamente isso! O JavaScript é uma linguagem de *script* que roda client-side, e, como a maior parte das SPA são implementadas a partir de bibliotecas (também chamadas de frameworks ou templates) derivadas do JavaScript, a afirmativa acaba se tornando verdadeira. (Gabarito: Certo)

Os principais frameworks que exploram essa tecnologia de SPA são:

- **Angular:** o angular é um framework robusto, baseado em **componentes** e injeção de dependências. Apesar de suportar JavaScript, ele trabalha primariamente com TypeScript, que adiciona tipagem estática e forte ao JavaScript.
- **React:** uma biblioteca de JavaScript, criada pela Meta (Facebook), focada na construção da página a partir de módulos reutilizáveis. Utiliza um conceito de DOM virtual para melhorar o desempenho e atualizações de interface, além de permitir integração com outras bibliotecas para roteamento e gerenciamento de estado (como o React Router e o Redux).
- **Vue.js:** é um framework progressivo para a construção de interfaces de usuário, permitindo a integração incremental em projetos existentes. Ele é conhecido por sua curva de aprendizado suave, flexibilidade e capacidade de ser utilizado tanto para projetos pequenos quanto para grandes aplicações complexas.
- **Svelte:** o Svelte é um framework que adota uma abordagem diferente, compilando o código dos componentes para JavaScript puro no tempo de build, eliminando a necessidade de um framework durante a execução. Ele oferece um desempenho superior em comparação com outros frameworks, devido à eliminação da camada de abstração no tempo de execução.

Um detalhe: apesar de grande parte das implementações terem sido desenvolvidas, em um primeiro momento, para JavaScript, **a grande maioria já traz suporte para o TypeScript**. Com o TS (TypeScript), introduzimos um controle de tipos superior, com uma tipagem forte (onde o tipo

precisa ser declarado), e uma tipagem estática (que não permite operações entre tipos diferentes). Isso acaba trazendo maior confiabilidade ao código.

QUESTÃO DE PROVA



(Inédita/Prof. Felipe Mathias) O Estado Alpha deseja implementar seu portal de transparência a partir de uma SPA (Single Page Application). Para tanto, precisa escolher um *framework* para desenvolver essa aplicação.

Assinale a alternativa que representa uma linguagem ou framework que não pode ser escolhida pelo ente:

- a) React.js
- b) Angular
- c) Vue.js
- d) Django
- e) Svelte

Comentários:

O framework que não pode ser escolhido é o **Django**. Ele é um framework de desenvolvimento web baseado em Python que é mais adequado para aplicações tradicionais baseadas em servidor (server-side) do que para Single Page Applications (SPAs), que geralmente utilizam frameworks JavaScript como React, Angular, Vue.js ou Svelte. (Gabarito: Letra D)

QUESTÕES COMENTADAS

01. (FUNDATEC/PROCERGS/2023) Assinale a alternativa que descreve corretamente o conceito de Single Page Applications (SPA).

- a) É um padrão arquitetural para o desenvolvimento de aplicações de banco de dados.
- b) É uma tecnologia para a criação de sites estáticos.
- c) É uma técnica para melhorar a velocidade de carregamento de páginas web.
- d) É uma abordagem para desenvolvimento de aplicações web que carregam uma única página HTML e atualizam dinamicamente partes da página conforme o usuário interage com a aplicação.
- e) É uma plataforma para desenvolvimento de aplicações de negócios para Windows.

Comentários:

Vamos analisar cada alternativa.

- a) Errado. É um padrão para desenvolvimento de websites.
- b) Errado. É uma tecnologia para sites dinâmicos.
- c) Errado. Embora a SPA possa melhorar a velocidade de carregamento, já que carregamos apenas a parte da página que estamos visualizando. Mas não é o foco do SPA, já que, por termos muitos arquivos numa mesma página, podemos ter situações em que o desempenho é pior.
- d) Certo. O SPA transforma aplicativos web para serem lidos em uma página única, carregando e atualizando dinamicamente o conteúdo.
- e) Errado. É para aplicações web, não Windows.

Portanto, correta a letra D.

Gabarito: Letra D

02. (CEBRASPE/TCE RJ/2022) Quanto ao desenvolvimento de sistemas web, julgue o item seguinte.

Uma aplicação construída como SPA (single page applications) carrega, quando necessário e de forma assíncrona, apenas a parte principal da página, mantendo os demais elementos fixos.

Comentários:

Perfeito. A SPA mantém elementos estruturantes fixos, que não precisarão ser carregados novamente, e partes dinâmicas que podem ser atualizadas, através de comunicações assíncronas.

Gabarito: Correto

03. (CEBRASPE/CODEVASF/2019) Acerca de desenvolvimento de sistemas, julgue o próximo item.

O modelo SPA (single page application) permite a otimização do desempenho da aplicação ao transferir a renderização para a máquina do cliente e diminuir o tráfego de dados entre cliente e servidor.

Comentários:

Item correto, caro aluno. As SPA utilizam tecnologias, bibliotecas ou frameworks derivados do JavaScript, uma linguagem de programação que roda *client-side*. Com isso, conseguimos transmitir o código para o seu navegador, que o interpreta e renderiza a imagem, gerando os elementos que você interage na página. Com isso, uma quantidade menor de dados trafega pela rede, otimizando a conexão.

Gabarito: Correto

04. (Instituto AOCP/PRODEB/2018) Sobre o Framework de desenvolvimento estrutural Angular, analise as assertivas e assinale a alternativa que aponta a(s) correta(s).

- I. O Framework Estrutural Angular é uma ótima opção para criar SPA (Single Page Applications).
- II. O Framework Estrutural Angular foi desenvolvido para criar aplicações web dinâmicas.
- III. O Framework Estrutural Angular foi desenvolvido para criar banco de dados relacionais.
- IV. O Framework Estrutural Angular é utilizado para a configuração de redes IPV6.

- a) Apenas II.
- b) Apenas I, III e IV.
- c) Apenas I e II.
- d) Apenas III e IV.
- e) Apenas II e III.

Comentários:

Mesmo a aula não tendo foco em Angular, sabendo que ele é uma tecnologia para criar SPAs, a questão fica tranquila. Vamos analisar os itens.

I. Certo. Como acabei de comentar, o Angular é um framework destinado a criar SPAs baseado em JavaScript, assim como React e VueJS.

II. Certo. Como criamos SPAs, eles obrigatoriamente são dinâmicos.
III e IV. Errado. Vide item I e II.

Portanto, corretos os itens I e II.

Gabarito: Letra C

05. (CEBRASPE/TCE PA/2016) Julgue o item que se segue, relativamente a desenvolvimento de sistemas web.

O Single Page Application, modelo de desenvolvimento de aplicações para Web e dispositivos móveis, utiliza templates que rodam basicamente no lado do cliente.

Comentários:

Deu para perceber que é uma cobrança recorrente sobre SPA né? Elas são aplicações web que rodam no lado do cliente, para diminuir o tráfego em rede - item correto.

Gabarito: Correto

LISTA DE QUESTÕES

01. (FUNDATEC/PROCERGS/2023) Assinale a alternativa que descreve corretamente o conceito de Single Page Applications (SPA).

- a) É um padrão arquitetural para o desenvolvimento de aplicações de banco de dados.
- b) É uma tecnologia para a criação de sites estáticos.
- c) É uma técnica para melhorar a velocidade de carregamento de páginas web.
- d) É uma abordagem para desenvolvimento de aplicações web que carregam uma única página HTML e atualizam dinamicamente partes da página conforme o usuário interage com a aplicação.
- e) É uma plataforma para desenvolvimento de aplicações de negócios para Windows.

02. (CEBRASPE/TCE RJ/2022) Quanto ao desenvolvimento de sistemas web, julgue o item seguinte.

Uma aplicação construída como SPA (single page applications) carrega, quando necessário e de forma assíncrona, apenas a parte principal da página, mantendo os demais elementos fixos.

03. (CEBRASPE/CODEVASF/2019) Acerca de desenvolvimento de sistemas, julgue o próximo item.

O modelo SPA (single page application) permite a otimização do desempenho da aplicação ao transferir a renderização para a máquina do cliente e diminuir o tráfego de dados entre cliente e servidor.

04. (Instituto AOCP/PRODEB/2018) Sobre o Framework de desenvolvimento estrutural Angular, analise as assertivas e assinale a alternativa que aponta a(s) correta(s).

- I. O Framework Estrutural Angular é uma ótima opção para criar SPA (Single Page Applications).
- II. O Framework Estrutural Angular foi desenvolvido para criar aplicações web dinâmicas.
- III. O Framework Estrutural Angular foi desenvolvido para criar banco de dados relacionais.
- IV. O Framework Estrutural Angular é utilizado para a configuração de redes IPV6.

- a) Apenas II.
- b) Apenas I, III e IV.
- c) Apenas I e II.
- d) Apenas III e IV.
- e) Apenas II e III.

05. (CEBRASPE/TCE PA/2016) Julgue o item que se segue, relativamente a desenvolvimento de sistemas web.

O Single Page Application, modelo de desenvolvimento de aplicações para Web e dispositivos móveis, utiliza templates que rodam basicamente no lado do cliente.

GABARITO

GABARITO



1. Letra D
2. Correto
3. Correto
4. Letra C
5. Correto

ANGULAR

Conceitos Gerais



O **Angular** é uma biblioteca (*framework*) destinado a **aplicações de página única** (SPA - *Single Page Application*) através da combinação de HTML, CSS e JavaScript. O Angular é uma *framework* open-source, mantido principalmente pelo Google. É, hoje, um dos principais *frameworks* quando se deseja criar uma SPA rodando no *client-side*, isso é, no próprio navegador do cliente - característico do JavaScript.

Existem duas “versões” do Angular: o AngularJS, que opera primariamente com HTML, CSS e JavaScript, e o Angular, que se refere às versões 2.0 e posteriores. Nessa aula vamos aprender o **Angular**, referente às versões Angular 2 e posteriores - sendo mais preciso, essa aula será baseada na versão estável mais recente, a versão 18. O AngularJS implementa, além da *stack* padrão composta por JavaScript, HTML e CSS, inovações trazidas pelo TypeScript.

Para compreender o Angular, é essencial ter conhecimento sobre três componentes:

- Classes de JavaScript
- Fundamentos do TypeScript
- Decorators do TypeScript

Caso você não tenha conhecimento sobre esses componentes, irei abordá-los de forma que você possa compreender o Angular nos próximos capítulos. Caso você já tenha conhecimento sobre esses componentes, sinta-se tranquilo para pular essas seções e continuar a partir do capítulo “Componentes”.

Classes JavaScript

As **classes de JavaScript** foram uma novidade introduzida pelo ECMAScript 2015, trazendo uma simplificação da linguagem para heranças baseadas em protótipos no JavaScript. O grande objetivo é adaptar o JavaScript aos padrões da orientação a objetos, permitindo a criação de códigos mais íntegros e escaláveis.

As classes são usadas para:

- **Definir objetos:** criar instâncias de objetos que possuem propriedades e métodos.
- **Herança:** permitir que uma classe (subclasse) herde características (propriedades e métodos) de outra classe (superclasse).
- **Encapsulamento:** organizar e encapsular dados e funcionalidades relacionadas.
- **Reutilização de código:** promover a reutilização de código através da herança e da composição.

A definição de uma classe é feita a partir da diretiva **class {}**. Veja um exemplo:

JavaScript

```
class Pessoa {  
  constructor(nome, idade) {  
    this.nome = nome;  
    this.idade = idade;  
  }  
  dizerOla() {  
    console.log(`Olá, meu nome é ${this.nome} e eu tenho  
    ${this.idade} anos.`);  
  }  
}
```

Com as classes definidas, podemos instanciá-las a partir da operação de construção (`new`). Veja:

JavaScript

```
const pessoa1 = new Pessoa('Felipe', 30);  
pessoa1.dizerOla();
```

Veja que passamos os valores para nome e idade como parâmetros do construtor e, por estarmos instanciando uma classe (ou seja, criando um objetos), herdamos os métodos - permitindo executar o `dizerOla()` originário da classe base. Podemos também estender classes para adicionar heranças. Vamos novamente a um exemplo.

JavaScript

```
class Funcionario extends Pessoa {  
  constructor(nome, idade, cargo) {  
    super(nome, idade); // Chama o construtor da classe base  
    (Pessoa)  
    this.cargo = cargo;  
  }  
  descreverTrabalho() {  
    console.log(`Eu sou um ${this.cargo}.`);  
  }  
}  
  
const funcionario1 = new Funcionario('Maria', 25, 'Engenheira');  
funcionario1.dizerOla();  
funcionario1.descreverTrabalho();
```

Também é possível definir métodos estáticos, que são métodos que pertencem a própria classe, e não a suas instâncias. Assim podemos chamar o método diretamente na classe, sem precisar criar uma instância dela. A definição é feita a partir da palavra-chave `static` seguido da função (método). Por exemplo, `static dizerOla()`.

Um ponto importante. Diferentemente das funções no JavaScript, as **classes não sofrem hoisting**. Então é necessário declarar a classe antes de poder usá-la.

Você viu a forma de constituir uma classe a partir de uma **declaração de classe**. Alternativamente, é possível criar uma classe a partir de uma **expressão de classe**. Nesse caso, criamos uma variável que receberá a classe - que pode ser com ou sem nome. Veja:

```
JavaScript

// sem nome
let Retangulo = class {
  constructor(altura, largura) {
    this.altura = altura;
    this.largura = largura;
  }
};

// nomeada
let Retangulo = class Retangulo {
  constructor(altura, largura) {
    this.altura = altura;
    this.largura = largura;
  }
};
```

Temos também alguns métodos específicos para as classes. Veja uma tabela-resumo com os principais métodos que você deve saber.



Método	Descrição
constructor	Método especial para criar e inicializar um objeto criado com uma classe. Só pode haver um método constructor por classe.
super	Chama o construtor da classe pai. Deve ser chamado antes de usar this em um construtor derivado.

get	Define um método que será chamado quando uma propriedade específica for acessada.
set	Define um método que será chamado quando uma propriedade específica for atribuída.
static	Define métodos estáticos para a classe, que são chamados diretamente na classe, não em instâncias da classe.
extends	Utilizado para criar uma classe filha que herda de outra classe.
methodName	Qualquer método definido dentro de uma classe que se torna uma propriedade do protótipo da instância da classe.
prototype	Propriedade interna de todas as funções (inclusive das funções construtoras de classes) que é utilizada para a herança prototípica.
name	Retorna o nome da classe. Este é um método de leitura.
length	Retorna o número de parâmetros esperados pelo método constructor.

(FGV/Pref. BH/2024) Analise o código Javascript, a seguir.

```
class Ave {  
  constructor(nome) {  
    this.nome = nome;  
  }  
  som() {  
    throw new Error("Implementar");  
  }  
}  
  
class Pato extends Ave {  
  constructor(nome) {  
    super(nome);  
  }  
  som() {  
    return "Quá quá!";  
  }  
}  
  
class Galinha extends Ave {  
  constructor(nome) {  
    super(nome);  
  }  
  som() {  
    return "Cocorocoo!";  
  }  
}
```

Sobre o código Javascript apresentado, analise as afirmativas a seguir e assinale (V) para a verdadeira e (F) para a falsa.

() A classe Ave é uma classe abstrata que fornece uma interface para outras classes de aves.

- () A classe Pato e a classe Galinha são exemplos de polimorfismo, pois ambas sobrescrevem o método som() da classe Ave.
- () O método som() na classe Ave pode ser chamado diretamente para fazer o som da ave.

As afirmativas são, respectivamente,

- a) F – V – F.
- b) V – F – F.
- c) V – V – F.
- d) V – F – V.

Comentários:

Vamos analisar cada afirmativa.

() A classe Ave é uma classe abstrata que fornece uma interface para outras classes de aves. Verdadeiro. A classe ave é considerada uma classe abstrata, pois serve como um esqueleto para suas instâncias.

() A classe Pato e a classe Galinha são exemplos de polimorfismo, pois ambas sobrescrevem o método som() da classe Ave.

Falso. A classe Pato e Galinha caracterizam a herança entre superclasse e subclasses - ou seja, entre ave e pato ou galinha. O polimorfismo existe sim, mas é no método som(), que possui diferentes funções em cada classe que é implementado.

() O método som() na classe Ave pode ser chamado diretamente para fazer o som da ave. Verdadeiro. Ao contrário de outras linguagens, não há limitação para chamar métodos de uma classe pai diretamente. Poderíamos fazer o seguinte:

```
ave = new Ave(); ave.som();
```

Como a classe pai prevê um resultado para o método, mesmo que seja lançar um erro, não há problema nenhum com essa ação.

Portanto, ficamos com V-F-V. (Gabarito: Letra D)

Fundamentos do TypeScript

Tradicionalmente, o JavaScript é uma linguagem de tipagem fraca, isso é, não há necessidade de declaração do tipo - isso pois o JavaScript faz uma inferência automática do tipo declarado, de acordo com seus tipos primitivos, como string e number.

```
let helloWorld = "Hello World";  
let helloWorld: string
```

O problema é que não há uma verificação constante de que os tipos primitivos designados estão sendo seguidos. É aí que entra o TypeScript. Ele traz todas as funcionalidades do JavaScript, e adiciona uma camada no topo para trazer **tipagem forte**, **estatática** e, conseqüentemente, um **controle de tipagem**. Com isso, temos mais consistência e robustez nas aplicações. Veja uma comparação entre declarações de tipos:

JavaScript

```
// JavaScript  
function greet(name) {  
  return "Hello, " + name;  
}  
  
// TypeScript  
function greet(name: string): string {  
  return "Hello, " + name;  
}
```

Podemos interagir com classes ou objetos também, através da definição de uma interface. Aqui, a interface servirá como um “acordo” um pouco diferente do tradicional na orientação a objetos - ao invés de definir os métodos que podem ser implementados, temos a definição dos tipos que a classe deve seguir. Veja:

JavaScript

```
interface User {  
  name: string;  
  id: number;  
}  
  
const user: User = {  
  name: "Felipe",  
  id: 0,  
}
```

Se tentarmos atribuir um número à variável `name`, por exemplo, o TypeScript apontará um erro no próprio editor do código. Veja:

```
Type 'number' is not assignable to type 'string'. ts(2322)
main.ts(2, 3): The expected type comes from property 'name' which is declared here on type 'User'
(property) User.name: string
```

Outro ponto interessante do TypeScript é a **criação de tipos complexos**. Podemos criar tipos complexos combinando os diferentes tipos primitivos. Há duas formas comuns de criação: a partir da **união**, e a partir da **generalização**.

Com a união, podemos declarar um tipo como um conjunto de elementos. Para isso, usamos o *pipe* `|` para indicar uma operação de OU lógico entre diversos tipos. Veja dois exemplos, onde criamos um tipo para valores *booleanos* `true` ou `false`, e outro tipo para conter os valores do “estado” de uma janela.

JavaScript

```
type myBool = true | false
type estadoJanela = "aberta" | "fechada" | "entreaberta"
```

Já a criação de tipos genéricos, chamada também de *generics*, prove variáveis para um tipo. Um exemplo comum é na criação de uma *array* - quando não atribuímos um *generics* para ela, podemos ter qualquer tipo atribuído a ela. Já com o *generics*, podemos especificar o tipo de dado contido nos seus valores.

JavaScript

```
type arrayDeTexto = Array<string>;
type arrayDeNumeros = Array<number>;
```

(IADES/UnDF/2022) A respeito das diferenças e das semelhanças entre as linguagens de programação TypeScript e JavaScript em suas formas puras, ou seja, sem considerar frameworks ou variações, assinale a alternativa correta.

- a) JavaScript é focada em client-side, enquanto TypeScript é focada em client-side e server-side.
- b) JavaScript é uma linguagem compilada, ao passo que TypeScript é interpretada.
- c) Ambas são linguagens fortemente tipadas.
- d) TypeScript, ao contrário de JavaScript, suporta tipagem estática.
- e) JavaScript surgiu como uma derivação do TypeScript e, por isso, a comunidade de TypeScript é consideravelmente maior do que a de JavaScript.

Comentários:

Vamos analisar cada alternativa.

- a) Errado. Tanto o JS quanto o TS são client-side.
- b) Errado. Javascript é interpretada e Typescript é compilada.
- c) Errado. O JS é fracamente tipado - é justamente essa a “diferença” implementada pelo TS.
- d) Certo. O TS suporta a linguagem estática, isso é, o tipo do dado deve ser declarado expressamente.

e) Errado. O TS que surgiu como uma derivação do JS.

Portanto, correta a letra D. (Gabarito: Letra D)

(FGV/TJ MS/2024) Observe o seguinte script em TypeScript:

```
interface Tribunal {  
  sigla: string;  
  id: integer;  
}  
  
const tjms: Tribunal = {  
  id: 4,  
  sigla: "TJMS"  
}
```

Ao ser executado em um interpretador com TypeScript na versão 4 ou mais recente, o script acima imprime um erro indicando:

- a) a indefinição do nome 'string';
- b) a indefinição do nome 'integer';
- c) a indefinição do nome 'interface';
- d) o uso inesperado de ';' ao declarar Tribunal;
- e) a atribuição indevida do id de tjms antes da sigla;

Comentários:

Essa questão vai no detalhe do detalhe. No JavaScript, não temos um tipo primitivo *integer*, apenas o tipo destinado a números - *number*. Por isso, ao fazermos a declaração `id: integer`, temos uma indefinição, já que o tipo não foi declarado. Alternativamente, poderíamos criar um tipo para ele, como `type integer = number` - mas, como não temos essa definição no código, o erro reside na indefinição do nome 'integer'.
(Gabarito: Letra B)

Decorators

Os **Decoradores** ou **Decorators** são **formas de incluir anotações e metadados** às classes, métodos, propriedades ou parâmetros de classe. As anotações já são elementos comuns em linguagens voltadas para o *backend*, como o Java, e agora podem ser também implementados no TypeScript - representando uma forma de **modificar ou estender o comportamento dos elementos** de um código de maneira declarativa.

Os Decorators têm suas raízes na programação orientada a aspectos (AOP) e em outras linguagens de programação que permitem modificações dinâmicas de comportamento. No JavaScript/TypeScript, os Decorators foram inspirados por outras linguagens como Python e Java, onde anotações e decoradores já eram comuns.

O principal objetivo dos Decorators é permitir que os desenvolvedores adicionem metadados e modifiquem o comportamento de classes e seus membros de uma maneira modular e reutilizável. Eles são especialmente úteis para:

- **Injeção de Dependência:** Fornecer serviços ou instâncias a uma classe ou método.
- **Validação:** Adicionar lógica de validação a propriedades de classes.
- **Logging e Monitoramento:** Adicionar automaticamente lógica de logging ou monitoramento a métodos.
- **Controle de Acesso:** Implementar verificações de segurança e controle de acesso a métodos.
- **Transformações de Propriedades:** Modificar ou transformar valores de propriedades.

Para habilitar decoradores, devemos modificar o arquivo `tsconfig.json`, que traz as configurações do TypeScript:

```
JSON
{
  "compilerOptions": {
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true
  }
}
```

Os decoradores são caracterizados pela presença do `@`, simbolizando que é uma anotação. Então, sempre que vir algo como `@decorator`, `@sealed`, entre outros. Veja um exemplo em código da aplicação de um decorador.

TypeScript

```
function sealed(constructor: Function) {  
  Object.seal(constructor);  
  Object.seal(constructor.prototype);  
}  
  
@sealed  
class Greeter {  
  greeting: string;  
  
  constructor(message: string) {  
    this.greeting = message;  
  }  
  
  greet() {  
    return `Hello, ${this.greeting}`;  
  }  
}
```

Acima temos um exemplo de decorador de classe. Agora é interessante que você saiba os principais decoradores utilizados, que se dividem entre decoradores de classe, de método, de propriedade e de parâmetro. Vou trazer novamente uma tabela-resumo com os principais e suas descrições.

DECORADOR	TIPO	DESCRIÇÃO
@sealed	Classe	Sela a classe e seu protótipo, impedindo que novas propriedades sejam adicionadas a eles.
@singleton	Classe	Garante que a classe tenha apenas uma instância e fornece um ponto de acesso global a ela.
@injectable	Classe	Marca uma classe como disponível para injeção de dependência.
@controller	Classe	Define uma classe como um controlador, especialmente em frameworks web como NestJS.
@component	Classe	Marca uma classe como um componente, especialmente em frameworks como Angular.
@log	Método	Adiciona logging ao método, registrando os argumentos com os quais o método é chamado e o valor de retorno.
@debounce	Método	Adiciona um atraso à execução do método para evitar múltiplas chamadas em um curto período de tempo.
@readonly	Método	Torna um método de uma classe não reescrevível.
@deprecated	Método	Marca um método como obsoleto, avisando os desenvolvedores para não usá-lo mais.
@autobind	Método	Liga automaticamente um método à instância atual quando usado como propriedade de classe.
@catchError	Método	Adiciona tratamento de exceções ao método, capturando e lidando com erros.

@measureTime	Método	Mede o tempo de execução de um método e registra o resultado.
@format	Propriedade	Converte o valor da propriedade para maiúsculas sempre que for definido.
@observable	Propriedade	Converte uma propriedade em uma observável, notificando mudanças.
@readonly	Propriedade	Torna uma propriedade de uma classe somente leitura.
@autobind	Propriedade	Liga automaticamente um método à instância atual quando usado como propriedade de classe.
@validate	Propriedade	Adiciona validação aos valores da propriedade, garantindo que atendam a certos critérios.
@enumerable	Propriedade	Define se a propriedade é enumerável ou não.
@nonenumerable	Propriedade	Define a propriedade como não enumerável.
@serializable	Propriedade	Marca a propriedade para ser incluída na serialização de objetos.
@required	Parâmetro	Marca o parâmetro do método como obrigatório, armazenando essa informação em metadados.
@inject	Parâmetro	Injeta uma dependência no parâmetro do método.
@validate	Parâmetro	Adiciona validação aos parâmetros do método, garantindo que atendam a certos critérios.
@queryParam	Parâmetro	Extraí um parâmetro da query string para o método.
@bodyParam	Parâmetro	Extraí um parâmetro do corpo da requisição para o método.
@headerParam	Parâmetro	Extraí um parâmetro dos cabeçalhos da requisição para o método.

(Inédita/Prof. Felipe Mathias) Acerca dos conhecimentos sobre linguagens de programação, julgue o item subsequente.

No TypeScript, os *Decorators* são formas de introduzir anotações ao código. O decorador @sealed é usado para impedir que novas propriedades sejam adicionadas a uma classe já construída.

Comentários:

O item está correto! Os Decoratos introduzem as anotações no contexto do TypeScript, adicionando novos comportamentos aos elementos. O @sealed é uma forma de selar a classe para que não sejam adicionadas novas propriedades a uma classe. (Gabarito: Certo)

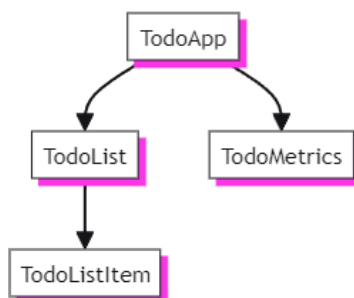
Componentes

Com as bases do conhecimento devidamente estabelecidas, podemos dar continuidade ao estudo do *framework* Angular. Os **componentes** são os **blocos fundamentais para criação de aplicações no Angular**. É com base neles que toda e qualquer aplicação desenvolvida em Angular se estrutura.

COMPONENTE → BLOCO FUNDAMENTAL DE CRIAÇÃO

Os componentes fornece estruturas para organizar o projeto em partes de fácil entendimento, com uma clara atribuição de responsabilidades, para que o código desenvolvido tenha uma manutenção facilitada e escale de melhor forma. Pense num componente como uma forma modular de compor uma aplicação completa.

Por exemplo, um TodoApp pode ser quebrado em outros componentes menores, cada um com uma função específica.



Um componente é essencialmente uma classe, e é definido a partir de um decorador **@Component**. Cada componente irá ser composto de três partes principais:

- **Metadado:** definido pelo decorador **@Component**, onde são especificadas as configurações do componente
- **Classe:** onde a lógica do componente é implementada
- **Template:** define a interface de usuário do componente

Veja um exemplo de uma simples implementação de um componente - lembrando que precisamos importar o elemento **Component** para o modelo, para poder trabalhar com o elemento do framework.

TypeScript

```
import { Component } from '@angular/core';

@Component ({
  selector: 'app-exemplo',
  templateUrl: './exemplo.component.html',
  styleUrls: ['./exemplo.component.css']
})
export class ExemploComponent {
  titulo = 'Exemplo de Componente Angular';
}
```

Acima, definimos os metadados do *decorator* `@Component`. O **selector** define o nome da tag HTML personalizada que será utilizada para incorporar o componente em um *template*. O valor do **selector** é uma string que segue a convenção de naming do Angular, geralmente no formato `app-nome-do-componente`.

O parâmetro **templateUrl** especifica o caminho do arquivo HTML que contém o *template* do componente. O *template* define a estrutura visual e a interface de usuário do componente, utilizando HTML, *bindings* de dados e diretivas Angular.

Por fim, o **styleUrls** é uma lista de URLs de arquivos CSS que contém os estilos aplicados ao componente. Essa propriedade permite encapsular estilos específicos para o componente, evitando que eles vazem para outros componentes ou partes da aplicação.

Em seguida, exportamos uma classe com a lógica do componente. Essa lógica envolve um código TypeScript que irá interagir com o *template* e o estilo para criar uma parte de aplicação web, a ser renderizada no navegador do usuário.

(CEBRASPE/TBG/2023) import { Component } from '@angular/core';

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  document.write('TBG!');
}
```

Em relação ao código precedente, julgue o item seguinte.

Considerando-se que o código apresentado é voltado para o Angular com Typescript, é correto afirmar que `@Component` é um método da classe de `AppComponent`.

Comentários:

O item está errado. O `@Component` não é um método, e sim um *decorator* responsável por estabelecer os componentes do Angular. (Gabarito: Errado)

O que você precisa entender é que, na forma mais usual de se trabalhar com o Angular, cada componente é tratado em um arquivo TypeScript separado. Esse arquivo conterá a declaração do componente, a sua lógica e uma forma de exportar o conteúdo dele. Em um arquivo central, chamado de **Componente Pai**, juntamos todos os componentes de forma a montar nossa aplicação, quase como peças de lego.

Vamos para um exemplo prático? Vamos criar um botão. Iniciamos com o arquivo TypeScript que é responsável por criar o componente.

TypeScript

```
import { Component, EventEmitter, Output } from '@angular/core';

@Component({
  selector: 'app-button',
  templateUrl: './button.component.html',
  styleUrls: ['./button.component.css']
})
export class ButtonComponent {
  // Propriedade para armazenar o texto do botão
  buttonText: string = 'Clique-me';

  // Evento que será emitido quando o botão for clicado
  @Output() buttonClick = new EventEmitter<void>();

  // Método para emitir o evento quando o botão é clicado
  onButtonClick() {
    this.buttonClick.emit();
  }
}
```

Passamos os parâmetros gerais e a lógica do botão. Agora precisamos de dois arquivos, um HTML e um CSS para criar a parte visual do botão, que será renderizada na aplicação. Vamos criá-los.

HTML

```
<button (click)="onButtonClick()">{{ buttonText }}</button>
```

Veja que temos uma expressão Angular (caracterizado pelas chaves duplas `{{ }}`), que irá inserir o valor que definimos lá no componente.

CSS

```
button {  
  padding: 10px 20px;  
  font-size: 16px;  
  color: white;  
  background-color: #007bff;  
  border: none;  
  border-radius: 4px;  
  cursor: pointer;  
  transition: background-color 0.3s;  
}  
  
button:hover {  
  background-color: #0056b3;  
}
```

Por fim, teremos um arquivo chamado de `app.module.ts`, que conterá a montagem da nossa aplicação, com os componentes criados.

TypeScript

```
import { NgModule } from '@angular/core';  
import { BrowserModule } from '@angular/platform-browser';  
import { AppComponent } from './app.component';  
import { ButtonComponent } from './button/button.component';  
  
@NgModule({  
  declarations: [  
    AppComponent,  
    ButtonComponent  
  ],  
  imports: [  
    BrowserModule  
  ],  
  providers: [],  
  bootstrap: [AppComponent]  
})  
export class AppModule { }
```

O componente pai recebe um tipo de Decorator especial: o **NgModule**. Ele é responsável por marcar uma classe como um módulo Angular e fornecer metadados que dizem ao Angular como compilar e executar esse módulo. Módulos são uma parte fundamental da arquitetura Angular, permitindo que você organize a aplicação em blocos coesos de funcionalidade.

(PR4 UFRJ/UFRJ/2018) Considerando o código a seguir, implementado em Angular, versão 4.4.0, responda a questão.

```
1. import { Component } from '@angular/core';
2.
3. @Component({
4.   selector: 'alunos',
5.   template: `
6.     <h1>TITULO</h1>
7.     <h2>Nome do Aluno: ALUNO</h2>
8.   `
9. })
10. export class AlunoComponent {
11.   title = 'Alunos inscritos';
12.   aluno = 'Nome do Aluno';
13. }
```

Para que o componente em questão apresente no navegador, na tag da linha 6, na declaração TITULO, o texto: "Alunos inscritos" e na tag da linha 7, na declaração ALUNO, o texto: "Nome do Aluno", é necessário alterar as declarações indicadas, respectivamente, por:

- a) this.title e this.aluno
- b) {{title}} e {{aluno}}
- c) {{this.title}} e {{this.aluno}}
- d) title e aluno
- e) \$this->title e \$this->aluno

Comentários:

Questão interessante. Trata-se de um componente Angular, onde definimos o *template* no próprio código - composto por dois títulos (h1 e h2). Precisamos substituir os valores alocados dentro do código HTML para que essa atualização seja dinâmica. Para isso, basta implementarmos as expressões Angular, que são caracterizadas pelo duplo par de chaves {{ }}. Portanto, colocaremos {{title}} e {{aluno}} para que a atualização se dê de forma dinâmica. (Gabarito: Letra B)

Aqui é interessante saber os principais decoradores que acompanham o Angular. Você já viu o @Component e @NgModule - vamos ver os demais.

DECORADOR	DESCRIÇÃO
@Injectable	Usado para definir uma classe como um serviço que pode ser injetado em componentes e outros serviços.
@Directive	Marca uma classe como uma diretiva personalizada. Diretivas permitem modificar o comportamento de elementos DOM.
@Pipe	Define uma classe como um <i>pipe</i> que pode ser usado para transformar dados em <i>templates</i> Angular.
@Input	Marca uma propriedade como entrada para receber dados de um componente pai.

@Output	Marca uma propriedade como saída para emitir eventos para um componente pai.
@HostListener	Escuta eventos no host do componente ou diretiva.
@HostBinding	Vincula uma propriedade de classe a uma propriedade do host.
@ViewChild	Acessa um elemento DOM ou componente filho dentro do <i>template</i> do componente.
@ContentChild	Acessa um elemento DOM ou componente filho dentro do conteúdo projetado no componente.

Um último conteúdo interessante de saber sobre os componentes é o seu **ciclo de vida**. Cada componente tem um ciclo de vida, que inicia quando o Angular inicializa o componente e o renderiza e continua, detectando mudanças, inserções de dados, e outras interações, fazendo atualizações tanto na visualização, quanto no componente. O Angular age basicamente como o Controller, numa arquitetura MVC. Esse ciclo de vida termina quando Angular destrói a instância do componente e remove seu *template* renderizado do DOM.

Para acessar e interagir com os diferentes momentos do ciclo de vida usam-se **hooks**. Eles são métodos especiais que fornecem aos desenvolvedores a capacidade de interagir com diferentes fases do ciclo de vida de um componente (Angular) ou diretiva (AngularJS). Eles permitem que você execute código personalizado em momentos específicos durante a criação, atualização e destruição de componentes e diretivas.

Novamente chegamos numa lista de elementos que você precisa saber. Como é de praxe, vamos para a tabela-resumo com os principais *hooks* de ciclo de vida. Seu uso é importante pois permite que os componentes interajam com mudanças no ciclo de vida - por exemplo, o `ngOnChanges()` permite que o componente performe alguma ação após alguma mudança de estado.

HOOK	DESCRIÇÃO
ngOnChanges	É chamado sempre que uma ou mais propriedades de entrada vinculadas a dados (@Input) de um componente ou diretiva são alteradas.
ngOnInit	É chamado uma vez, após a primeira execução de ngOnChanges. Ideal para inicialização de lógica de componente.
ngDoCheck	É chamado durante cada ciclo de verificação de mudanças.
ngAfterContentInit	É chamado uma vez, após a inserção do conteúdo projetado no componente.
ngAfterContentChecked	É chamado após cada verificação de mudanças no conteúdo projetado.
ngAfterViewInit	É chamado uma vez, após a inicialização das visualizações do componente e suas visualizações filhas.
ngAfterViewChecked	É chamado após cada verificação de mudanças nas visualizações do componente e suas visualizações filhas.
ngOnDestroy	É chamado uma vez, antes do Angular destruir o componente ou diretiva.

Veja um exemplo de implementação do *hook* `ngOnDestroy`.

TypeScript

```
import { Component, OnDestroy } from '@angular/core';

@Component({
  selector: 'app-exemplo',
  template: `<p>Componente será destruído</p>`
})
export class ExemploComponent implements OnDestroy {
  ngOnDestroy() {
    console.log('ngOnDestroy: O componente está sendo destruído');
  }
}
```

Sua aplicação é interessante por diversos motivos. Nesse caso em específico, temos um *log* no console que indica a destruição do componente, excelente para acompanhamento do ciclo de vida dos componentes.

Angular Elements

O **Angular Elements** é um conjunto de componentes customizados, também chamados de Componentes Web, destinados a definir elementos HTML de uma forma agnóstica a *frameworks*, tirando a dependência pura do Angular. De forma geral, o Angular Elements fornece uma biblioteca que permite **empacotar os componentes Angular em componentes personalizados**, que **podem ser utilizados em qualquer aplicação web**, independente do *framework*.

Elementos personalizados são uma tecnologia da Web baseada em APIs de Componentes Web, que permitem a criação de novos elementos HTML. Eles fazem parte das especificações dos Componentes Web, que também incluem Shadow DOM e HTML Templates. Com elementos personalizados, desenvolvedores podem definir novos tipos de elementos e seu comportamento, que podem ser utilizados como qualquer elemento HTML padrão.

O **Shadow DOM** é uma tecnologia de encapsulamento utilizada em componentes web para criar um sub-árvore de DOM isolada e protegida dentro de um elemento. Ele permite que o **componente mantenha seu estilo e estrutura internos sem ser afetado pelo estilo e estrutura do documento principal**, garantindo que as regras de CSS e o comportamento do JavaScript dentro do Shadow DOM não interfiram no resto da página e vice-versa.

A criação de componentes customizados usualmente segue o seguinte fluxo:

- 1) **Empacotamento:** o componente Angular é empacotado junto com as dependências necessárias, como o Angular core.
- 2) **Conversão:** o componente é convertido em um elemento personalizado, que pode ser registrado no DOM do navegador.

Para fazer a conversão, implementamos a interface `NgElementConstructor`, que cria uma classe construtora capaz de fazer uma conversão do componente em uma classe customizada. Ele utiliza a função nativa dos navegadores `customElement.define()` para registrar o construtor configurado, e associar uma tag customizada a esse elemento, usando o registro da API `CustomElementRegistry` do navegador.

Em termos práticos, utilizamos a função `createCustomElement()` dentro do código, definindo qual componente será convertido e outros parâmetros de código. Posteriormente, devemos registrar o elemento com a API, usando o comando `customElements.define()`. Veja um exemplo onde criamos um elemento customizado.

TypeScript

```
import { createCustomElement } from '@angular/elements';  
import { Injector } from '@angular/core';  
import { MyComponent } from './my-component';  
  
const myElement = createCustomElement(MyComponent, { injector:  
injector });  
customElements.define('my-element', myElement);
```

No código acima:

- Criamos a `const myElement`, que irá transformar o componente `MyComponent` em um elemento customizado. Além disso, passamos um argumento de objeto de configuração, incluindo o `injector` do Angular, permitindo a injeção de dependências.
- Registramos o `myElement` como elemento dentro do navegador a partir do `customElements.define()`, que envia para a API qual é o elemento e qual o seu nome - no caso, o nome será `my-element`.

(FGV/TJ TO/2022) Com relação à plataforma Angular, é possível converter um componente numa classe que pode ser registrada no browser e incluída no DOM.

A função que permite essa operação é denominada:

- a) `addElement()`
- b) `createAndRegister()`
- c) `createCustomElement()`
- d) `putElement()`
- e) `registerElement()`

Comentários:

Questão tranquila. A função que permite convertermos um componente Angular em uma classe (elemento) que pode ser registrado no DOM e ser utilizado por outros *frameworks* é a função `createCustomElement()`.
(Gabarito: Letra C)

Angular CLI

O **Angular CLI** (*Command Line Interface*) é uma interface de linha de comando responsável por inicializar, desenvolver, gerenciar e manter aplicações Angular diretamente de terminais de comando. Basicamente, temos um conjunto de comandos que fazem algumas atividades do desenvolvimento a partir de um *console*.

Os comandos da Angular CLI seguem a seguinte sintaxe base:

```
ng <argumentos> [opções]
```

Apenas um ponto de atenção:

- A maior parte dos comandos possui um *alias*, um apelido
- Opções são elencadas por um prefixo `--`
- Apelidos das opções são elencados por um prefixo `-`

Aqui não tem segredo - os comandos CLI são bem diretos e, muitas vezes, autoexplicativos. Vamos recorrer ao nosso recurso preferido: as tabelas-resumo.



DECORADOR	DESCRIÇÃO
ng add	Adiciona bibliotecas e pacotes ao seu projeto Angular, configurando automaticamente os arquivos necessários.
ng analytics	Gera relatórios e configura o uso de dados de análise para o projeto Angular.
ng build	Compila a aplicação Angular para distribuição, criando um diretório com os arquivos estáticos resultantes.
ng cache	Gerencia a configuração e o estado do cache para o build Angular.
ng completion	Configura a conclusão automática de comandos do Angular CLI no terminal.
ng config	Lê ou define configurações do Angular.
ng deploy	Implanta a aplicação em um serviço de hospedagem configurado.
ng e2e	Executa testes de ponta a ponta (end-to-end) para a aplicação Angular.
ng extract-i18n	Extraí mensagens de texto para internacionalização (i18n) da aplicação.
ng generate	Gera e modifica arquivos do projeto Angular, como componentes, serviços, módulos, entre outros.

<code>ng lint</code>	Executa verificações de qualidade de código utilizando ferramentas de linting configuradas.
<code>ng new</code>	Cria um novo projeto Angular com uma estrutura inicial.
<code>ng serve</code>	Inicia um servidor de desenvolvimento, recompilando a aplicação automaticamente a cada mudança.
<code>ng test</code>	Executa testes unitários para a aplicação Angular.
<code>ng update</code>	Atualiza as dependências e o código do projeto Angular para as versões mais recentes.
<code>ng version</code>	Exibe a versão atual do Angular CLI e dos pacotes Angular utilizados no projeto.

(PR4 UFRJ/UFRJ/2023) O angular é um framework de desenvolvimento baseado em TypeScript que inclui uma estrutura baseada em componentes, que são trechos de código reutilizáveis escritos em HTML, TypeScript e CSS, para criar aplicativos da Web. Utilizando o angular cli, assinale a alternativa correta para criar um novo componente.

- a) `ng build component <component-name>`
- b) `ng create component <component-name>`
- c) `ng new component <component-name>`
- d) `ng generate component <component-name>`
- e) `ng append component <component-name>`

Comentários:

O comando CLI responsável por criar componentes é o comando `ng generate`. Damos o comando e especificamos o tipo de objeto que queremos gerar - no caso, para um componente, usamos o comando `ng generate component <nome-do-componente>`. (Gabarito: Letra D)

QUESTÕES COMENTADAS

01. (CESGRANRIO/IPEA/2024) Admita que a área de desenvolvimento de softwares do Ipea pretenda utilizar uma linguagem usando um framework para TypeScript, de modo a construir uma aplicação web de uma só página para mostrar dados escolares de municípios que possuem menos de 5.000 habitantes. Nessa situação, a interação e a navegação entre as sessões da página ocorrerão de modo a não ser necessário o recarregamento da página a cada alteração.

Um framework de uso adequado para essa situação é o

- a) Scrum
- b) Django
- c) Express
- d) Angular
- e) ASPNET

Comentários:

Das alternativas apresentadas, a única que aponta um *framework* que utiliza o TypeScript é a letra D, com o Angular. Quanto aos demais:

- Scrum: metodologia de desenvolvimento de software;
- Django: framework web de Python;
- Express: framework para aplicações web com NodeJS;
- ASP.net: plataforma de desenvolvimento web para Microsoft;

Portanto, correta a letra D.

Gabarito: Letra D

02. (FUNCERN/IF RN/2024) No Angular, o Two-Way Binding permite uma comunicação bidirecional entre a classe TypeScript do componente e o template (interface do usuário). Considerando essa afirmação, sobre a configuração de Two-Way Binding em um componente Angular, é correto afirmar que o Two-Way Binding

- a) é configurado usando a diretiva [value] em combinação com o evento (change).
- b) é indicado apenas para componentes de entrada de dados, como campos de texto e caixas de seleção.
- c) é configurado apenas na classe TypeScript do componente e não requer nenhuma alteração no modelo da interface do usuário.

- d) é usado para atualizar automaticamente o modelo da interface do usuário com base nas mudanças na classe TypeScript do componente.

Comentários:

O *two-way data binding* é uma forma de realizar uma conexão entre o valor armazenado no Angular, com o valor exibido na interface Web. Das alternativas apresentadas, a letra D apresenta a melhor explicação para a abordagem.

Gabarito: Letra D

03. (FUNDATEC/PROCERGS/2023) Qual das seguintes alternativas NÃO é uma aplicação comum do framework Angular no desenvolvimento de interfaces de front-end?

- a) Implementação de rotas para navegação entre páginas.
- b) Utilização de diretivas para manipulação do DOM.
- c) Realização de requisições HTTP para acesso a APIs REST.
- d) Criação de animações e transições de página.
- e) Integração direta com banco de dados.

Comentários:

Vamos analisar cada alternativa - procurando a incorreta.

- a) Certo. O *route* é uma das formas de implementarmos múltiplas páginas dentro do Angular.
- b) Certo. As diretivas são formas de manipular o DOM/HTML, integrando o Angular diretamente a ele.
- c) Certo. Podemos implementar requisições HTTP no Angular.
- d) Certo. Esse é justamente um dos objetivos do Angular - implementar interfaces de usuário, animações, transições, entre outros.
- e) Errado. O Angular não se integra diretamente com bancos de dados. É necessário uma API intermediária para a comunicação, usualmente uma API RESTful.

Portanto, INCORRETA a letra E.

Gabarito: Letra E

04. (FUNDATEC/PROCERGS/2023) Tendo em vista as estruturas de template do Angular e suas definições, relacione a Coluna 1 ' Coluna 2.

Coluna 1

1. interpolation
2. property binding
3. event binding
4. class and style binding
5. template variables

Coluna 2

() <input #campo type="text"><button (click)="salvar(campo.value)">Salvar</button>
()
() <h1>{{ titulo }}</h1>
()
() <div [class.ativo]="isAtivo" [style.backgroundColor]="cor"></div>
() <button (click)="botaoClicado()">Clique aqui</button>

A ordem correta de preenchimento dos parênteses, de cima para baixo, é:

- a) 1 – 2 – 3 – 5 – 4.
- b) 5 – 1 – 2 – 4 – 3.
- c) 5 – 4 – 1 – 3 – 2.
- d) 4 – 2 – 5 – 3 – 1.
- e) 3 – 5 – 4 – 2 – 1.

Comentários:

Vamos fazer a associação.

() <input #campo type="text"><button (click)="salvar(campo.value)">Salvar</button>
(5) Esse item aponta o uso de *template variables*, caracterizado por um símbolo hash #.

() <h1>{{ titulo }}</h1>
(1) O item expõe o uso de uma expressão Angular, abordagem também chamada de interpolação.

()
(2) Temos um *property binding*, onde vinculamos uma propriedade a uma variável Angular, usando um par de colchetes [].

() <div [class.ativo]="isAtivo" [style.backgroundColor]="cor"></div>
(4) Esse é um exemplo de *class binding*, vinculando classes CSS a variáveis Angular.

() <button (click)="botaoClicado()">Clique aqui</button>
(3) Por fim, temos a vinculação de um evento ao Angular - o *event binding*.

Ficamos com 5-1-2-4-3 - correta a letra B.

Gabarito: Letra B

05. (CEBRASPE/TBG/2023) import { Component } from '@angular/core';

```
@Component({  
  selector: 'app-root',  
  templateUrl: './app.component.html',  
  styleUrls: ['./app.component.css']  
})  
export class AppComponent {  
  document.write ('TBG!');  
}
```

Em relação ao código precedente, julgue o item seguinte.

Considerando-se que o código apresentado é voltado para o Angular com Typescript, é correto afirmar que @Component é um método da classe de AppComponent.

Comentários:

Cuidado! Elementos demarcados com @ são *decorators*, e não métodos de classe. Portanto, incorreta a afirmativa.

Gabarito: Errado

06. (PR4/UFRJ/2023) O angular é um framework de desenvolvimento baseado em TypeScript que inclui uma estrutura baseada em componentes, que são trechos de código reutilizáveis escritos em HTML, TypeScript e CSS, para criar aplicativos da Web. Utilizando o angular cli, assinale a alternativa correta para criar um novo componente.

- a) ng build component <component-name>
- b) ng create component <component-name>
- c) ng new component <component-name>
- d) ng generate component <component-name>
- e) ng append component <component-name>

Comentários:

Questão bem direta: para criarmos objetos no Angular CLI, utilizamos o comando `ng generate`. No caso de um componente, usamos `ng generate component <nome>`.

Gabarito: Letra D

07. (FCC/TJ BA/2023) Em condições ideais, na interface de linha de comando, um Analista foi até o diretório raiz de um projeto Angular e utilizou o comando `ng lint` para

- a) iniciar o servidor de desenvolvimento e disponibilizar o aplicativo em um navegador web.
- b) gerar um novo componente chamado lint na aplicação Angular.
- c) atualizar a versão do Angular e suas dependências para a versão mais recente.
- d) verificar o código-fonte em busca de problemas de formatação e qualidade de código.
- e) adicionar um pacote de terceiro chamado lint a aplicação Angular.

Comentários:

O comando `ng lint` é uma ferramenta de “controle de qualidade”. Ele vasculha o código procurando problemas de formatação e avaliando a qualidade do código. A alternativa que melhor o explica é a letra D.

Gabarito: Letra D

08. (FCC/TRT 15/2023) Para envolver vários componentes Angular Material e aplicar estilos comuns a campos de texto, como a linha inferior, rótulo flutuante e mensagens de dica, um Técnico deve utilizar o componente

- a) `<form-angular-material>`
- b) `<mat-form-field>`
- c) `<m:formField>`
- d) `<mat: formField>`
- e) `<m:form>`

Comentários:

A questão explora conhecimentos do Angular Material, um conjunto de componentes de interface de usuário. Especificamente nessa questão, temos o `<mat-form-field>`, um componente usado para agrupar vários componentes do Angular Material e aplicar estilos de campo de texto comuns, como sublinhado, rótulo flutuante e mensagens de dica.

Gabarito: Letra B

09. (CETREDE/UFC/2022) Sobre os Frameworks de desenvolvimento de software mais populares do mercado, é correto afirmar que:

- a) o Angular é uma plataforma e framework para a construção de aplicações single-page do lado cliente usando HTML e TypeScript.
- b) o React é uma biblioteca JavaScript baseada em componentes para a construção de aplicações back-end com acesso direto à SGBD.
- c) o Node.js é um ambiente de desenvolvimento JavaScript apenas para Frontend.
- d) o Mongoose é uma biblioteca para a modelagem de dados de aplicações desenvolvidas em React para o MongoDB.

Comentários:

Vamos analisar cada alternativa.

- a) Certo. O Angular é de fato uma plataforma e framework desenvolvido pela Google, projetado para a construção de aplicações single-page (SPA) do lado cliente, utilizando HTML e TypeScript. Ele fornece uma arquitetura robusta e uma série de ferramentas para desenvolvimento front-end.
- b) Errado. O React é uma biblioteca JavaScript desenvolvida pelo Facebook, usada principalmente para a construção de interfaces de usuário (UI) baseadas em componentes. Ele é focado no desenvolvimento front-end e não é utilizado diretamente para aplicações back-end ou para acesso a Sistemas de Gerenciamento de Banco de Dados (SGBD).
- c) Errado. O Node.js é um ambiente de execução JavaScript que permite a execução de código JavaScript no lado do servidor (back-end). Ele é amplamente utilizado para o desenvolvimento de aplicações server-side, permitindo a construção de servidores web e APIs. Node.js não é restrito ao desenvolvimento front-end.
- d) Errado. O Mongoose é uma biblioteca JavaScript que fornece uma solução baseada em esquemas para a modelagem de dados em aplicações Node.js que utilizam o banco de dados MongoDB. Ele facilita a interação com o MongoDB ao fornecer uma camada de abstração sobre o banco de dados, mas não é específico para aplicações desenvolvidas com React.

Portanto, correta a letra A.

Gabarito: Letra A

10. (FGV/TJ TO/2022) Com relação à plataforma Angular, é possível converter um componente numa classe que pode ser registrada no browser e incluída no DOM.

A função que permite essa operação é denominada:

- a) `addElement()`
- b) `createAndRegister()`
- c) `createCustomElement()`

- d) putElement()
- e) registerElement()

Comentários:

Para podermos registrar elementos no DOM e permitir integração com outros Frameworks, criando um componente manipulável, a função a ser utilizada é a `createCustomElement()`.

Gabarito: Letra C

11. (FCC/TRT 22/2022) Em um site desenvolvido utilizando o framework Angular, um Técnico observou no interior da tag `<body>` do arquivo `index.html` um elemento Angular definido no arquivo `app/app.component.ts` que será o primeiro componente carregado e o contêiner para os outros componentes. Trata-se do elemento

- a) `<angular-root>`
- b) `<render>`
- c) `<document-root>`
- d) `<app-root>`
- e) `<main-app>`

Comentários:

A questão cobra conhecimentos acerca da estruturação do arquivo que “monta” a aplicação do Angular. Nesse caso, o elemento que irá ser carregado primeiramente, por conter as dependências e todas as definições da página, é o `<app-root>`.

Gabarito: Letra D

12. (IDECAN/TJ PI/2022) O Angular é uma grande plataforma de desenvolvimento para aplicações. Com ele podemos desenvolver aplicações web e também aplicações mobile. Ela conta com templates, componentes, módulos, serviços e diversas ferramentas de infraestrutura. Tudo isso foi criado para tornar o desenvolvimento de aplicações mais simples e rápido. A partir da sua versão 9 o Angular passou a contar, por padrão, com um novo compilador.

Ele é responsável pela renderização de conteúdo e sua chegada trouxe mais performance para o ambiente de desenvolvimento Angular. Marque a alternativa que indica, corretamente, o nome deste compilador.

- a) V8
- b) Ivy
- c) Gecko

- d) ECMA
- e) Chromium

Comentários:

Questão meio não trivial. Mas o compilador padrão do Angular é o IVY.

Gabarito: Letra B

13. (FCM/IF AM/2022) Com relação à diretiva *ngFor do Angular é INCORRETO afirmar que

- a) é uma estrutura de repetição.
- b) não afeta a estrutura do DOM.
- c) serve para estender o poder do HTML.
- d) é uma diretiva estrutural.
- e) não é uma diretiva de atributo.

Comentários:

A diretiva ngFor implementa o bloco de repetição do JavaScript *for*. Vamos analisar as alternativas, procurando a INCORRETA.

- a) Certo. O *for* é uma estrutura de repetição.
- b) Errado. As diretivas do Angular alteram o DOM.
- c) Certo. Esse é o objetivo das diretivas do Angular.
- d) Certo. O ngFor é uma diretiva estrutural, pois trabalha com a estrutura do componente.
- e) Certo. O ngFor é uma diretiva estrutural.

Portanto, a alternativa incorreta é a letra B.

Gabarito: Letra B

14. (FCM/IF AM/2022) O desenvolvimento de aplicativos móveis depende da instalação de algumas plataformas, editores de código e frameworks, além de pacotes e módulos. Há diversas ferramentas gratuitas que podem ser utilizadas pelos programadores durante o desenvolvimento de suas aplicações.

Entre elas existe um framework e plataforma para desenvolvimento de interfaces de aplicações móveis e Web baseadas em JavaScript, CSS ou HTML. É uma ferramenta mantida pela equipe do Google que permite o desenvolvimento front-end baseado em Typescript.

Esta ferramenta é o

- a) Ionic.
- b) Angular.
- c) Node.js.
- d) Node Package Manager (Npm).
- e) Apache Cordova.

Comentários:

Das alternativas apresentadas, apenas o Angular permite trabalhar com TypeScript para desenvolvimento front-end. Quanto às demais:

- Ionic: framework de desenvolvimento móvel híbrido
- Node.js: servidor local
- NPM: ferramenta de gerenciamento dos pacotes do Node
- Apache Cordova: framework de desenvolvimento móvel com uma abordagem um pouco diferente, englobando o código HTML+JS em um container para executar em dispositivos móveis.

Portanto, correta a letra B.

Gabarito: Letra B

15. (FCM/IF AM/2022) O Angular tem como objetivo facilitar o desenvolvimento de aplicações para a web e dispositivos móveis.

Este framework é baseado em qual linguagem?

- a) CSS
- b) HTML
- c) JavaScript
- d) C/C++
- e) Java

Comentários:

Se levarmos ao pé da letra, o Angular é baseado em TypeScript - mas não teríamos resposta. Então, numa interpretação mais ampla, podemos dizer que o Angular se baseia em JavaScript, o que é 100% verdade se estivermos abordando as versões iniciais, do AngularJS.

Gabarito: Letra C

16. (FCM/UFJF/2019) Uma aplicação em Angular é feita por meio de componentes. O par de metadados obrigatórios no decorador @Component para a definição de uma classe como componente é

- a) view e filter.
- b) tag e constructor.
- c) styleUrls e provider.
- d) HTMLpage e provider.
- e) selector e templateUrl.

Comentários:

Para definir uma classe como um componente em uma aplicação Angular, é necessário utilizar o decorador @Component. Dentro deste decorador, dois metadados são obrigatórios para a definição de um componente: selector e templateUrl ou template. O selector define um seletor CSS, e o templateUrl, ou somente template, define o caminho para o HTML do componente.

Gabarito: Letra E

17. (FCM/UFJF/2019) No Framework Angular, o tipo de envio ou sincronização de dados que permite atuar dentro da classe de um componente é

- - a) interpolation.
 - b) event binding.
 - c) property binding.
 - d) two-way data binding.
 - e) one-way data binding.

Comentários:

Quando falamos em sincronização de dados, isso é, sincronizar as duas pontas dos dados - sua view e seu armazenamnto -, falamos de *two-way binding*. Esse deveria ser o gabarito da questão - mas, por algum motivo, a banca considerou Event Binding como gabarito para ela, erroneamente.

Gabarito da Banca: Letra B
Gabarito do Professor: Letra D

18. (PR4 UFRJ/UFRJ/2018) Considerando o código a seguir, implementado em Angular, versão 4.4.0, responda a questão.

```
1. import { Component } from '@angular/core';
```

```
2.  
3.  @Component({  
4.    selector: 'alunos',  
5.    template: `  
6.      <h1>TITULO</h1>  
7.      <h2>Nome do Aluno: ALUNO</h2>  
8.    `,  
9.  })  
10.  export class AlunoComponent {  
11.    title = 'Alunos inscritos';  
12.    aluno = 'Nome do Aluno';  
13.  }
```

Para que o componente em questão apresente no navegador, na tag da linha 6, na declaração TITULO, o texto: "Alunos inscritos" e na tag da linha 7, na declaração ALUNO, o texto: "Nome do Aluno", é necessário alterar as declarações indicadas, respectivamente, por:

- a) this.title e this.aluno
- b) {{title}} e {{aluno}}
- c) {{this.title}} e {{this.aluno}}
- d) title e aluno
- e) \$this->title e \$this->aluno

Comentários:

Questão interessante. Trata-se de um componente Angular, onde definimos o *template* no próprio código - composto por dois títulos (h1 e h2). Precisamos substituir os valores alocados dentro do código HTML para que essa atualização seja dinâmica. Para isso, basta implementarmos as expressões Angular, que são caracterizadas pelo duplo par de chaves {{ }}. Portanto, colocaremos {{title}} e {{aluno}} para que a atualização se dê de forma dinâmica.

Gabarito: Letra B

19. (PR4 UFRJ/UFRJ/2018) Considerando o código a seguir, implementado em Angular, versão 4.4.0, responda a questão.

```
1. import { Component } from '@angular/core';  
2.  
3.  @Component({  
4.    selector: 'alunos',  
5.    template: `  
6.      <h1>TITULO</h1>
```

```
7.    <h2>Nome do Aluno: ALUNO</h2>
8.    `
9.  })
10.  export class AlunoComponent {
11.    title = 'Alunos inscritos';
12.    aluno = 'Nome do Aluno';
13.  }
```

Assinale a alternativa que apresenta a tag necessária para declarar o uso deste componente em uma página html considerando um projeto Angular.

- a) <AlunosComponent> e </AlunosComponent>
- b) <alunos> e </alunos>
- c) <Component> e </Component>
- d) <app> e </app>
- e) <selector> e </selector>

Comentários:

A tag que irá criar o componente é definida no nosso selector. Portanto, teremos uma tag <alunos> e </alunos> para invocar o componente.

Gabarito: Letra B

20. (FCC/DPE AM/2018) Considere a página web abaixo que utiliza Angular.

```
<!DOCTYPE html>
<html>
  <head>
    <script src="angular.min.js"></script>
  </head>
  <body>
    <div ng-app="">
      <form>
        <p>Nome: <input type="text" ng-model="nome"></p>
      </form>
    </div>
  </body>
</html>
```

Considere os comandos:

- I. `<p ng-bind="nome"></p>`
- II. `<p>{{nome}}</p>`
- III. `<p ng-print="nome"></p>`
- IV. `<p>{$nome}</p>`

Para que o que for digitado no campo nome seja exibido simultaneamente em um parágrafo, na lacuna I podem ser utilizados os comandos que constam APENAS nos itens

- a) III e IV.
- b) I e III.
- c) II e IV.
- d) I e IV.
- e) I e II.

Comentários:

Queremos alguma forma de implementarmos um parágrafo `<p>` que exiba o nome digitado no *input*. Para isso, precisamos implementar a variável *nome*, definido no *ng-model*, de alguma forma. Vamos analisar cada item:

- I. Certo. O *ng-bind* irá associar o valor do conteúdo de `<p>` à variável *nome*, tendo o resultado que queremos.
- II. Certo. Aqui usamos uma interpolação, com expressões Angular, para inserir o nome.
- III. Errado. Não existe a diretiva *ng-print*.
- IV. Errado. A interpolação deve ser feita com `{{ }}`, e não `{ $ }`.

Portanto, corretos os itens I e II.

Gabarito: Letra E

21. (Instituto AOCP/PRODEB/2018) Sobre o Framework de desenvolvimento estrutural Angular, analise as assertivas e assinale a alternativa que aponta a(s) correta(s).

- I. O Framework Estrutural Angular é uma ótima opção para criar SPA (Single Page Applications).
 - II. O Framework Estrutural Angular foi desenvolvido para criar aplicações web dinâmicas.
 - III. O Framework Estrutural Angular foi desenvolvido para criar banco de dados relacionais.
 - IV. O Framework Estrutural Angular é utilizado para a configuração de redes IPV6.
- a) Apenas II.
 - b) Apenas I, III e IV.
 - c) Apenas I e II.
 - d) Apenas III e IV.

e) Apenas II e III.

Comentários:

Vamos analisar cada item.

I. Certo. O Angular é especializado em criar SPAs, sendo uma ótima opção.

II. Certo. O Angular implementa comunicações assíncronas, permitindo aplicações dinâmicas.

III e IV. Errado. O *framework* cria apps web, não bancos de dado ou configurações de redes.

Portanto, corretos os itens I e II.

Gabarito: Letra C

22. (QUADRIX/CRM PR/2018) No que se refere a ambientes para desenvolvimento de sistemas, julgue o item a seguir.

Angular 2 é um IDE com suporte para linguagens Java e JavaScript e versões para diversos sistemas operacionais.

Comentários:

O Angular 2 foi a primeira versão do *framework* a implementar o TypeScript. Ele não é uma IDE, é uma biblioteca de *stack web* para criação de SPAs, usando TypeScript.

Gabarito: Errado

LISTA DE QUESTÕES

01. (CESGRANRIO/IPEA/2024) Admita que a área de desenvolvimento de softwares do Ipea pretenda utilizar uma linguagem usando um framework para TypeScript, de modo a construir uma aplicação web de uma só página para mostrar dados escolares de municípios que possuem menos de 5.000 habitantes. Nessa situação, a interação e a navegação entre as sessões da página ocorrerão de modo a não ser necessário o recarregamento da página a cada alteração.

Um framework de uso adequado para essa situação é o

- a) Scrum
- b) Django
- c) Express
- d) Angular
- e) ASPNET

02. (FUNCERN/IF RN/2024) No Angular, o Two-Way Binding permite uma comunicação bidirecional entre a classe TypeScript do componente e o template (interface do usuário). Considerando essa afirmação, sobre a configuração de Two-Way Binding em um componente Angular, é correto afirmar que o Two-Way Binding

- a) é configurado usando a diretiva [value] em combinação com o evento (change).
- b) é indicado apenas para componentes de entrada de dados, como campos de texto e caixas de seleção.
- c) é configurado apenas na classe TypeScript do componente e não requer nenhuma alteração no modelo da interface do usuário.
- d) é usado para atualizar automaticamente o modelo da interface do usuário com base nas mudanças na classe TypeScript do componente.

03. (FUNDATEC/PROCERGS/2023) Qual das seguintes alternativas NÃO é uma aplicação comum do framework Angular no desenvolvimento de interfaces de front-end?

- a) Implementação de rotas para navegação entre páginas.
- b) Utilização de diretivas para manipulação do DOM.
- c) Realização de requisições HTTP para acesso a APIs REST.
- d) Criação de animações e transições de página.
- e) Integração direta com banco de dados.

04. (FUNDATEC/PROCERGS/2023) Tendo em vista as estruturas de template do Angular e suas definições, relacione a Coluna 1 ' Coluna 2.

Coluna 1

1. interpolation
2. property binding
3. event binding
4. class and style binding
5. template variables

Coluna 2

- () <input #campo type="text"><button (click)="salvar(campo.value)">Salvar</button>
- () <h1>{{ titulo }}</h1>
- ()
- () <div [class.ativo]="isAtivo" [style.backgroundColor]="cor"></div>
- () <button (click)="botaoClicado()">Clique aqui</button>

A ordem correta de preenchimento dos parênteses, de cima para baixo, é:

- a) 1 – 2 – 3 – 5 – 4.
- b) 5 – 1 – 2 – 4 – 3.
- c) 5 – 4 – 1 – 3 – 2.
- d) 4 – 2 – 5 – 3 – 1.
- e) 3 – 5 – 4 – 2 – 1.

05. (CEBRASPE/TBG/2023) import { Component } from '@angular/core';

```
@Component({  
  selector: 'app-root',  
  templateUrl: './app.component.html',  
  styleUrls: ['./app.component.css']  
})  
export class AppComponent {  
  document.write ('TBG!');  
}
```

Em relação ao código precedente, julgue o item seguinte.

Considerando-se que o código apresentado é voltado para o Angular com Typescript, é correto afirmar que @Component é um método da classe de AppComponent.

06. (PR4/UFRJ/2023) O angular é um framework de desenvolvimento baseado em TypeScript que inclui uma estrutura baseada em componentes, que são trechos de código reutilizáveis escritos

em HTML, TypeScript e CSS, para criar aplicativos da Web. Utilizando o angular cli, assinale a alternativa correta para criar um novo componente.

- a) ng build component <component-name>
- b) ng create component <component-name>
- c) ng new component <component-name>
- d) ng generate component <component-name>
- e) ng append component <component-name>

07. (FCC/TJ BA/2023) Em condições ideais, na interface de linha de comando, um Analista foi até o diretório raiz de um projeto Angular e utilizou o comando ng lint para

- a) iniciar o servidor de desenvolvimento e disponibilizar o aplicativo em um navegador web.
- b) gerar um novo componente chamado lint na aplicação Angular.
- c) atualizar a versão do Angular e suas dependências para a versão mais recente.
- d) verificar o código-fonte em busca de problemas de formatação e qualidade de código.
- e) adicionar um pacote de terceiro chamado lint a aplicação Angular.

08. (FCC/TRT 15/2023) Para envolver vários componentes Angular Material e aplicar estilos comuns a campos de texto, como a linha inferior, rótulo flutuante e mensagens de dica, um Técnico deve utilizar o componente

- a) <form-angular-material>
- b) <mat-form-field>
- c) <m:formField>
- d) <mat: formField>
- e) <m:form>

09. (CETREDE/UFC/2022) Sobre os Frameworks de desenvolvimento de software mais populares do mercado, é correto afirmar que:

- a) o Angular é uma plataforma e framework para a construção de aplicações single-page do lado cliente usando HTML e TypeScript.
- b) o React é uma biblioteca JavaScript baseada em componentes para a construção de aplicações back-end com acesso direto à SGBD.
- c) o Node.js é um ambiente de desenvolvimento JavaScript apenas para Frontend.
- d) o Mongoose é uma biblioteca para a modelagem de dados de aplicações desenvolvidas em React para o MongoDB.

10. (FGV/TJ TO/2022) Com relação à plataforma Angular, é possível converter um componente numa classe que pode ser registrada no browser e incluída no DOM.

A função que permite essa operação é denominada:

- a) addElement()
- b) createAndRegister()
- c) createCustomElement()
- d) putElement()
- e) registerElement()

11. (FCC/TRT 22/2022) Em um site desenvolvido utilizando o framework Angular, um Técnico observou no interior da tag <body> do arquivo index.html um elemento Angular definido no arquivo app/app.component.ts que será o primeiro componente carregado e o contêiner para os outros componentes. Trata-se do elemento

- a) <angular-root>
- b) <render>
- c) <document-root>
- d) <app-root>
- e) <main-app>

12. (IDECAN/TJ PI/2022) O Angular é uma grande plataforma de desenvolvimento para aplicações. Com ele podemos desenvolver aplicações web e também aplicações mobile. Ela conta com templates, componentes, módulos, serviços e diversas ferramentas de infraestrutura. Tudo isso foi criado para tornar o desenvolvimento de aplicações mais simples e rápido. A partir da sua versão 9 o Angular passou a contar, por padrão, com um novo compilador.

Ele é responsável pela renderização de conteúdo e sua chegada trouxe mais performance para o ambiente de desenvolvimento Angular. Marque a alternativa que indica, corretamente, o nome deste compilador.

- a) V8
- b) Ivy
- c) Gecko
- d) ECMA
- e) Chromium

13. (FCM/IF AM/2022) Com relação à diretiva *ngFor do Angular é INCORRETO afirmar que

- a) é uma estrutura de repetição.
- b) não afeta a estrutura do DOM.
- c) serve para estender o poder do HTML.
- d) é uma diretiva estrutural.
- e) não é uma diretiva de atributo.

14. (FCM/IF AM/2022) O desenvolvimento de aplicativos móveis depende da instalação de algumas plataformas, editores de código e frameworks, além de pacotes e módulos. Há diversas ferramentas gratuitas que podem ser utilizadas pelos programadores durante o desenvolvimento de suas aplicações.

Entre elas existe um framework e plataforma para desenvolvimento de interfaces de aplicações móveis e Web baseadas em JavaScript, CSS ou HTML. É uma ferramenta mantida pela equipe do Google que permite o desenvolvimento front-end baseado em Typescript.

Esta ferramenta é o

- a) Ionic.
- b) Angular.
- c) Node.js.
- d) Node Package Manager (Npm).
- e) Apache Cordova.

15. (FCM/IF AM/2022) O Angular tem como objetivo facilitar o desenvolvimento de aplicações para a web e dispositivos móveis.

Este framework é baseado em qual linguagem?

- a) CSS
- b) HTML
- c) JavaScript
- d) C/C++
- e) Java

16. (FCM/UFJF/2019) Uma aplicação em Angular é feita por meio de componentes. O par de metadados obrigatórios no decoration @Component para a definição de uma classe como componente é

- a) view e filter.
- b) tag e constructor.
- c) styleUrls e provider.
- d) HTMLpage e provider.
- e) selector e templateUrl.

17. (FCM/UFJF/2019) No Framework Angular, o tipo de envio ou sincronização de dados que permite atuar dentro da classe de um componente é

- a) interpolation.
- b) event binding.
- c) property binding.
- d) two-way data binding.
- e) one-way data binding.

18. (PR4 UFRJ/UFRJ/2018) Considerando o código a seguir, implementado em Angular, versão 4.4.0, responda a questão.

```
1. import { Component } from '@angular/core';  
2.  
3.   @Component({  
4.     selector: 'alunos',  
5.     template: `  
6.       <h1>TITULO</h1>  
7.       <h2>Nome do Aluno: ALUNO</h2>  
8.     `,  
9.   })  
10.  export class AlunoComponent {  
11.    title = 'Alunos inscritos';  
12.    aluno = 'Nome do Aluno';  
13.  }
```

Para que o componente em questão apresente no navegador, na tag da linha 6, na declaração TITULO, o texto: "Alunos inscritos" e na tag da linha 7, na declaração ALUNO, o texto: "Nome do Aluno", é necessário alterar as declarações indicadas, respectivamente, por:

- a) this.title e this.aluno
- b) {{title}} e {{aluno}}
- c) {{this.title}} e {{this.aluno}}
- d) title e aluno
- e) \$this->title e \$this->aluno

19. (PR4 UFRJ/UFRJ/2018) Considerando o código a seguir, implementado em Angular, versão 4.4.0, responda a questão.

```
1. import { Component } from '@angular/core';  
2.  
3.   @Component({  
4.     selector: 'alunos',  
5.     template: `  
6.       <h1>TITULO</h1>
```

```
7.    <h2>Nome do Aluno: ALUNO</h2>
8.    `
9.  })
10.  export class AlunoComponent {
11.    title = 'Alunos inscritos';
12.    aluno = 'Nome do Aluno';
13.  }
```

Assinale a alternativa que apresenta a tag necessária para declarar o uso deste componente em uma página html considerando um projeto Angular.

- a) <AlunosComponent> e </AlunosComponent>
- b) <alunos> e </alunos>
- c) <Component> e </Component>
- d) <app> e </app>
- e) <selector> e </selector>

20. (FCC/DPE AM/2018) Considere a página web abaixo que utiliza Angular.

```
<!DOCTYPE html>
<html>
  <head>
    <script src="angular.min.js"></script>
  </head>
  <body>
    <div ng-app="">
      <form>
        <p>Nome: <input type="text" ng-model="nome"></p>
      </form>
      |
    </div>
  </body>
</html>
```

Considere os comandos:

- I. <p ng-bind="nome"></p>
- II. <p>{{nome}}</p>
- III. <p ng-print="nome"></p>
- IV. <p>{\$nome}</p>

Para que o que for digitado no campo nome seja exibido simultaneamente em um parágrafo, na lacuna I podem ser utilizados os comandos que constam APENAS nos itens

- a) III e IV.
- b) I e III.
- c) II e IV.
- d) I e IV.
- e) I e II.

21. (Instituto AOCP/PRODEB/2018) Sobre o Framework de desenvolvimento estrutural Angular, analise as assertivas e assinale a alternativa que aponta a(s) correta(s).

- I. O Framework Estrutural Angular é uma ótima opção para criar SPA (Single Page Applications).
- II. O Framework Estrutural Angular foi desenvolvido para criar aplicações web dinâmicas.
- III. O Framework Estrutural Angular foi desenvolvido para criar banco de dados relacionais.
- IV. O Framework Estrutural Angular é utilizado para a configuração de redes IPV6.

- a) Apenas II.
- b) Apenas I, III e IV.
- c) Apenas I e II.
- d) Apenas III e IV.
- e) Apenas II e III.

22. (QUADRIX/CRM PR/2018) No que se refere a ambientes para desenvolvimento de sistemas, julgue o item a seguir.

Angular 2 é um IDE com suporte para linguagens Java e JavaScript e versões para diversos sistemas operacionais.

GABARITO

GABARITO



1. Letra D
2. Letra D
3. Letra E
4. Letra B
5. Errado
6. Letra D
7. Letra D
8. Letra B

9. Letra A
10. Letra C
11. Letra D
12. Letra B
13. Letra B
14. Letra B
15. Letra C
16. Letra E

17. Letra B
18. Letra B
19. Letra B
20. Letra E
21. Letra C
22. Errado

ANGULARJS

Conceitos Gerais



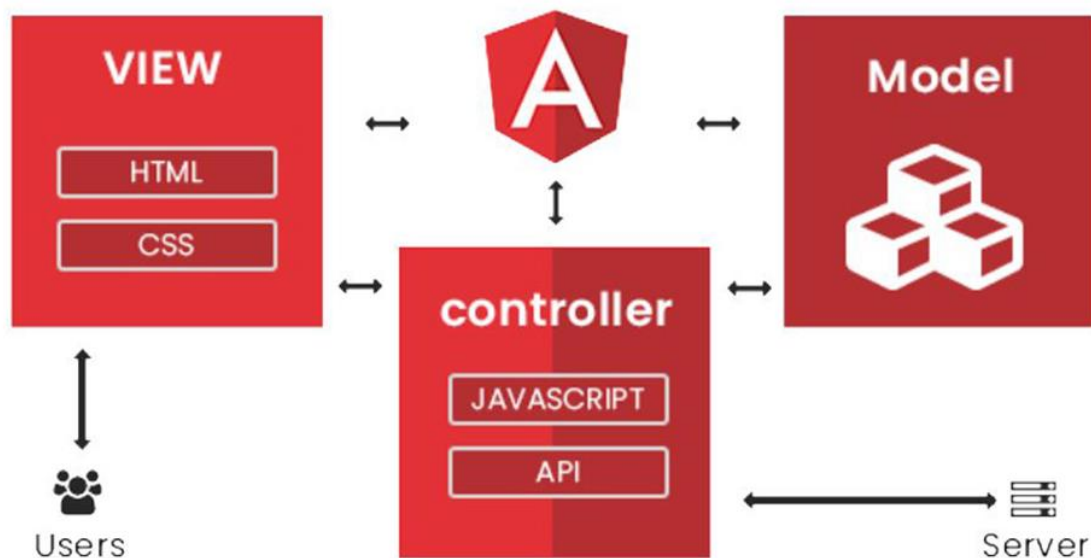
O **AngularJS** é uma biblioteca (*framework*) destinado a **aplicações de página única** (SPA - *Single Page Application*) através da combinação de HTML, CSS e JavaScript. O Angular é uma *framework* open-source, mantido principalmente pelo Google. É, hoje, um dos principais *frameworks* quando se deseja criar uma SPA rodando no *client-side*, isso é, no próprio navegador do cliente - característico do JavaScript.

Um ponto inicial de destaque para a biblioteca é o uso da arquitetura MVC. Não é nosso objetivo hoje estudá-la, mas vamos relembrar brevemente o que a arquitetura MVC traz para o contexto do desenvolvimento.

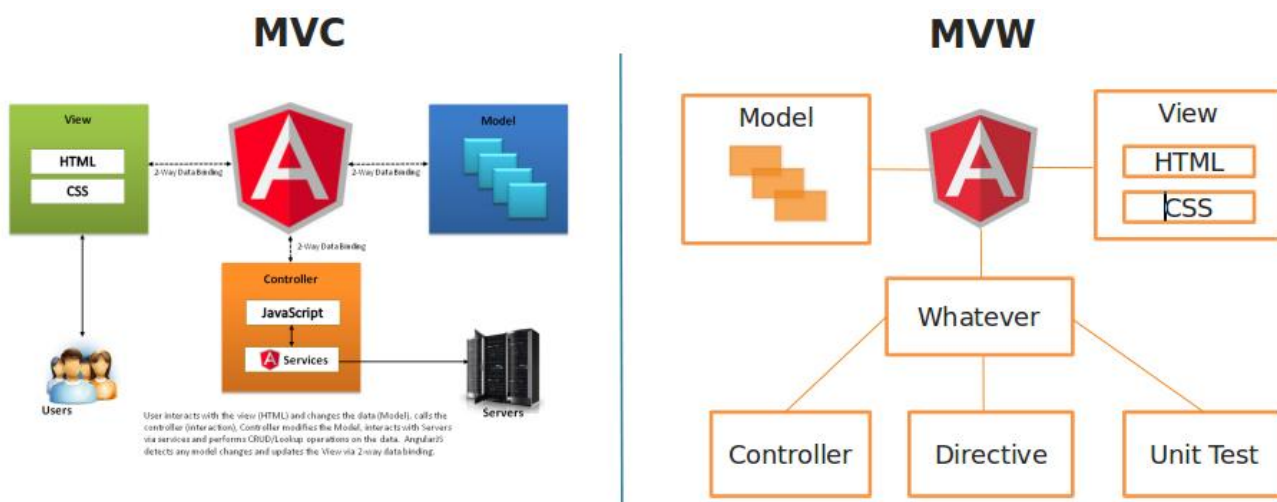
MVC, ou **Model-View-Controller**, é um tipo de arquitetura em camadas - mais especificamente, três camadas. Cada camada age com uma responsabilidade distinta. As camadas são:

- **Model (modelo):** representa os dados da aplicação.
- **View (visão):** exibe os dados e captura as ações do usuário através de uma interface de usuário.
- **Controller:** contém a lógica que interage com o *model*, e atualiza a *view*.

Na camada do modelo, temos os dados e bancos de dados atuando como repositório dessas informações. Na *view*, temos arquivos codificados em HTML e CSS, para criar visualizações ao usuário final - visualizações essas que são *renderizadas* no navegador do usuário. Por fim, o controlador envolve um conjunto de APIs que são manipuladas a partir de códigos JavaScript.



Apesar da construção feita em MVC, o Google “vende” o Angular como adepta ao modelo MVW - Model-View-Whatever. Isso quer dizer que podemos ter qualquer coisa ocupando o espaço do controlador: testes unitários, diretivas, ou um próprio controlador.



Um detalhe: o **Angular possui várias versões**. Sua versão original, chamada de **AngularJS**, equivale às versões anteriores ao Angular 2 - ou seja, as versões 1.x. Isso pois a versão do Angular 2 foi “reescrita” para ser baseada em TypeScript, não mais puramente em JavaScript. Então cuidado, pois AngularJS e Angular, apesar de serem derivados de um mesmo framework, são coisas distintas. Essa aula terá como foco o AngularJS.

(CESGRANRIO/IPEA/2024) Admita que a área de desenvolvimento de softwares do Ipea pretenda utilizar uma linguagem usando um framework para TypeScript, de modo a construir uma aplicação web de uma só página para mostrar dados escolares de municípios que possuem menos de 5.000

habitantes. Nessa situação, a interação e a navegação entre as sessões da página ocorrerão de modo a não ser necessário o recarregamento da página a cada alteração.

Um framework de uso adequado para essa situação é o

- a) Scrum
- b) Django
- c) Express
- d) Angular
- e) ASPNET

Comentários:

Das alternativas apresentadas, a única que tem um *framework* que é capaz de lidar com o TypeScript é a alternativa D - Angular. Um detalhe importante, se a alternativa fosse "AngularJS" ela estaria incorreta. (Gabarito: Letra D)

Expressões

Expressões são construções que permitem a **vinculação dinâmica entre os dados e a interface do usuário** - ou seja, atuam como o "Controller", na arquitetura MVC. Elas são usadas para executar operações simples e retornar valores que podem ser exibidos diretamente na interface. As expressões são semelhantes às expressões JavaScript, mas são avaliadas dentro do contexto do escopo AngularJS, que oferece algumas funcionalidades adicionais e restrições específicas para garantir segurança e desempenho.

Usualmente, uma expressão é escrita dentro de uma *tag* do HTML, e é caracterizada pela presença de um conjunto de chaves duplas - `{{ expressão }}`. Alternativamente, as expressões podem estar dentro de diretivas, um assunto que a gente verá logo mais - mas adiantando, são basicamente propriedades alocadas dentro de uma *tag*.

Veja um exemplo simples de expressões dentro do HTML:

```
HTML
<div ng-app="">
  <p> Expressão simples: {{ 5 + 5 }} </p>
</div>
```

Então temos, de forma "leiga", um espaço para escrever JavaScript dentro do HTML. Digo JavaScript pois a sintaxe do Angular é idêntica ao JavaScript em praticamente todos os aspectos - números, *strings*, objetos e arrays. Porém temos um detalhe importante: **expressões do AngularJS não suportam condicionais, loops e exceções**.

QUESTÃO DE PROVA



(CEBRASPE/STM/2018) Julgue o item subsequente, a respeito de programação web.

Após ser executada, a expressão Angular JS

```
<div ng-app="">
<p>Resultado: {( 5 + 12 )}</p>
</div>
```

produzirá, como resultado, Resultado: 17.

Comentários:

A expressão AngularJS possui erro de escrita. O correto seria {{ 5 + 12 }} (par duplo de chaves), e não da forma que foi escrito - assim, não teremos nenhum resultado. (Gabarito: Errado)

O AngularJS também traz algumas coisas que não existem nativamente no JavaScript. A ferramenta de maior destaque são os **filtros**. Os **filtros permitem formatar e transformar dados diretamente nas expressões** antes de exibi-los na interface do usuário. Eles são utilizados dentro das expressões com o caractere pipe **|** e podem ser aplicados a variáveis, strings, números, arrays, entre outros.

Temos vários filtros:

FILTRO	DESCRIÇÃO	EXEMPLO
currency	Formata um número de forma monetária	{{ preço currency }}, {{ preço currency: "R\$" }}
date	Formata uma data de acordo com um padrão especificado	{{ data date: 'dd/MM/yyyy' }}, {{ data date: 'fullDate' }}
filter	Filtra um <i>array</i> com base em um critério específico	{{ array filter: {nome: 'João'} }}
json	Converte um objeto em uma string JSON formatada	{{ objeto json }}
limitTo	Limita o número de itens em uma array ou caracteres em uma <i>string</i>	{{ array limitTo: 5 }}
lowercase, uppercase	Transforma o texto em caracteres minúsculos ou maiúsculos, respectivamente	{{ text lowercase }}, {{ texto uppercase }}
number	Formata um número de acordo com a especificação de casas decimais	{{ numero number: 2 }}
orderBy	Ordena um array com base em uma expressão	{{ array orderBy: 'nome' }}

Entre outros filtros que são de interpretação bem literal.

Módulos

Um **módulo** no AngularJS é um **contêiner para diferentes partes de uma aplicação**, incluindo controladores, serviços, diretivas, filtros, e configurações. Cada aplicação AngularJS possui pelo menos um módulo raiz, que pode depender de outros módulos. Para criar um módulo, atribuímos a função `angular.module` a uma variável. Veja:

```
HTML

<script>
  var app = angular.module("myApp", []);
</script>
```

O parâmetro `myApp`, citado na sintaxe acima, refere-se a um elemento HTML em que a aplicação irá rodar - marcado com a diretiva `ng-app`. A array vazia `[]` indica que não temos dependências entre módulos.

Após a criação, podemos adicionar diferentes componentes ao módulo. As principais formas de interação são pela adição de **controladores** e **diretivas**. Veja como é feita a atribuição de um controlador:

```
HTML

<script>
  app.controller('meuCtrl', function($scope) {
    $scope.mensagem = 'Olá, Mundo!'; });
</script>
```

Na sintaxe acima:

- **`app.controller`** → indica que estamos definindo um controlador. Ela recebe dois argumentos - o primeiro é o nome do controlador (no caso, `meuCtrl`), e o segundo a função que define a lógica do controlador.
- **`$scope`** → objeto que se refere ao escopo do modelo e permite a vinculação de dados entre o controlador e a visualização
- **`$scope.mensagem = "Olá, mundo!"`** → definimos uma propriedade chamada de mensagem. Isso significa que essa propriedade está disponível no escopo do controlador e pode ser acessada na visualização associada.

Quando esse controlador é associado a um elemento HTML usando a diretiva `ng-controller`, o AngularJS cria uma nova instância do `$scope` para aquele elemento e seus descendentes. A função

do controlador é então executada, configurando o `$scope` de acordo com o que está definido na função.

De forma geral, os módulos organizam o código em partes menores, reutilizáveis, trazendo desacoplamento à aplicação. Porém, para termos o reuso dos módulos, é possível ter certo nível de acoplamento entre eles - resultando nas **dependências**. Um módulo pode depender de outros módulos, o que permite a reutilização de funcionalidades e a criação de componentes modulares e desacoplados.

Para definir dependências entre módulos, definimo-las dentro da construção do módulo, na função `angular.module`. No mesmo exemplo que trabalhamos anteriormente, podemos criar uma dependência dentro do espaço de colchetes que ficou em branco:

```
HTML

<script>
    var app = angular.module("myApp", ["Dependência"]);
</script>
```

Para criar essa modularidade, é comum que cada módulo seja trabalhado num arquivo JavaScript separado. Para termos acesso a esses módulos dentro do arquivo HTML, basta inserirmos o "endereço" do arquivo JavaScript dentro da tag `<script src='endereço'> </script>`. Aqui não temos muitas exigências técnicas de onde importar esses scripts - apenas uma que é mandatória. Precisamos primeiramente carregar a biblioteca do Angular antes de carregar os módulos, justamente para a aplicação entender o que é um módulo. Por essa razão, temos que importar módulos apenas após a definição da importação do Angular.

Diretivas

O objetivo do AngularJS é **estender as capacidades do HTML**. Por ser uma biblioteca de JavaScript, ele pode ser adicionado em uma página escrita em HTML a partir de uma tag `<script>`. Veja o corpo da tag:

```
HTML

<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js
">
</script>
```

Apesar de parecer uma introdução boba e “pouco relevante”, veja essa questão da FGV:

(FGV/TRT 13/2022) Considere o código a seguir.

```
<!DOCTYPE html>
<html>
<script src="-----"></script>
<body>
<div ng-app="">
<p>Endereço: <input type="text" ng-model="nome"></p>
<p>Confirmando: {{ nome }}</p>
</div>
</body>
</html>
```

Assinale a referência que substitui corretamente o trecho tracejado.

- a) [https://ajax.googleapis.com/ajax/libs/jquery/ 3.6.0/jquery.min.js](https://ajax.googleapis.com/ajax/libs/jquery/3.6.0/jquery.min.js)
- b) [https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/ js/bootstrap.min.js](https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/js/bootstrap.min.js)
- c) [https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/ css/bootstrap.min.css](https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css)
- d) [https://ajax.googleapis.com/ajax/libs/angularjs/ 1.6.9/angular.min.js](https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js)
- e) <https://ajax.googleapis.com/ajax/libs/graphs/2.5/js>

Comentários:

Veja que é uma questão simples - mas exige o conhecimento prévio do link para importar o *script* do AngularJS. A alternativa que apresenta o endereço correto é a letra D. (Gabarito: Letra D)

Essa extensão do HTML é feita a partir das **diretivas**. Elas são marcadores especiais no HTML que o AngularJS reconhece e utiliza para anexar comportamento específico a um elemento DOM ou até mesmo transformar o DOM. Elas são identificadas a partir do seu posicionamento dentro das tags. Por exemplo, podemos ter uma tag de parágrafo escrita `<p ng-bind="x">` - aqui, o `ng-bind` corresponde a uma diretiva.

Veja um exemplo de código com a implementação das diretivas:

```
HTML

<div ng-app="">
  <p>Name: <input type="text" ng-model="name"></p>
  <p ng-bind="name"></p>
</div>
```

Nesse exemplo acima:

- **ng-app**: define que estamos trabalhando com um app AngularJS
- **ng-model**: vincula o valor do campo de `input` à variável `name`
- **ng-bind**: vincula o conteúdo do parágrafo `<p>` para a variável `name`

Existem diversas diretivas “pré-prontas”, criadas pelo próprio *framework*. Vou trazer uma tabelinha para você com as principais - muita atenção na leitura, é um assunto que é recorrente e envolve um certo grau de *decoreba* da sua parte.



DIRETIVA	DESCRIÇÃO
ng-app	Inicializa uma aplicação AngularJS.
ng-bind	Liga o conteúdo de um elemento HTML a uma expressão do escopo, atualizando-o automaticamente.

ng-model	Liga o valor de um controle HTML (como input, select, textarea) ao modelo de dados do AngularJS.
ng-repeat	Repete um conjunto de elementos HTML para cada item de uma coleção.
ng-if	Renderiza o elemento HTML apenas se a expressão for verdadeira.
ng-show	Mostra ou esconde o elemento HTML com base na avaliação de uma expressão.
ng-hide	Esconde ou mostra o elemento HTML com base na avaliação de uma expressão.
ng-class	Adiciona ou remove classes CSS em um elemento com base na avaliação de uma expressão.
ng-style	Aplica estilos CSS a um elemento com base na avaliação de uma expressão.
ng-click	Define uma função a ser chamada quando o elemento é clicado.
ng-submit	Define uma função a ser chamada quando um formulário é enviado.
ng-change	Define uma função a ser chamada quando o valor de um elemento de formulário muda.
ng-init	Inicializa dados em um escopo.
ng-include	Inclui um fragmento de HTML em um elemento.
ng-switch	Condicionalmente troca entre elementos de acordo com a avaliação de uma expressão.
ng-options	Popula um elemento <select> com opções a partir de uma coleção.
ng-view	Renderiza a vista (view) associada a uma rota definida pelo \$routeProvider.
ng-href	Define dinamicamente o valor do atributo href em um link.
ng-src	Define dinamicamente o valor do atributo src em uma imagem.
ng-disabled	Habilita ou desabilita um elemento com base na avaliação de uma expressão.

(SUGEP UFRPE/UFRPE/2022) AngularJS é um framework JavaScript para desenvolvimento de aplicações web. No desenvolvimento de páginas web dinâmicas, é muito importante a utilização de laços. Qual a diretiva desse framework que permite repetir um elemento DOM uma vez, para cada item de uma coleção?

- a) ng-for
- b) ng-while
- c) ng-repeat
- d) ng-iterate
- e) ng-loop

Comentários:

A diretiva que faz repetições de elementos do DOM para cada item de uma coleção é a `ng-repeat`. Quanto às demais diretivas trazidos pela questão:

- `ng-for`: nativa do Angular 2+, é utilizada para iterar sobre uma coleção de itens e gerar elementos HTML para cada item da coleção.
- `ng-while`: não existe
- `ng-iterate`: não existe
- `ng-loop`: não existe

Portanto, o gabarito é a letra C. (Gabarito: Letra C)

Então já sabe: sempre que vir algum atributo dentro de uma `tag` HTML com o prefixo `ng-`, estamos falando de alguma diretiva do Angular.

Um evento interessante ocorre nas diretivas: o chamado **data binding**, ou **vinculação de dados**. Veja o seguinte código:

```
HTML

<div ng-app="">
  <p>Nome: <input type="text" ng-model="primeiroNome"></p>
  <p>Você escreveu: {{ primeiroNome }}</p>
</div>
```

Na sintaxe acima, definimos a diretiva `ng-model` dentro de um campo de input no HTML, associada a um valor chamado de `primeiroNome`. Com isso, associamos um valor ao dado "primeiroNome" e, sempre que o chamarmos, teremos como retorno o valor associado. E essa vinculação serve nas duas direções (*two-way binding*), já que tanto a alteração do valor do input, quanto da propriedade, alteram o valor alocado.

Então, de forma geral, podemos considerar o Data Binding como uma forma de sincronização entre a camada *model* e a camada *view*.

TWO WAY BINDING → SINCRONIZAR VALOR DO MODEL COM APRESENTAÇÃO DA VIEW

Lembre-se, o *model* é um conjunto de dados que a aplicação lida para funcionar - nomes de funcionário, valores retirados de bancos de dados, alocados em *inputs*, entre outros.

Controllers

Os **controllers**, ou controladores, são ferramentas do AngularJS responsáveis por controlar os dados da aplicação - fazendo a ponte entre a camada de modelo, e de visualização. Eles funcionam como objetos de JavaScript, e são criados a partir de construtores de objeto do JavaScript. A diretiva `ng-controller` é a responsável por definir o controlador da aplicação.

Veja um exemplo em código da criação de um *controller*:

```
HTML

<div ng-app="myApp" ng-controller="myCtrl">

  Nome: <input type="text" ng-model="firstName"><br>
  Sobrenome: <input type="text" ng-model="lastName"><br>
  <br>
  Nome completo: {{firstName + " " + lastName}}

</div>

<script>
  var app = angular.module('myApp', []);

  app.controller('myCtrl', function($scope) {
    $scope.firstName = "João";
    $scope.lastName = "da Silva";
  });
</script>
```

No código acima, temos o aplicativo Angular chamado de `myApp`, e um controlador, chamado de `myCtrl`. Criamos, no script, o `app` e o controlador. Falando especificamente do controlador, temos a definição nos parâmetros:

- O nome da virável - `myCtrl`
- A função de controle do controlador, com o parâmetro `$scope` (que representa o objeto da aplicação, com suas variáveis e funções)
- São criadas duas propriedades dentro do controlador - `firstName` e `lastName`. Essas propriedades são quase como variáveis, sempre que a *view* chamar esses valores, o controlador irá pegar o valor armazenado para a propriedade no *model* e substituí-lo na *view*.

É possível também implementar **métodos** como propriedades dentro de um controlador. São, basicamente, funções alocadas a alguma propriedade. Definimos a propriedade, rodamos um

`function() {}` padrão do JavaScript. Veja um exemplo, onde adaptamos o código para “Nome Completo” também ser definido a partir de uma propriedade.

```
HTML

<div ng-app="myApp" ng-controller="myCtrl">

  Nome: <input type="text" ng-model="firstName"><br>
  Sobrenome: <input type="text" ng-model="lastName"><br>
  <br>
  Nome completo: {{fullName()}}

</div>

<script>
  var app = angular.module('myApp', []);
  app.controller('myCtrl', function($scope) {
    $scope.firstName = "João";
    $scope.lastName = "da Silva";
    $scope.fullName = function() {
      return $scope.firstName + " " + $scope.lastName;
    };
  });
</script>
```

Definimos a propriedade `fullName`, que roda uma função com retorno concatenando nome e sobrenome. Para chamarmos o nome completo na parte do HTML, simplesmente usamos uma expressão com o nome da propriedade. Isso é permitido pois definimos a propriedade como `$scope`, ou seja, acessável por toda a aplicação.

É importante aqui saber as **limitações do escopo**. No exemplo anterior, toda a aplicação está dentro de uma mesma `<div>` no HTML, o que quer dizer que, ao atribuirmos um controlador a essa `div`, todos seus elementos-filho poderão acessar essas variáveis de escopo. Mas imagine o seguinte caso:

HTML

```
<div ng-app="myApp">
  <div ng-app="myApp" ng-controller="myCtrl">
    First Name: <input type="text" ng-model="firstName"><br>
    Last Name: <input type="text" ng-model="lastName"><br>
    <br>
    Full Name: {{fullName()}}
  </div>
  <p>
    Essa foi a aplicação que gerou o {{fullName()}}
  </p>
</div>

<script>
  var app = angular.module('myApp', []);
  app.controller('myCtrl', function($scope) {
    $scope.firstName = "João";
    $scope.lastName = "da Silva";
    $scope.fullName = function() {
      return $scope.firstName + " " + $scope.lastName;
    };
  });
</script>
```

Nesse caso, o parágrafo `<p>` não poderia acessar a propriedade `{{fullName()}}`, já que o controlador não faz parte do seu escopo. Podemos resolver esse “problema” adicionando escopos raiz, chamados de **\$rootScope**. Nesses casos, as propriedades podem ser acessadas a partir de qualquer ponto da aplicação.

Services

Services são componentes destinados a promover a **modularidade** e a **reutilização do código**. Eles são utilizados para compartilhar dados e funcionalidades entre diferentes partes de uma aplicação AngularJS. Esses serviços podem ser usados autonomamente ou passados como parâmetros da função do controlador, junto do \$scope, por exemplo - desde que tenha sido alocado como uma dependência previamente.

Temos diversos tipos de **services** pré-prontos no AngularJS - aproximadamente 30 deles. Os principais são:

SERVICE	DESCRIÇÃO
\$http	Faz comunicações com servidores remotos por meio de requisições HTTP - GET, POST, PUT etc., retornando promessas.
\$q	Serviço de implementação de promessas do AngularJS, que ajuda na execução de operações assíncronas, permitindo o encadeamento e tratamento de promessas.
\$timeout	É um wrapper do window.setTimeout que permite a execução de funções após um determinado tempo, retornando uma promessa que pode ser cancelada.
\$interval	É um wrapper do window.setInterval que permite a execução repetida de uma função em intervalos de tempo específicos, retornando uma promessa que pode ser cancelada.
\$rootScope	O escopo raiz que é criado no topo da hierarquia de escopos do AngularJS. Ele é injetado automaticamente e pode ser usado para compartilhar dados entre escopos.
\$location	Fornece uma interface para ler ou alterar a URL da aplicação no navegador. Permite a navegação programática e o controle de parâmetros de consulta.
\$route	Fornece serviços para roteamento e navegação em aplicações de página única (SPA). Permite mapear URLs para templates e controladores específicos.
\$compile	Um serviço que compila um template HTML e linka o template compilado com um escopo, criando uma função de ligação (linking function).
\$log	Serviço de logging que encapsula os métodos console.log, console.error, console.warn, etc. Facilita o registro de mensagens e erros para depuração.
\$filter	Aplica filtros às expressões. Filtros podem ser usados em templates, controllers ou serviços para formatar dados para exibição.

Veja que muitas das informações obtidas pelos serviços são retiradas do DOM, e poderiam ser utilizadas diretamente. Por exemplo, o `$location` traz a mesma informações que o `windows.location` (forma de manipulação do DOM através do JavaScript). Porém, como o Angular monitora frequentemente alterações no DOM, é mais recomendável e confiável a utilização dos serviços nesses casos.

Os services funcionam como objetos do tipo **singleton**. Isso é, possuem apenas uma ocorrência em toda a aplicação e possuem um ponto de acesso global.

(Inédita/Prof. Felipe Mathias) Acerca dos conhecimentos sobre o framework AngularJS, julgue o item subsequente.

Os services funcionam como objetos do tipo singleton, trazendo um elemento de instanciação única e de acesso global

Comentários:

Perfeito! Os services são um exemplo de implementação de objetos Singleton, elaborados pela Gang of Four nos padrões de projetos orientados a objetos. (Gabarito: Certo)

Nesse contexto, o serviço `$http` acaba se tornando importante por implementar requisições, *promisses* e a programação assíncrona. Para implementarmos os diferentes métodos HTTP, usamos o “atalho de método”. Os atalhos são, entre outros:

- `.delete()`
- `.get()`
- `.head()`
- `.jsonp()`
- `.patch()`
- `.post()`
- `.put()`

Veja que são bem autoexplicativos - método `.get()`, por exemplo, implementa requisições do tipo GET, destinadas a retornar valores. Veja um exemplo de código onde implementamos um controlador com o serviço `$http` para fazermos requisições de informações (GET) para determinada API.

JavaScript

```
// Definindo o módulo AngularJS
var app = angular.module('userApp', []);

// Definindo o controlador
app.controller('UserController', ['$scope', '$http', function($scope,
$http) {
    $scope.users = [];
    $scope.errorMessage = '';

    // Função para buscar usuários
    $scope.fetchUsers = function() {
        $http.get('https://jsonplaceholder.typicode.com/users')
            .then(function(response) {
                // Sucesso na requisição
                $scope.users = response.data;
            })
            .catch(function(error) {
                // Erro na requisição
                $scope.errorMessage = 'Erro ao buscar usuários: ' +
error.message;
            });
    };
}]);
```

Os métodos são chamados de abreviações pois permitem que escrevamos as requisições de forma muito mais ágil. Precisaríamos abrir a requisição, definir o método, definir a url, e em seguida fazermos o tratamento (then e catch). Assim, definimos `$http.get('url')` de forma direta, ganhando velocidade na produção.

Angular API

A **API Global AngularJS** é uma coletânea de funções para performar tarefas comuns em programação, como comparar objetos, iterar sobre eles ou converter dados. Aqui não tem muito segredo: é saber quais são as funções e qual seu objetivo. Vou trazer uma tabela-resumo para que você pegue a essência das principais funções.

FUNÇÃO	DESCRIÇÃO
<code>isNumber(value)</code>	Verifica se o valor fornecido é um número. Retorna true se for um número, caso contrário, false.
<code>isString(value)</code>	Verifica se o valor fornecido é uma string. Retorna true se for uma string, caso contrário, false.
<code>isArray(value)</code>	Verifica se o valor fornecido é um array. Retorna true se for um array, caso contrário, false.
<code>isObject(value)</code>	Verifica se o valor fornecido é um objeto. Retorna true se for um objeto, caso contrário, false.
<code>isFunction(value)</code>	Verifica se o valor fornecido é uma função. Retorna true se for uma função, caso contrário, false.
<code>isDefined(value)</code>	Verifica se o valor fornecido está definido. Retorna true se estiver definido, caso contrário, false.
<code>isUndefined(value)</code>	Verifica se o valor fornecido está indefinido. Retorna true se estiver indefinido, caso contrário, false.
<code>lowercase(string)</code>	Converte a string fornecida para letras minúsculas.
<code>uppercase(string)</code>	Converte a string fornecida para letras maiúsculas.
<code>copy(source, [destination])</code>	Faz uma cópia profunda do objeto fonte para o destino (se fornecido) ou retorna uma nova cópia do objeto.
<code>equals(o1, o2)</code>	Verifica se dois objetos ou valores são iguais. Realiza uma comparação profunda.
<code>extend(destination, source)</code>	Copia todas as propriedades enumeráveis do objeto fonte para o objeto destino.
<code>noop()</code>	Função vazia que não faz nada. Útil como placeholder.
<code>identity(value)</code>	Retorna o valor fornecido. Frequentemente usado como função de retorno.
<code>element(element)</code>	Envolve um elemento DOM no objeto jqLite/jQuery AngularJS.
<code>isNumber(value)</code>	Verifica se o valor fornecido é um número. Retorna true se for um número, caso contrário, false.
<code>isString(value)</code>	Verifica se o valor fornecido é uma string. Retorna true se for uma string, caso contrário, false.
<code>isArray(value)</code>	Verifica se o valor fornecido é um array. Retorna true se for um array, caso contrário, false.

Routing

O AngularJS é usado para criar aplicações de página única, ou SPA. Com isso, temos uma aplicação inteira rodando em uma página única. Porém, em alguns casos é interessante termos outras páginas com funcionalidades exclusivas, formando uma aplicação um pouco mais complexa. Para esses casos, devemos usar um componente chamado de **router** (roteador), implementando a partir do **módulo ngRoute**.

Para o utilizarmos, precisamos primeiramente importá-lo de forma autônoma, já que ele não é uma aplicação nativa do AngularJS.

HTML

```
<script  
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular-  
route.js"></script>
```

Em seguida, o **ngRoute** deve ser adicionado como uma dependência do módulo referente à aplicação.

JavaScript

```
var app = angular.module("myApp", ["ngRoute"]);
```

Agora, temos acesso ao **\$routeProvider**, que pode ser passado como parâmetro de funções e ser usado para direcionar a outras páginas dentro da aplicação. Imagine cada página uma SPA autônoma e independente, mas que em conjunto formam a aplicação completa. Veja um exemplo, onde roteamos para três páginas: **main.htm**, **london.htm** e **paris.htm**.

JavaScript

```
var app = angular.module("myApp", ["ngRoute"]);
app.config(function($routeProvider) {
  $routeProvider
    .when("/", {
      templateUrl : "main.htm"
    })
    .when("/london", {
      templateUrl : "london.htm"
    })
    .when("/paris", {
      templateUrl : "paris.htm"
    });
});
```

QUESTÕES COMENTADAS

01. (UFRPE/UFRPE/2022) AngularJS é um framework JavaScript para desenvolvimento de aplicações web. No desenvolvimento de páginas web dinâmicas, é muito importante a utilização de laços. Qual a diretiva desse framework que permite repetir um elemento DOM uma vez, para cada item de uma coleção?

- a) ng-for
- b) ng-while
- c) ng-repeat
- d) ng-iterate
- e) ng-loop

Comentários:

Diretivas são formas de ligarmos o AngularJS ao HTML, dentro de suas tags. Quando queremos repetir um elemento DOM para cada item de uma coleção, a diretiva indicada é o ng-repeat (bem literal rs).

Gabarito: Letra C

02. (CEBRASPE/FUB/2022) Julgue o item a seguir, a respeito de projeto e desenvolvimento de sistemas.

Em AngularJS, a característica two-way data binding permite que, quando os dados no modelo são alterados, a exibição reflita a alteração e, quando os dados na exibição são alterados, o modelo também seja atualizado; isso acontece de forma imediata e automática, garantindo que o modelo e a visualização sejam atualizados o tempo todo.

Comentários:

O item está correto. O *two-way data binding* é uma forma de vinculação bidirecional entre visualização e dados - alterações em qualquer uma das pontas refletirão na outra automaticamente, mantendo os dados sempre atualizados e consistentes.

Gabarito: Certo

03. (CONSULPLAN/MPE PA/2022) Uma aplicação AngularJS é formada, basicamente, por componentes desacoplados e que têm um papel bem definido na arquitetura da aplicação. Assinale o componente responsável por ampliar a capacidade do HTML, viabilizando novos comportamentos na view.

- a) Serviços.
- b) Diretivas.
- c) Controller.
- d) Injetor de dependências.

Comentários:

Para ampliarmos a capacidade do HTML, inserindo uma integração com o Angular e permitindo trabalharmos com *scripts* direcionados, usamos as diretivas.

Gabarito: Letra B

04. (CONSULPLAN/MPE PA/2022) O AngularJS tem vários filtros, que podem ser executados aplicando um pipe a uma expressão existente na view. O filtro nativo do AngularJS que tem a função de formatar um número em valor monetário é:

- a) filter
- b) json
- c) limitTo
- d) currency

Comentários:

O *filter* é uma forma de formatarmos variáveis de acordo com algum padrão. Para a formatação de números em formato monetário, usamos o filtro nativo *currency*.

Gabarito: Letra D

05. (FGV/TRT 13/2022) Considere o código a seguir.

```
<!DOCTYPE html>
<html>
<script src="-----"></script>
<body>
<div ng-app="">
<p>Endereço: <input type="text" ng-model="nome"></p>
<p>Confirmando: {{ nome }}</p>
</div>
</body>
</html>
```

Assinale a referência que substitui corretamente o trecho tracejado.

- a) [https://ajax.googleapis.com/ajax/libs/jquery/ 3.6.0/jquery.min.js](https://ajax.googleapis.com/ajax/libs/jquery/3.6.0/jquery.min.js)
- b) [https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/ js/bootstrap.min.js](https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/js/bootstrap.min.js)
- c) [https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/ css/bootstrap.min.css](https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css)
- d) [https://ajax.googleapis.com/ajax/libs/angularjs/ 1.6.9/angular.min.js](https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js)
- e) <https://ajax.googleapis.com/ajax/libs/graphs/2.5/js>

Comentários:

Para resolvermos a questão, precisamos do link de referência para a importação do AngularJS. O link é [https://ajax.googleapis.com/ajax/libs/angularjs/ 1.6.9/angular.min.js](https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js).

Gabarito: Letra D

06. (QUADRIX/PRODAM/2022) O componente de aplicação single-page AngularJS que é responsável por ampliar a capacidade do HTML, viabilizando novos comportamentos na view, é chamado de

- a) model.
- b) injetor de dependências.
- c) diretivas.
- d) serviços.
- e) controller.

Comentários:

Questão tranquila. O elemento que estende a capacidade do HTML são as diretivas.

Gabarito: Letra C

07. (QUADRIX/PRODAM/2022) O AngularJS cria as visões dinamicamente, em tempo de execução, ao combinar os templates com as propriedades passadas a eles por meio do objeto

- a) \$injector.
- b) \$timeout.
- c) \$scope.
- d) \$destroy.
- e) \$on.

Comentários:

O objeto citado pela questão é o `$scope`. Ele define o escopo geral da aplicação, combinando os diferentes elementos em um grande “escopo global”.

Gabarito: Letra C

08. (CEBRASPE/PGDF/2021) Julgue o próximo item, relativo a desenvolvimento web em Java.

De acordo com o código subsequente, cada caractere digitado no campo do formulário é impresso no lugar de `{{ name }}`.

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/
angularjs/1.6.9/angular.min.js"></script>
<body>

<div ng-app="">

<p><input type="text" ng-model="name"></p>
<p>:: {{ name }}</p>

</div>
</body>
</html>
```

Comentários:

Perfeito! Com o `ng-model`, vinculamos os dados do *input* com a variável Angular `name`. Com isso, cada caractere digitado será impresso no lugar da expressão Angular `{{ name }}`.

Gabarito: Certo

09. (Instituto AOC/SANESUL/2021) Qual será o resultado da aplicação da diretiva “`ng-switch`” do Framework de Javascript Angular JS?

- a) Define o elemento raiz de um aplicativo.
- b) Especifica um comportamento em eventos de desfoque.
- c) Especifica classes CSS em elementos HTML.
- d) Impede a oscilação quando seu aplicativo está sendo carregado.
- e) É uma condição que será usada para mostrar/ocultar elementos-filho.

Comentários:

A diretiva ng-switch é direcionada a ativar ou desativar a visibilidade de um elemento. Pense no *switch* como um interruptor - ele "liga" ou "desliga" o componente.

Gabarito: Letra E

10. (Instituto AOCP/SANESUL/2021) O AngularJS é um framework JavaScript de código aberto para programação de aplicativos de página única, sendo que ele implementa o padrão de arquitetura MVC.

Sobre o framework AngularJS, é correto afirmar que

- a) a diretiva ng-model inicializa as variáveis da aplicação.
- b) a diretiva ng-app define um app AngularJS, ng-init inicializa as variáveis da aplicação e ng-bind vincula o conteúdo do elemento com o valor de uma variável.
- c) a diretiva ng-app inicializa as variáveis da aplicação enquanto ng-bind vincula o conteúdo do elemento com o valor de uma variável.
- d) o ng-bind efetua uma ligação de dados bidirecional enquanto o ng-model efetua uma ligação de dados unidirecional.
- e) a diretiva ng-bind define o modelo de dados com um binding bidirecional.

Comentários:

Vamos analisar cada alternativa.

- a) Errado. A diretiva ng-model é usada para criar uma ligação de dados bidirecional entre o elemento HTML (tipicamente inputs, selects, textareas) e a variável do modelo de dados no escopo AngularJS, mas não para inicializar variáveis.
- b) Certo. A diretiva ng-app define um aplicativo AngularJS e indica ao AngularJS que aquele elemento e seus filhos fazem parte de uma aplicação AngularJS. A diretiva ng-init pode ser usada para inicializar variáveis no escopo do AngularJS, e a diretiva ng-bind vincula o conteúdo de um elemento ao valor de uma expressão do escopo.
- c) Errado. A diretiva ng-app não inicializa variáveis; ela apenas define o início de uma aplicação AngularJS. A parte sobre ng-bind está correta, mas a afirmação como um todo está errada por causa do erro sobre ng-app.
- d) Errado. A diretiva ng-bind efetua uma ligação de dados unidirecional, atualizando o conteúdo do elemento quando o valor da expressão no escopo muda, mas não atualiza o escopo quando o conteúdo do elemento muda. Já a diretiva ng-model cria uma ligação de dados bidirecional entre o elemento HTML e a variável do modelo de dados no escopo.
- e) Errado. A diretiva ng-bind efetua uma ligação de dados unidirecional, não bidirecional.

Portanto, o gabarito é a letra B.

11. (CEBRASPE/STM/2018) Julgue o item subsequente, a respeito de programação web.

O serviço Angular JS \$http é usado para fazer uma chamada Ajax para o servidor.

Comentários:

O \$http é a forma de inserirmos requisições HTTP dentro do Angular. Enviamos uma requisição para a API, que a encaminha para o servidor - então sim, é uma forma de nos comunicarmos de forma assíncrona (Ajax) com o servidor.

Gabarito: Certo

12. (CEBRASPE/STM/2018) Julgue o item subsequente, a respeito de programação web.

Após ser executada, a expressão Angular JS

```
<div ng-app="">
<p>Resultado: {{ 5 + 12 }}</p>
</div>
```

produzirá, como resultado, Resultado: 17.

Comentários:

A questão insere uma expressão Angular de forma incorreta. A expressão deveria estar escrita com um par de duplas chaves {{ }}, não da forma que foi escrito. Nesse caso, não teremos retorno algum.

Gabarito: Errado

13. (CESGRANRIO/BASA/2018) É um exemplo de uma expressão Angular (Angular Expression) em AngularJS:

- a) <p>Candidato aprovado: {{ candidato.nome }}</p>
- b) <p>Candidato aprovado: [[candidato.nome]]</p>
- c) <p>Candidato aprovado: <%= candidato.nome %></p>
- d) <p>Candidato aprovado: <?= candidato.nome ?> </p>
- e) <p>Candidato aprovado: <js:angular value="candidato. nome"/></p>

Comentários:

A expressão Angular é implementada com o par de chaves duplas {{ }}. Portanto, correta a letra A.

Gabarito: Letra A

14. (CESGRANRIO/BB/2018) AngularJS segue um modelo MVC. Qual a diretiva correta em AngularJS para ligar um elemento de entrada de dados da visão, como um campo input do tipo texto, a um elemento do modelo, como uma variável do tipo string?

- a) ngView
- b) ngValue
- c) ngBind
- d) ngModel
- e) ngLink

Comentários:

A diretiva que vincula um campo de *input* a um dado, fazendo um *two-way binding*, é a ngModel.

Gabarito: Letra D

15. (FCC/TRT 2/2018) Considere a página web abaixo, que utiliza o Angular JS versão 1.6.9.

```
<!DOCTYPE html>
<html>
  <head>
    <title> TRT </title>
    <script src="angular.min.js">
    </script>
  </head>
  <body>
    <div ng-app="" ng-init="a=5;b=4">
      <p>Total: <span ng-bind="a * b"></span></p>
    </div>
  </body>
</html>
```

O resultado do cálculo envolvendo as variáveis a e b é mostrado pela instrução <p>Total: </p>. Outra forma de realizar o mesmo procedimento é usando a instrução

- a) <p>Total: </p>

- b) `<p>Total: {{ a * b }}</p>`
- c) `<p>Total: javascript.calc(a * b)</p>`
- d) `<p>Total: <script>Math.calc(a * b)</script></p>`
- e) `<p>Total: <script ng-math="a * b"></script></p>`

Comentários:

Uma forma alternativa de expressarmos o componente é através do uso de expressões Angular, com a interpolação. Nesse caso, pegaríamos as variáveis a e b, e alocaríamos no par de chaves - ficando com `{{ a * b }}`.

Gabarito: Letra B

16. (CEBRASPE/FUB/2018) Com relação ao uso dos frameworks AngularJS e Hibernate, julgue o item a seguir.

Eventos do AngularJS podem ser usados para associar diferentes ações a diferentes elementos HTML; por exemplo, um evento AngularJS pode ser usado para associar uma ação relacionada à seleção de um elemento HTML por meio do uso de um mouse.

Comentários:

Perfeito. Os eventos são diferentes interações com o aplicativo - clicar, passar o *mouse*, carregar a página, entre outros. Com eles podemos “captar” essas interações e realizar ações baseadas nelas.

Gabarito: Certo

17. (ProGPe UFSCAR/UFSCAR/2018) Os fragmentos de código referentes aos arquivos (I) index.html e (II) app.js evidenciam o emprego de:

```
<body>
  <tabs>
    <pane title="RESULTADO">
      <span>Data: {{ '2018-02-01' | date:'shortDate' }}</span><br>
      <div ng-controller="ContadorTentativa">
        <div ng-repeat="Cont in tentativas">
          <ng-pluralize count="Cont" when="tentForm"></ng-pluralize>
        </div>
      </div>
    </pane>
  </tabs>
</body>
```

I

```
.controller('ContadorTentativa', function($scope, $locale) {  
  $scope.tentativas = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10];  
  if ($locale.id === 'en-us') {  
    $scope.tentForm = {  
      0: 'Nenhuma tentativa',  
      one: '{} tentativa',  
      other: '{} tentativas'  
    };  
  } else {  
    $scope.tentForm = {  
      0: 'Algo errado',  
      one: '{} erro',  
      other: '{} erros'  
    };  
  }  
});
```

II

- a) QueryJS.
- b) AngularJS.
- c) Facelets.
- d) Portlets.
- e) PrimeFaces.

Comentários:

Temos alguns elementos que nos permitem identificar o uso do *framework* AngularJS. Por exemplo, as diretivas ng-repeat, a expressão Angular {{ }}, o componente <ng-pluralize>, e o objeto \$scope.

Gabarito: Letra B

18. (FCC/TRT 11/2017) Considere o fragmento de código HTML abaixo.

```
<body>  
<div>  
  <label>Processo N°:</label>  
  <input type="text" ng-model="processo">  
  <p>O número do processo é {{processo}}.</p>  
</div>  
</body>
```

Este fragmento evidencia o uso de

- a) QueryJS.
- b) Facelets.

- c) AngularJS.
- d) Portlets.
- e) PrimeFaces.

Comentários:

Temos dois elementos que permitem identificar o *framework* utilizado:

- A diretiva ng-model
- O uso de expressões Angular com {{ processo }}

Com isso, conseguimos concluir que o AngularJS foi utilizado na aplicação.

Gabarito: Letra C

▪

LISTA DE QUESTÕES

01. (UFRPE/UFRPE/2022) AngularJS é um framework JavaScript para desenvolvimento de aplicações web. No desenvolvimento de páginas web dinâmicas, é muito importante a utilização de laços. Qual a diretiva desse framework que permite repetir um elemento DOM uma vez, para cada item de uma coleção?

- a) ng-for
- b) ng-while
- c) ng-repeat
- d) ng-iterate
- e) ng-loop

02. (CEBRASPE/FUB/2022) Julgue o item a seguir, a respeito de projeto e desenvolvimento de sistemas.

Em AngularJS, a característica two-way data binding permite que, quando os dados no modelo são alterados, a exibição reflita a alteração e, quando os dados na exibição são alterados, o modelo também seja atualizado; isso acontece de forma imediata e automática, garantindo que o modelo e a visualização sejam atualizados o tempo todo.

03. (CONSULPLAN/MPE PA/2022) Uma aplicação AngularJS é formada, basicamente, por componentes desacoplados e que têm um papel bem definido na arquitetura da aplicação. Assinale o componente responsável por ampliar a capacidade do HTML, viabilizando novos comportamentos na view.

- a) Serviços.
- b) Diretivas.
- c) Controller.
- d) Injetor de dependências.

04. (CONSULPLAN/MPE PA/2022) O AngularJS tem vários filtros, que podem ser executados aplicando um pipe a uma expressão existente na view. O filtro nativo do AngularJS que tem a função de formatar um número em valor monetário é:

- a) filter
- b) json
- c) limitTo
- d) currency

05. (FGV/TRT 13/2022) Considere o código a seguir.

```
<!DOCTYPE html>
<html>
<script src="-----"></script>
<body>
<div ng-app="">
<p>Endereço: <input type="text" ng-model="nome"></p>
<p>Confirmando: {{ nome }}</p>
</div>
</body>
</html>
```

Assinale a referência que substitui corretamente o trecho tracejado.

- a) [https://ajax.googleapis.com/ajax/libs/jquery/ 3.6.0/jquery.min.js](https://ajax.googleapis.com/ajax/libs/jquery/3.6.0/jquery.min.js)
- b) [https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/ js/bootstrap.min.js](https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/js/bootstrap.min.js)
- c) [https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/ css/bootstrap.min.css](https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css)
- d) [https://ajax.googleapis.com/ajax/libs/angularjs/ 1.6.9/angular.min.js](https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js)
- e) <https://ajax.googleapis.com/ajax/libs/graphs/2.5/js>

06. (QUADRIX/PRODAM/2022) O componente de aplicação single-page AngularJS que é responsável por ampliar a capacidade do HTML, viabilizando novos comportamentos na view, é chamado de

- a) model.
- b) injetor de dependências.
- c) diretivas.
- d) serviços.
- e) controller.

07. (QUADRIX/PRODAM/2022) O AngularJS cria as visões dinamicamente, em tempo de execução, ao combinar os templates com as propriedades passadas a eles por meio do objeto

- a) \$injector.
- b) \$timeout.
- c) \$scope.
- d) \$destroy.
- e) \$on.

08. (CEBRASPE/PGDF/2021) Julgue o próximo item, relativo a desenvolvimento web em Java.

De acordo com o código subsequente, cada caractere digitado no campo do formulário é impresso no lugar de {{ name }}.

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/
angularjs/1.6.9/angular.min.js"></script>
<body>

<div ng-app="">

<p><input type="text" ng-model="name"></p>
<p>:: {{ name }}</p>

</div>
</body>
</html>
```

09. (Instituto AOCP/SANESUL/2021) Qual será o resultado da aplicação da diretiva “ng-switch” do Framework de Javascript Angular JS?

- a) Define o elemento raiz de um aplicativo.
- b) Especifica um comportamento em eventos de desfoque.
- c) Especifica classes CSS em elementos HTML.
- d) Impede a oscilação quando seu aplicativo está sendo carregado.
- e) É uma condição que será usada para mostrar/ocultar elementos-filho.

10. (Instituto AOCP/SANESUL/2021) O AngularJS é um framework JavaScript de código aberto para programação de aplicativos de página única, sendo que ele implementa o padrão de arquitetura MVC.

Sobre o framework AngularJS, é correto afirmar que

- a) a diretiva ng-model inicializa as variáveis da aplicação.
- b) a diretiva ng-app define um app AngularJS, ng-init inicializa as variáveis da aplicação e ng-bind vincula o conteúdo do elemento com o valor de uma variável.
- c) a diretiva ng-app inicializa as variáveis da aplicação enquanto ng-bind vincula o conteúdo do elemento com o valor de uma variável.
- d) o ng-bind efetua uma ligação de dados bidirecional enquanto o ng-model efetua uma ligação de dados unidirecional.
- e) a diretiva ng-bind define o modelo de dados com um binding bidirecional.

11. (CEBRASPE/STM/2018) Julgue o item subsequente, a respeito de programação web.

O serviço Angular JS \$http é usado para fazer uma chamada Ajax para o servidor.

12. (CEBRASPE/STM/2018) Julgue o item subsequente, a respeito de programação web.

Após ser executada, a expressão Angular JS

```
<div ng-app="">  
<p>Resultado: {{ 5 + 12 }}</p>  
</div>
```

produzirá, como resultado, Resultado: 17.

13. (CESGRANRIO/BASA/2018) É um exemplo de uma expressão Angular (Angular Expression) em AngularJS:

- a) <p>Candidato aprovado: {{ candidato.nome }}</p>
- b) <p>Candidato aprovado: [[candidato.nome]]</p>
- c) <p>Candidato aprovado: <%= candidato.nome %></p>
- d) <p>Candidato aprovado: <?= candidato.nome ?> </p>
- e) <p>Candidato aprovado: <js:angular value="candidato. nome"/></p>

14. (CESGRANRIO/BB/2018) AngularJS segue um modelo MVC. Qual a diretiva correta em AngularJS para ligar um elemento de entrada de dados da visão, como um campo input do tipo texto, a um elemento do modelo, como uma variável do tipo string?

- a) ngView
- b) ngValue
- c) ngBind
- d) ngModel
- e) ngLink

15. (FCC/TRT 2/2018) Considere a página web abaixo, que utiliza o Angular JS versão 1.6.9.

```
<!DOCTYPE html>
<html>
  <head>
    <title> TRT </title>
    <script src="angular.min.js">
    </script>
  </head>
  <body>
    <div ng-app="" ng-init="a=5;b=4">
      <p>Total: <span ng-bind="a * b"></span></p>
    </div>
  </body>
</html>
```

O resultado do cálculo envolvendo as variáveis a e b é mostrado pela instrução `<p>Total: <span ng-bind="a * b"></span></p>`. Outra forma de realizar o mesmo procedimento é usando a instrução

- a) `<p>Total: <span ng-calc="a * b"></span></p>`
- b) `<p>Total: {{ a * b }}</p>`
- c) `<p>Total: javascript.calc(a * b)</p>`
- d) `<p>Total: <script>Math.calc(a * b)</script></p>`
- e) `<p>Total: <script ng-math="a * b"></script></p>`

16. (CEBRASPE/FUB/2018) Com relação ao uso dos frameworks AngularJS e Hibernate, julgue o item a seguir.

Eventos do AngularJS podem ser usados para associar diferentes ações a diferentes elementos HTML; por exemplo, um evento AngularJS pode ser usado para associar uma ação relacionada à seleção de um elemento HTML por meio do uso de um mouse.

17. (ProGPe UFSCAR/UFSCAR/2018) Os fragmentos de código referentes aos arquivos (I) index.html e (II) app.js evidenciam o emprego de:

```
<body>
  <tabs>
    <pane title="RESULTADO">
      <span>Data: {{ '2018-02-01' | date:'shortDate' }}</span><br>
      <div ng-controller="ContadorTentativa">
        <div ng-repeat="Cont in tentativas">
          <ng-pluralize count="Cont" when="tentForm"></ng-pluralize>
        </div>
      </div>
    </pane>
  </tabs>
</body>
```

I

```
.controller('ContadorTentativa', function($scope, $locale) {  
  $scope.tentativas = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10];  
  if ($locale.id == 'en-us') {  
    $scope.tentForm = {  
      0: 'Nenhuma tentativa',  
      one: '{} tentativa',  
      other: '{} tentativas'  
    };  
  } else {  
    $scope.tentForm = {  
      0: 'Algo errado',  
      one: '{} erro',  
      other: '{} erros'  
    };  
  }  
});
```

II

- a) QueryJS.
- b) AngularJS.
- c) Facelets.
- d) Portlets.
- e) PrimeFaces.

18. (FCC/TRT 11/2017) Considere o fragmento de código HTML abaixo.

```
<body>  
<div>  
  <label>Processo N°:</label>  
  <input type="text" ng-model="processo">  
  <p>O número do processo é {{processo}}.</p>  
</div>  
</body>
```

Este fragmento evidencia o uso de

- a) QueryJS.
- b) Facelets.
- c) AngularJS.
- d) Portlets.
- e) PrimeFaces.

GABARITO

GABARITO

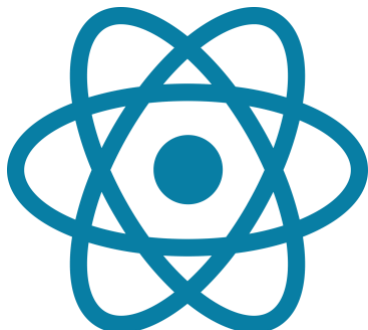


1. Letra C
2. Certo
3. Letra B
4. Letra D
5. Letra D
6. Letra C
7. Letra C
8. Certo
9. Letra E

10. Letra B
11. Certo
12. Errado
13. Letra A
14. Letra D
15. Letra B
16. Certo
17. Letra B
18. Letra C

REACT

Conceitos Gerais



O **ReactJS**, ou simplesmente **React**, é uma biblioteca de JavaScript utilizada para renderizar interfaces de usuários (UI), a partir de pequenos módulos chamados de **componentes**. Criado pela Meta (Facebook), o React é o framework mais popular quando se trata de criar aplicações de página única, ou Single Page Application (SPA). Além disso, o React possui um "irmão", o React Native, que permite aproveitar boa parte da sintaxe React na criação de aplicativos móveis através de uma abordagem chamada de desenvolvimento híbrido.

Falando especificamente do React, o cerne da biblioteca gira em torno da criação de pequenas partes do código, chamadas de componentes. Cada componente pode abranger desde uma única **tag** HTML, até páginas inteiras. Idealmente, quanto menor o componente, melhor - garantindo maior modularidade, reutilização e manutenibilidade. Outra característica marcante do React é o uso de um **DOM virtual**.

Os componentes podem (e devem) ser aninhados uns no outros. Então, por exemplo, podemos ter um componente relativo a uma seção numa página - um *header*, por exemplo. Nele, podemos ter componentes de botões, de imagens, e outras partes que formam o "todo".

Para iniciar o aplicativo React, existem várias abordagens. Até 2023, a ferramenta mais comum era o `create-react-app`, um instalador que criava diretórios, baixava as dependências e deixava o ambiente "pronto" de forma automática. Embora ainda possa ser utilizada, a ferramenta se encontra depreciada, e seu uso não é recomendado. A sintaxe nesses casos utilizava o Node Package eXecute (npx) - ficando `npx create-react-app <nome-do-app>`.

(AVANÇASP/Pref. Americana/2023) Para criar um projeto padrão em react, chamado projetoteste, a linha de comando correta é

- a) `npx-create-app-projetoteste`
- b) `projetoteste-npx-create`
- c) `npm create-projetoteste`
- d) `npx create-react-app projetoteste`
- e) `npx -app-projetoteste`.

Comentários:

Como a questão foi aplicada em 2023, a forma mais recomendada para iniciar um projeto em React era com o uso do `npx create-react-app projetoteste`. (Gabarito: Letra D)

Hoje em dia, temos duas escolhas válidas:

- Utilizando o **Vite**, onde teremos uma abordagem de **renderização baseada em client-side**, sendo a abordagem mais rápida. Porém, temos limitações em comunicações assíncronas e dependências de uso de bibliotecas externas. Seria o caso ideal quando não temos servidor ou quando utilizamos uma API externa para comunicações com um servidor.
- Utilizando algum framework full-stack. O mais indicado é o **Next.js**, embora seja possível o uso do Remix e do Gatsby. Aqui, a **renderização é server-side**, o que habilita o uso de comunicações assíncronas, utilizando componentes `async` e `await`.

Para essa aula, usaremos a abordagem mais recomendada - com base no *framework full-stack* Next.js. Para habilitarmos a utilização do React na máquina, precisamos utilizar um comando no terminal (CMD ou outro), que instalará o `react-dom`, elemento com todas as dependências e componentes necessários para começarmos a trabalhar com o React.

CMD

```
npm install react react-dom
```

Feita a instalação, podemos começar a trabalhar com o React dentro de nossos aplicativos. Um componente é uma função JavaScript, que exporta uma estrutura muito similar ao HTML. Essa estrutura é escrita numa **linguagem chamada JSX**, uma forma de misturarmos HTML e JavaScript numa mesma sintaxe. Ela parece muito com o HTML e termos de estruturação, mas é mais estrita e pode mostrar imagens dinâmicas.

Veja um exemplo de um componente React:

JavaScript

```
export default function Button() {  
  return (  
    <button className="button">  
      Esse botão foi feito em JSX  
    </button>  
  )  
}
```

Esse componente foi criado e exportado com o nome `Button`. Digamos que estamos trabalhando com uma seção `<header>` no mesmo diretório em que criamos esse botão, e queremos importar esse botão para o componente, para trabalhá-lo dentro do código. Fariamos algo nesse estilo:

JavaScript

```
import Button from './Button.js'

export default function Header() {
  return (
    <header className="container-pai">
      <Button />
      <Button />
    </header>
  )
}
```

As tags `<Button />` utilizadas são as implementações do botão criado anteriormente, dentro da nossa seção de cabeçalho. Perceba também a similaridade entre JSX e HTML - com algumas diferenças, por exemplo a forma de definirmos uma classe com o `className=""`.

Grande parte das mudanças nos atributos das tags envolve a simples implementação do camelCase, em outros casos, uma mudança na sintaxe. Trouxe uma tabela para você ver os principais atributos e como eles ficam no React/JSX.

HTML	JSX	Descrição
class	className	Define uma ou mais classes CSS para um elemento
for	htmlFor	Especifica a qual elemento de formulário um <code><label></code> está associado
tabindex	tabIndex	Define a ordem de tabulação do elemento
readonly	readOnly	Indica que um campo de entrada não pode ser modificado
maxlength	maxLength	Define o número máximo de caracteres permitidos em um campo de entrada
contenteditable	contentEditable	Indica se o conteúdo de um elemento é editável
colspan	colSpan	Define o número de colunas que uma célula <code><td></code> ou <code><th></code> deve abranger
rowspan	rowSpan	Define o número de linhas que uma célula <code><td></code> ou <code><th></code> deve abranger
accesskey	accessKey	Especifica uma tecla de atalho para ativar/focar um elemento
spellcheck	spellCheck	Indica se o elemento deve ter a verificação ortográfica ativada
autofocus	autoFocus	Define que um elemento deve receber o foco automático quando a página é carregada

autocomplete	autoComplete	Indica se o navegador deve sugerir automaticamente valores pré-preenchidos baseados em entradas anteriores
enctype	encType	Define o tipo de codificação usado ao enviar dados de formulário
novalidate	noValidate	Indica que os dados do formulário não devem ser validados ao serem enviados
inputmode	inputMode	Define o tipo de entrada esperado para um campo de entrada
ismap	isMap	Indica que uma imagem está associada a um mapa de imagem do lado do servidor
usemap	useMap	Define a referência a um mapa de imagem

Basicamente, qualquer atributo composto por mais de uma palavra é convertido pra camelCase.

Após a criação do projeto, precisamos iniciar um servidor local - assim, conseguiremos ver em tempo real como está ficando o projeto. Para iniciar o servidor do NodeJS, basta executarmos o comando **npm start**. A partir daí, o projeto fica disponível para ser acessado a partir do navegador no endereço *http://localhost:3000/*.

(FCC/TRT 23/2022) Considerando que o npx e Node.js estão instalados, um Técnico digitou os comandos abaixo no prompt do Windows para criar e executar um aplicativo React chamado website.

```
npm install create-react-app
npx create-react-app
website
cd website
npm start
```

No navegador, para aparecer o aplicativo React criado, deve ser utilizado o endereço

- a) <https://localhost:8080/>
- b) <http://127.0.0.1:8080/>
- c) <http://localhost:3000/>
- d) <https://127.0.0.1:3000/>
- e) <http://localhost:80/>

Comentários:

Como vimos, o endereço para acessar o aplicativo é *http://localhost:3000/*. (Gabarito: Letra C)

create-react-app

Embora esteja depreciado, muitas questões ainda cobram conhecimentos acerca do `create-react-app`. Vamos relembrar como usamos o comando na nossa CLI para fazer uma instalação completa de um aplicativo React:

CMD

```
npx create-react-app <nome-do-app>
```

Ao criar o aplicativo, o `create-react-app` cria alguns diretórios (pastas) para alocar arquivos específicos. Na pasta raiz, onde o aplicativo é criado, são geradas três pastas:

- A pasta `node_modules`, onde ficam armazenadas todas as dependências de terceiros necessárias para o seu projeto. Estas dependências são listadas no arquivo `package.json`.
- A pasta `public`, que contém arquivos estáticos que você deseja servir diretamente. Isso inclui o arquivo `index.html`, onde seu aplicativo React é montado, além de outros arquivos estáticos como imagens, ícones, fontes e outros recursos que não precisam ser processados pelo Webpack.
- A pasta `src` (source) contém o código-fonte do seu aplicativo React. É onde você desenvolve seus componentes, estilos, utilitários e outras lógicas de aplicação.

A estrutura típica de pastas num aplicativo React (my-app) segue essa seguinte estrutura:

```
my-app/  
├── node_modules/  
├── public/  
│   ├── favicon.ico  
│   ├── index.html  
│   └── manifest.json  
├── src/  
│   ├── App.css  
│   ├── App.js  
│   ├── App.test.js  
│   ├── index.css  
│   ├── index.js  
│   ├── logo.svg  
│   └── setupTests.js  
├── .gitignore  
├── package.json  
└── package-lock.json
```

— README.md

Temos alguns arquivos importantes nessa estrutura.

- **App.js:** serve como o ponto de partida para a construção da interface do usuário. O App.js pode conter a estrutura principal da aplicação, e é comum que outros componentes sejam importados e utilizados dentro dele.
- **.gitignore:** define quais arquivos e pastas devem ser ignorados pelo Git. Isso evita que arquivos indesejados ou desnecessários, como node_modules ou arquivos de configuração local, sejam versionados no repositório.
- **package.json:** contém informações sobre o projeto, incluindo nome, versão, descrição e, principalmente, as dependências do projeto. O package.json também pode definir scripts que ajudam a automatizar tarefas comuns (como start, build, test).
- **package-lock.json:** é gerado automaticamente pelo npm para registrar o estado exato das dependências do projeto. Ele assegura que as instalações subsequentes sejam consistentes com as versões exatas que foram originalmente instaladas, ajudando a evitar problemas de compatibilidade.
- **manifest.json:** contém metadados sobre o aplicativo, úteis para aplicações web progressivas (PWAs). Ele pode definir informações como o nome do aplicativo, ícones para diferentes tamanhos de tela, cores de tema, e comportamento do aplicativo quando "instalado" na tela inicial de um dispositivo móvel. Isso melhora a integração do aplicativo com dispositivos móveis e navegadores modernos.

(AVANÇASP/CM Sorocaba/2022) Considere que foi criado um projeto padrão em react utilizando o create-react-app. Onde, por padrão, ficarão armazenadas todas as dependências de bibliotecas utilizadas?

- a) Na pasta src.
- b) No arquivo .gitignore
- c) No arquivo package.json
- d) No arquivo index.
- e) Na pasta node_modules.

Comentários:

A pasta que armazena as dependências é a pasta node_modules. (Gabarito: Letra E)

JSX

Já vimos um pouco sobre - mas vamos aprofundar mais agora. O **JSX**, ou **JavaScript XML**, é uma extensão de sintaxe para JavaScript que é comumente usada com a biblioteca React para criar interfaces de usuário. Ele permite que você escreva código que se assemelha ao HTML dentro de arquivos JavaScript.

Embora JSX se pareça com HTML, ele é, na verdade, JavaScript. O código JSX é transformado em chamadas de função JavaScript por um compilador (geralmente Babel), que converte o JSX em funções `React.createElement`. Essa função cria e retorna um elemento React, que é um objeto descrevendo o que deve ser exibido na interface de usuário.

Quando vamos criar um elemento com o `React.createElement`, devemos passar três argumentos:

- Tipo de elemento
- Propriedades (`props`) do elemento
- Elementos filhos

Isso torna o processo um pouco mais trabalhoso. Por exemplo, poderíamos ter o seguinte código em JavaScript:

JavaScript

```
const element = React.createElement('h1', null, 'Olá, mundo!');
```

Nesse caso, definimos a criação de um `<h1>`, sem *props*, isso é, sem dados compartilhados entre elementos pai e filho, e com o conteúdo 'Olá, mundo!'. Veja outra comparação entre a criação de um elemento qualquer no JavaScript tradicional, e no JSX. Abaixo, criaremos um título `<h1>`, com o texto "Olá, mundo!" das duas formas.

JavaScript

```
//JSX
const element = <h1>Olá, mundo!</h1>;

//JavaScript
const element = document.createElement('h1');
element.textContent = 'Hello, world!';
```

Podemos exportar essa constante criada, chamada de `element`, como um componente - para isso, transformaríamos sua sintaxe numa função, da seguinte forma:

JavaScript

```
export default function element() {  
  return (  
    <h1>  
      Olá, mundo!  
    </h1>  
  )  
}
```

Uma grande vantagem é que o JSX permite a inclusão de **expressões JavaScript** dentro do código HTML. Para isso, usamos um par de chaves {}, indicando se tratar de uma sintaxe JavaScript (mesmo estando escrevendo o código num arquivo .js). Digamos que, ao invés de escrever "Olá, mundo!", queremos substituir o "mundo" pelo nome do usuário - nesse caso, Joãozinho.

JavaScript

```
let user = "João"  
  
export default function element() {  
  return (  
    <h1>  
      Olá, { user }  
    </h1>  
  )  
}
```

(FCC/PGE AM/2022) Considere que um desenvolvedor está criando um aplicativo usando React e React Native e deseja criar um elemento h1 contendo o título Amazonas, aplicando a classe de estilo CSS de nome tit e armazenando em uma constante chamada elemento. Para realizar esta tarefa, ele terá que utilizar a instrução

- a) let elemento = ReactNative.createElement('h1',{className:'tit'},'Amazonas');
- b) let const elemento = React.createElement('h1'.tit'),'Amazonas');
- c) const elemento = ReactNative.createElement('h1',{className:'tit'},'Amazonas');
- d) const elemento = React.createElement('h1',{className:'tit'},'Amazonas');
- e) const elemento = React.createElement('h1'.tit,'Amazonas');

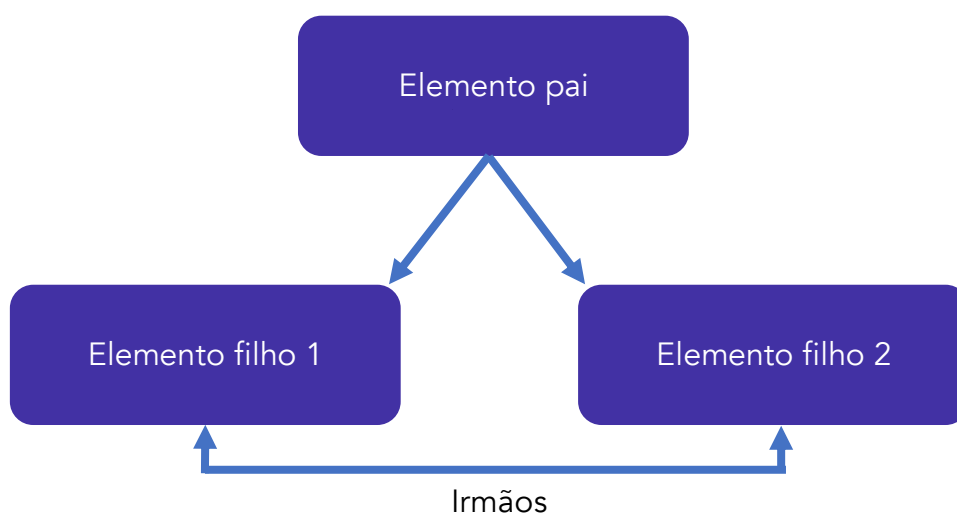
Comentários:

A questão quer que criemos um elemento em JavaScript "tradicional". Para isso, devemos definir uma const e utilizar o React.createElement, passando como parâmetro o tipo de elemento (h1), as props (className="tit") e o conteúdo ('Amazonas'). Ficamos então com `const elemento = React.createElement('h1', {className: 'tit'}, 'Amazonas')`. (Gabarito: Letra D)

Props

Uma das grandes dificuldades em *frameworks* baseados em componentes é compartilhar parâmetros, dados e propriedades entre diferentes componentes. Nesse sentido, temos um conceito utilizado em React que visa facilitar esse trabalho - as **props** (abreviação de *properties*). No react, temos 2 “relações” entre componentes:

- Pai/filho, trazendo uma relação vertical e hierárquica
- Irmãos (*siblings*), trazendo uma relação horizontal



O React incentiva o uso de um fluxo unidirecional de passagem de dados. Por isso, a forma mais comum e “padrão” na passagem de propriedades acontece de um elemento pai → elemento filho. Veja um exemplo de dois diferentes componentes interagindo:

```
JavaScript
export default function ComponenteFilho(props) {
  return (
    <div>
      <p> Aluno aprovado: {props.aluno} </p>
    </div>
  )
}
```

Acima, criamos um componente filho que utiliza a propriedade `props.aluno`. Essa propriedade foi herdada da classe pai - vamos criá-la.

JavaScript

```
import ComponenteFilho from './ComponenteFilho.jsx'

export default function ComponentePai() {
  let aluno = 'João da Silva'

  return (
    <div>
      <h1> Estratégia Concursos </h1>
      <ComponenteFilho aluno={aluno} />
    </div>
  )
}
```

Ao criarmos o componente pai, importamos os componentes filhos - quase que numa herança inversa. Enquanto o componente filho `ComponenteFilho` utilizou a propriedade `props.aluno`, o componente pai `ComponentePai` define o que é esse aluno. No caso, ele é uma variável que recebe o nome 'João da Silva'. Ao chamarmos o componente, associamos a variável criada no componente pai à propriedade, na chamada do componente filho - portanto, ali em `<ComponenteFilho aluno={aluno}/>`.

QUESTÃO DE PROVA



(CPCON UEPB/UEPB/2021) Considere o seguinte trecho de código Reactjs:

```
class HelloMessage extends React.Component {
  render() {
    return (
      <div>
        Hello {this.props.name}
      </div>
    );
  }
}
```

```
}  
ReactDOM.render(  
  <HelloMessage name="Taylor" />,  
  document.getElementById('hello-example')  
);
```

Componentes React implementam um método `render()` que recebe dados de entrada e retorna o que se quer mostrar. Baseado nisso, o resultado da execução do código acima é

- a) Taylor.
- b) Hello Taylor.
- c) name=Taylor.
- d) hello-example.
- e) Hello name.

Comentários:

A questão aborda alguns conceitos que serão vistos - mas o cerne da questão gira em torno das props. No componente, temos esse trecho de código:

```
<div>  
  Hello {this.props.name}  
</div>
```

E posteriormente, quando vamos renderizar o componente, passamos a seguinte estrutura:

```
<HelloMessage name="Taylor" />,
```

Isso quer dizer que estamos associando o valor "Taylor" à propriedade `name`. Então, quando digitamos `Hello {this.props.name}`, estamos, na realidade, digitando `Hello Taylor`. Portanto, correta a letra B. (Gabarito: Letra B)

Já a comunicação entre irmãos não ocorre de forma direta. É possível compartilhar dados entre componentes irmãos através de um pai em comum, mas **não é possível passar diretamente as propriedades entre dois irmãos**.

Componente de Classe

Vimos uma forma “básica” para criarmos componentes em React. Existe uma outra forma, um pouco mais arcaica, e particularmente útil quando queremos acessar o estado interno de cada componente. Nesses casos, fazemos os chamados **componentes de classe**. Essa abordagem acabou caindo em desuso graças à criação dos *hooks*, assunto que veremos logo em seguida, que permitem o acesso ao estado dos componentes criados através de funções.

Vamos ver um exemplo de criação de componente de classe:

```
JavaScript

import React, { Component } from 'react';

class MyComponent extends Component {
  render() {
    return (
      <div>
        <h1>Hello, World!</h1>
      </div>
    );
  }
}

export default MyComponent;
```

Veja que estamos importando o elemento `{ Component }` do React - um elemento padrão, da própria biblioteca do React. Em seguida, através da herança (lá da orientação a objetos), fazemos uma extensão do `Component`, criando um componente próprio, o `MyComponent`.

Temos diversos métodos que podem ser usados na classe que estende o `Component`. Porém, apenas uma delas é obrigatória: o `render()`. Ele retorna o JSX que define a interface de usuário do componente, trazendo para o navegador o que deve ser renderizado.

QUESTÃO DE PROVA



(FCC/TJ CE/2022) Para criar um componente React chamado App, uma Analista usou o código abaixo.

```
class App extends React.Component {  
  
}
```

O único método necessário dentro desta classe, que recebe os dados de entrada e retorna o que deve ser exibido, é o componente

- a) showApp()
- b) root()
- c) createApp()
- d) handler()
- e) render()

Comentários:

Questão tranquila. O único método necessário (obrigatório) dentro de uma classe de componente é o método render(). (Gabarito: Letra E)

Temos outros métodos que podem ser usados nas classes:

Método	Descrição
render()	Retorna o JSX que define a interface de usuário do componente
constructor(props)	Inicializa o estado e vincula métodos
componentDidMount()	Chamado imediatamente após o componente ser montado
componentDidUpdate()	Chamado imediatamente após uma atualização no DOM
componentWillUnmount()	Chamado imediatamente após o componente ser destruído
componentDidCatch()	Chama após lançamento de erro por um componente filho

Renderização de Lista

Existem casos em que é interessante realizar uma renderização repetitiva de elementos. Por exemplo, os *cards* de vídeo no YouTube - já percebeu que todos eles são iguais? Todos contêm a *thumbnail*, o título do vídeo, e outros elementos repetitivos. Aqui está a chave: elementos repetitivos.

Não há necessidade de alocar elemento por elemento na página. Para esses casos, usamos uma abordagem chamada de **renderização de lista**. Digamos que estamos criando cards com os 10 primeiros colocados em um concurso público - e cada card será composto de uma foto do aluno, seu nome e uma frase de efeito.



Imagine ter que criar um elemento para cada *card* que quisermos implementar. Para 4, 5 *cards* ok, é um trabalho pequeno - mas podemos ter centenas deles, no que chamamos de "carrossel infinito". Para "facilitar" esse trabalho, podemos:

- Organizar os dados de cada card numa estrutura de array de objetos (ou um arquivo JSON), onde cada objeto terá propriedades como foto, nome e frase.
- Realizar um mapeamento (`map()`) desse objeto para renderizar cada card autonomamente. Alternativamente, se quisermos filtrar por algum valor, por exemplo um valor para "aprovado", poderíamos usar o `filter()`.

Vou criar um objeto fictício aqui, com 5 alunos aprovados distintos. Veja como fica o objeto em JavaScript.

JavaScript

```
const objetos = [  
  {  
    imagem: './01.jpg',  
    nome: 'Alice Santos',  
    frase: 'Acredite em si mesmo.'  
  },  
  {  
    imagem: './02.jpg',  
    nome: 'Bruno Oliveira',  
    frase: 'O sucesso é uma jornada.'  
  },  
  {  
    imagem: './03.jpg',  
    nome: 'Carla Mendes',  
    frase: 'Inspire-se diariamente.'  
  },  
  {  
    imagem: './04.jpg',  
    nome: 'Diego Lima',  
    frase: 'A persistência é a chave.'  
  },  
  {  
    imagem: './05.jpg',  
    nome: 'Elaine Costa',  
    frase: 'Sonhe grande, realize mais.'  
  }  
];
```

Agora, podemos navegar nas propriedades de cada elemento dessa array. Vamos criar um componente para o card:

JavaScript

```
export default function Card({imagem, nome, frase}) {  
  return (  
    <div className="card">  
      <img src={imagem} />  
      <h1>{nome}</h1>  
      <p>{frase}</p>  
    </div>  
  )  
}
```

Veja que criamos o componente para 1 card, com sua estrutura em JSX. Agora, precisamos criar uma seção que irá conter esses cards. Vou chamá-lo de CardList. Nele consolidamos todos os valores que geramos até agora - os dados da array objetos, e o componente Card.

JavaScript

```
import { objetos } from './data'
import Card from './Card'

export default function CardList() {
  return (
    <div className="card-list">
      {objetos.map((objeto, index) => (
        <Card
          key={index}
          imagem={objeto.imagem}
          nome={objeto.nome}
          frase={objeto.frase} />
      ))}
    </div>
  )
}
```

Veja que objetos é um arquivo JavaScript, então ele deve ser importado com o par de chaves {} para indicar que estamos trabalhando com script. Além disso, a demarcação do par de chaves indicando que estamos inserindo a função map() dentro do código.

Lembrando que isso é possível devido à herança vertical. O componente pai, o CardList, ele herda todas as informações dos seus elementos filhos, agregando-as. Com isso temos acesso a tudo que queremos num só componente consolidado.

Essa abordagem é interessante quando trabalhamos com uma API externa. Podemos importar os dados em um arquivo JSON, fazer um parse para converter a estrutura em uma array de objetos, e tratar com os objetos num arquivo atualizado dinamicamente por terceiros. Inclusive, esse é o ponto “chave” para contratação em muitos processos seletivos para vagas de desenvolvedores júnior - se o dev souber lidar com requisições para APIs e renderização em lista, ele é considerado apto a tomar posse num cargo de desenvolvedor júnior front-end.

Hooks

Hooks são **funções especiais** no React que permitem que você "conecte" estados e funcionalidades de ciclo de vida em componentes funcionais. Antes da introdução dos Hooks na versão 16.8, esses recursos estavam disponíveis apenas em componentes de classe. Com os Hooks, você pode usar estados e outros recursos do React sem escrever uma classe.

Temos vários *hooks* disponíveis. Os principais são:

- **useState**: permite adicionar estado local a componentes funcionais.
- **useEffect**: realiza efeitos colaterais em componentes funcionais, como buscar dados, alterar o DOM, ou configurar subscrições.
- **useContext**: permite acessar o valor de um contexto React sem a necessidade de um componente consumidor.
- **useReducer**: alternativa ao useState para gerenciamento de estados mais complexos. Utiliza um redutor para manipular o estado com base em ações.
- **useCallback**: memoriza funções, retornando uma versão memorizada de uma função que só muda se uma das dependências mudar.
- **useMemo**: memoriza valores calculados, retornando um valor memorizado que só muda se uma das dependências mudar.
- **useRef**: permite criar referências mutáveis que persistem entre renderizações. Pode ser usado para acessar elementos DOM diretamente.
- **useImperativeHandle**: personaliza a instância de um ref exposto a componentes pais ao usar refs.
- **useLayoutEffect**: semelhante ao useEffect, mas é executado de forma síncrona após todas as mutações do DOM. Útil para ler o layout do DOM e re-renderizar de forma síncrona.
- **useDebugValue**: permite adicionar uma etiqueta de depuração a hooks customizados.

São inúmeros *hooks* e é praticamente impossível abordarmos todos eles em profundidade numa só aula. Vamos focar nos mais importantes, e que possuem mais chance de serem cobrados na sua prova: `useState`, `useEffect`, `useContext`, `useReducer` e `useCallback`.

useState

O hook **useState** é utilizado para **adicionar um estado local variável a um componente**. Como qualquer *hook*, devemos importá-lo ao componente e chamá-lo no topo do componente, para inicializá-lo logo no começo do componente. Esse *hook* envolve o uso de duas variáveis distintas: uma variável para armazenar o estado inicial, e outra para armazenar o estado modificado. Por padrão, convencionou-se em usar pares de nome para essas variáveis como `estado` e `setEstado`.

Imagine uma seguinte aplicação: um botão que conta a quantidade de vezes que foi apertado. Ele terá um estado inicial, o valor do contador zerado, e um estado que é alterado a cada clique.

```
JavaScript

import React, { useState } from 'react';

export default function Counter() {
  //definição do hook useState
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Você clicou {count} vezes</p>
      <button onClick={() => setCount(count + 1)}>
        Clique aqui
      </button>
    </div>
  );
}
```

Quando definimos `const [count, setCount] = useState(0)` estamos:

- **Definindo uma variável inicial**, chamada de `count`, que recebe um valor inicial ao inicializar o componente. No caso, o valor é definido na função `useState()`, e é 0. Em caso de reinicializações do componente, o estado é “resetado” e volta ao valor definido.
- **Definindo uma variável que irá armazenar as mudanças de estado**, chamada de `setCount`. Toda vez que iniciamos o componente, `count = setCount`.

Acontece que o *framework* não interpreta duas variáveis distintas - elas servem apenas para organizar a memória dos valores. A variável mesmo, que poderá ser chamada em outros pontos do código, será a primeira denominada dentro do par de colchetes - no nosso caso, a `count`. É como se tivéssemos um estado inicial e um ponteiro, apontado para uma variável que armazena as mudanças de estado (o `setState`).

Por esse motivo, sempre que quisermos apontar a quantidade de cliques no nosso exemplo, ou quando quisermos acessar o valor do estado, devemos usar o valor da variável criada - `{ count }` - e não da variável responsável por lidar com o estado. Porém, alterações de estado devem ser direcionados à variável `setCount`. Por isso a função `onClick` atualiza o `setCount`, e no *display* da contagem utilizamos o `{ count }`.

ATUALIZAÇÕES DE ESTADO → VARIÁVEL ATUALIZÁVEL
ACESSO AO VALOR ATUAL DO ESTADO → VARIÁVEL BASE

(CESGRANRIO/BB/2023) O React Native 0.59 introduziu o conceito de Hooks.

Entre os Hooks, tem-se o useState, que permite

- a) calcular o estado de um CEP ou ZIP de acordo com o Locale.
- b) chamar estados específicos do engine React para alterar seu comportamento.
- c) declarar uma classe que segue o padrão de design state.
- d) criar uma enumeration que representa estados.
- e) manter um estado local em uma função de um componente funcional.

Comentários:

Questão tranquila. O useState permite manter um estado local, em função de um componente funcional (ou seja, um componente criado a partir de uma função). (Gabarito: Letra E)

Um caso muito útil para o uso do useState são em campos de entrada. Inicializamos a variável de estado como uma string vazia, e a cada interação (apertar de caractere) o valor dela é atualizado. Assim fica mais fácil submeter o valor da variável ao banco de dados.

useEffect

O hook **useEffect** permite a **execução de efeitos colaterais em componentes funcionais**. Efeitos colaterais podem incluir operações como buscar dados, configurar subscrições, ou manipular o DOM diretamente. O **useEffect** combina a funcionalidade dos métodos de ciclo de vida **componentDidMount**, **componentDidUpdate**, e **componentWillUnmount** dos componentes de classe.

O uso do useEffect é um pouco mais "chatinho". Veja uma sintaxe base para o comando:

JavaScript

```
useEffect(() => {  
  // Código do efeito  
  return () => {  
    // Código de limpeza opcional  
  };  
}, [dependências]);
```

Digamos que você quer atualizar o valor do título da página, no nosso exemplo anterior do contador, a cada clique. Ou seja, na *tab* do navegador, irá aparecer um título "Você clicou *n* vezes". Precisamos de uma forma de atualizar esse *n* com o valor que temos armazenados no **useState** - mas não podemos utilizar o valor do estado diretamente, já que o título não é um elemento automaticamente atualizável. Para isso, usamos o **useEffect**. Então, de forma geral,

utilizamos o `useEffect()` quando precisamos atualizar esses elementos que não são automaticamente atualizáveis. Esse tipo de atualização é chamado de “efeito colateral”.

Veja em código como funcionaria nosso exemplo:

```
JavaScript

import React, { useState, useEffect } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Você clicou ${count} vezes`;
  }, [count]); // O efeito só é re-executado quando `count` muda

  return (
    <div>
      <p>Você clicou {count} vezes</p>
      <button onClick={() => setCount(count + 1)}>
        Clique aqui
      </button>
    </div>
  );
}
```

Então, sempre que tivermos uma mudança de valor em `useState`, o `useEffect` é chamado e executado, atualizando o valor de estado nos lugares alocados. No nosso caso, no `document.title` (título do documento).

Um dos casos mais comuns de uso do `useEffect` é em interações com APIs. Ele permite buscar dados de uma API sempre que as dependências especificadas forem atualizadas. Ao monitorar mudanças no estado ou nas propriedades, o `useEffect` pode garantir que a aplicação de página única (SPA) seja mais interativa e dinâmica, atualizando os dados automaticamente com base nas mudanças de estado.

ESTA É DIFÍCIL!



(PRGP UNIFEI/UNIFEI/2022) Observe o código JSX abaixo. Os dados advindos da url serão requisitados quantas vezes?

```
import React, { useState, useEffect } from "react";

export default function Usuarios() {
  const [usuarios, setUsuarios] = useState([]);
  const url = "https://jsonplaceholder.typicode.com/users";

  useEffect(() => {
    fetch(url)
      .then((resp) => resp.json())
      .then((resp) => setUsuarios(resp))
      .catch((err) => console.log(err));
  }, [url]);

  return (
    <div>
      {usuarios.map((item) => {
        return (
          <div key={item.id}>
            <span>{item.name} -- </span>
            <span>{item.email} </span>
          </div>
        );
      })}
    </div>
  );
}
```

- a) Todas as vezes que o estado da variável usuarios mudar.
- b) Uma vez.
- c) O número de vezes que o map iterar sobre o array usuarios.
- d) Nenhuma vez.

Comentários:

Numa primeira análise, por termos `useState` e `useEffect` você até pode pensar “ah, vamos fazer um *fetch* toda vez que atualizarmos o estado da variável usuarios”, já que é ela que está sendo lida no `useState`. Porém o `useEffect` não utiliza nenhuma variável de estado atualizável - ela está utilizando a constante url, que é definida no começo do código e nunca modificada. Por esse motivo, só iremos executar o `useEffect` uma única vez, ao carregarmos a página. Excelente questão. (Gabarito: Letra B)

useContext

O hook `useContext` é utilizado para **consumir um contexto criado com `React.createContext`**. Ele fornece uma maneira de compartilhar valores entre componentes sem precisar passar explicitamente propriedades através de cada nível da árvore de componentes.

Isso é particularmente útil para dados que são usados por muitos componentes, como o tema da aplicação, a autenticação do usuário, ou configurações globais. Lembram que tínhamos problemas na comunicação horizontal de *props*, né? O `useContext()` resolve esse pequeno empasse.

Para utilizarmos o `useContext`:

- Criamos um contexto, com o `React.createContext`. Esse contexto é armazenado num tipo de componente provedor, chamado de **provider**.
- Esse componente provider envolve todos os componentes que precisam acessar os valores do contexto. Esse componente recebe uma propriedade `value` que contém os dados a serem compartilhados.
- Os componentes que precisam consumir o contexto usam o **hook `useContext`**.

Para consumir um contexto, temos uma abordagem muito parecida com o `useState`. Criamos duas variáveis, a variável base e a atualizável, e associamos o valor do contexto à expressão. Ficaremos com algo como `const [contexto, useContexto] = useContext(Provider)`, onde o `Provider` é o contexto que queremos utilizar.

useCallback

O hook `useCallback` é utilizado para memorização de funções. Ele retorna uma versão memorizada da função que só é recriada quando as dependências especificadas mudam. Isso

pode melhorar o desempenho ao evitar re-renderizações desnecessárias de componentes que dependem dessas funções, especialmente quando essas funções são passadas como props para componentes filhos.

Imagine um componente que tem uma função para incrementar um contador e passa essa função para um componente filho:

```
JavaScript

export default function Parent() {
  const [count, setCount] = useState(0);

  const increment = useCallback(() => {
    setCount(count + 1);
  }, [count]);

  return (
    <div>
      <p>Contagem: {count}</p>
      <Child increment={increment} />
    </div>
  );
}

function Child({ increment }) {
  console.log('Child renderizado');
  return <button onClick={increment}>Incrementar</button>;
}
```

Neste exemplo, toda vez que o componente Parent é re-renderizado, a função increment é recriada. Se não tivéssemos o useCallback, o componente filho seria re-renderizado mesmo que não houvesse alterações, causando um excesso de consumo de banda e memória.

(FGV/TJ SE/2023) O analista José está implementando a interface gráfica da aplicação web JusticaSE, utilizando a biblioteca JavaScript React. A JusticaSE possui uma tela de dashboard, implementada pelo componente TelaD, que demanda alto custo computacional para ser renderizada. A fim de reduzir o custo de renderização da tela de dashboard, José encapsulou determinadas functions de TelaD em um Hook nativo da biblioteca React. O Hook utilizado por José memoriza a definição de uma function ao longo de várias renderizações de um componente. José utilizou o React Hook:

- a) useState;
- b) useEffect;
- c) useMemo;
- d) useCallback;

e) useContext.

Comentários:

Queremos um *hook* que irá encapsular as funções, e memorizar sua definição ao longo de diversas renderizações. A função responsável por isso é a useMemo. (Gabarito: Letra C)

Router

O React foi criado para criar as *Single Page Applications*, ou SPAs. Porém, essas aplicações teriam uma única página inicialmente, ficando demasiadamente simples. Para casos em que é necessário implementar múltiplas páginas numa mesma SPA, utilizamos o **React Router**. Pode soar estranho, mas na verdade não estamos criando “páginas” novas, e sim componentes novos que acabam atuando como uma nova seção na aplicação.

O Router é uma nova biblioteca, autônoma em relação ao React padrão. Por esse motivo, ele deve ser importado separadamente, assim como seus componentes. Usualmente, o importamos da seguinte forma:

JavaScript

```
import {  
  createBrowserRouter,  
  RouterProvider,  
  Route,  
  Link,  
} from "react-router-dom";
```

Aqui é interessante que você saiba o funcionamento de cada componente:

- **BrowserRouter:** um componente de roteador que usa a API de histórico HTML5 para gerenciar a navegação e as URLs no navegador.
- **HashRouter:** um componente de roteador que usa o hash da URL (parte após o símbolo #) para gerenciar o roteamento.
- **Route:** define um mapeamento entre um caminho de URL e um componente. Quando o caminho da URL corresponde ao path especificado, o componente é renderizado.
- **Link:** componente que cria links de navegação dentro da aplicação. Substitui o comportamento padrão dos links `<a>`, evitando recarregamentos de página.
- **NavLink:** similar ao Link, mas com suporte para estilos ativos, permitindo que você estilize o link com base na rota ativa.
- **Switch:** envolve um conjunto de Route e renderiza apenas o primeiro Route que corresponde ao caminho da URL. Usado para garantir que só uma rota seja renderizada por vez.
- **Redirect:** redireciona programaticamente para uma nova rota. É útil para redirecionar usuários após uma ação, como o login.

Para criar um aplicativo com o Router, envolvemos todos os componentes do arquivo de montagem da página, onde elencamos todos os componentes. Veja um exemplo em código de como ficaria um aplicativo com o Router:

JavaScript

```
export default function App() {  
  return (  
    <Router>  
      <div>  
        <Link to="/">Home</Link>  
        <Link to="/about">About</Link>  
  
        <Switch>  
          <Route path="/" exact component={Home} />  
          <Route path="/about" component={About} />  
          <Route component={NotFound} />  
        </Switch>  
      </div>  
    </Router>  
  );  
}
```

(CONSULPLAN/SEED PR/2022) React é uma biblioteca JavaScript que simplifica e agiliza a tarefa de desenvolvimento de interfaces de usuário interativas e de alto desempenho. Sobre tal biblioteca, assinale a afirmativa correta.

- a) Pelo fato de ser uma biblioteca robusta, React é indicado somente para projetos de grande porte.
- b) Em geral, para usar uma funcionalidade React em um script, não é necessário importá-la para dentro do script.
- c) Pelo fato do objeto props ser do tipo somente leitura, não é possível alterar os valores passados aos componentes filho com uso de props.
- d) Router e Route são componentes que possuem funcionalidades idênticas. Ambos fazem parte da biblioteca React Router e são usados em um projeto de rotas (links).

Comentários:

Vamos analisar cada alternativa.

- a) Errado. O React é indicado para qualquer tamanho de projeto - embora não seja tão aconselhado para grandes projetos por limitações do *framework*.
- b) Errado. Precisamos importar o React para utilizá-lo.
- c) Certo. O *props* de fato é somente leitura nos componentes filho. Nesse caso, não podemos alterar seu valor.
- d) Errado. Router é o componente que envolve nosso projeto, e Route são formas de criação de caminhos. Ambos pertencem ao React Router.

Assim, ficamos com o gabarito na letra C. (Gabarito: Letra C)

QUESTÕES COMENTADAS

01. (FGV/CGE PB/2024) Considere o seguinte código em JavaScript com React:

```
import React from 'react';

function Number() {
  let x = 5, y = 5;
  const [n, setN] = React.useState(x);

  React.useEffect(() => {
    setN(x => x + n);
    setN(y => x + y);
  }, [])

  return n;
}
```

Ao se utilizar o componente funcional Number, o valor retornado por Number após a renderização final será:

- a) 5;
- b) 10;
- c) 15;
- d) 20;
- e) 55.

Comentários:

Estamos com um componente utilizando useState e useEffect. O useEffect é chamado sempre que algum de seus elementos "monitorados" tem seu estado alterado. Quando rodamos o React.useState(x), estamos atribuindo o valor de x para a variável n - criando uma alteração de estado e chamando o useEffect.

Nesse caso, teremos duas interações:

- $x = x + n \rightarrow x = 5 + 5 = 10$
- $y = x + y \rightarrow y = 5 + 5 = 10$

"Mas Felipe, por que usamos $x = 5$ na segunda equação, se ele foi atualizado para 10 na primeira?"

É uma particularidade do React. O valor de x dentro da função de setN ainda é 5, pois setN não atualiza o valor de x, ele apenas usa o valor antigo para a segunda chamada - ele só atualiza o valor de n (ou setN). Nesse caso, a segunda operação irá sobrescrever a primeira, e alocar o valor 10 dentro de n - sendo esse o retorno do programa.

Gabarito: Letra B

02. (FGV/ALEP/2024) Sobre a biblioteca de Javascript React, assinale a afirmativa **incorreta**.

- a) React permite que a interface do usuário seja construída por meio de componentes reutilizáveis
- b) React manipula os objetos DOM reais do navegador diretamente, melhorando o desempenho das atualizações na interface do usuário.
- c) React Props são como argumentos de função em JavaScript e atributos em HTML
- d) JSX facilita a escrita e a adição de HTML no React
- e) o React renderiza HTML para a página da web usando uma função chamada createRoot() e seu método render()

Comentários:

Vamos analisar cada alternativa.

- a) Certo. É exatamente esse o core do React.
- b) Errado. O React utiliza um DOM virtual, não o DOM real.
- c) Certo. As *props* funcionam como argumentos de uma função, que são passados aos elementos filho no modo somente leitura.
- d) Certo. JSX facilita a escrita por permitir digitar quase que um HTML puro dentro de *scripts* em JavaScript.
- e) Certo. O createRoot() cria uma raiz para exibir os componentes React dentro do DOM, e o render() insere esses componetnes no DOM.

Portanto, a alternativa incorreta é a letra B.

Gabarito: Letra B

03. (CESGRANRIO/BB/2023) O React Native 0.59 introduziu o conceito de Hooks.

Entre os Hooks, tem-se o usestate, que permite

- a) calcular o estado de um CEP ou ZIP de acordo com o Locale.
- b) chamar estados específicos do engine React para alterar seu comportamento.
- c) declarar uma classe que segue o padrão de design state.

- d) criar uma enumeration que representa estados.
- e) manter um estado local em uma função de um componente funcional.

Comentários:

Apesar da pergunta ser sobre o React Native, a funcionalidade do hook useState é a mesma do React - manter um estado local em uma função, dentro de um componente funcional. Portanto, correta a letra E.

Gabarito: Letra E

04. (AVANÇASP/PREF. AMERICANA/2023) Um arquivo React é conhecido por ser um aplicativo de página única é uma aplicação web ou site que consiste de uma única página web (só tem um html), essa característica é conhecida como:

- a) FDA.
- b) SPA.
- c) APE.
- d) PUN.
- e) UPG.

Comentários:

Questão tranquila. Aplicações de página única são chamadas de SPA, ou Single Page Application.

Gabarito: Letra B

05. (AVANÇASP/PREF. AMERICANA/2023) Para criar um projeto padrão em react, chamado projetoteste, a linha de comando correta é

- a) npx-create-app-projetoteste
- b) projetoteste-npx-create
- c) npm create-projetoteste
- d) npx create-react-app projetoteste
- e) npx -app-projetoteste.

Comentários:

A questão exige conhecimento acerca da forma depreciada para criação de aplicativos React, usando o componente create-react-app. Nesse caso, a sintaxe para a criação de um aplicativo com o nome projetoteste seria `npx create-react-app projetoteste`.

Gabarito: Letra D

06. (AVANÇASP/PREF. AMERICANA/2023) No React um conceito de programação onde uma representação ideal, ou “virtual”, da interface do usuário é mantida em memória e sincronizada com o DOM “real” por uma biblioteca como o ReactDOM é conhecido como:

- a) DOMReact.
- b) ReactDOM.
- c) DOM
- d) DOMPrincipal
- e) VirtualDOM

Comentários:

O recurso citado pela questão é o VirtualDOM. O virtual DOM (VDOM) é um conceito de programação onde uma representação ideal, ou “virtual”, da interface do usuário é mantida em memória e sincronizada com o DOM “real” por uma biblioteca como o ReactDOM. Esse processo é chamado de reconciliação.

Gabarito: Letra E

07. (CEBRASPE/SERPRO/2023) O React é compatível com outras bibliotecas e frameworks, como o Redux e o AngularJS, e pode ser integrado a essas tecnologias. Em relação aos procedimentos necessários para realizar uma integração do React com o AngularJS, julgue o item subsequente.

Para permitir a integração entre os frameworks React e AngularJS, um passo a ser executado é criar um componente React e renderizá-lo dentro desse elemento, usando-se o método ReactDOM.render().

Comentários:

Pense que, para integrarmos o React ao Angular, precisamos que os componentes criados sejam listados no DOM da aplicação. Isso é garantido a partir da utilização da função render(), possibilitada pelo ReactDOM. Portanto o item está correto.

Gabarito: Certo

08. (FGV/TCE SP/2023) A analista Ana está desenvolvendo a interface gráfica da aplicação web TCEDash, utilizando React com componentes funcionais. Ana precisa que o componente TelaUm, do TCEDash, execute determinada sequência de instruções a cada nova renderização. Porém, a sequência de instruções deve ser garantidamente executada antes de o navegador web atualizar visualmente a tela.

Para garantir a execução antes da atualização visual da tela, Ana deve implementar a sequência de instruções dentro do hook do React:

- a) `useEffect`;
- b) `useTransition`;
- c) `useLayoutEffect`;
- d) `useDeferredValue`;
- e) `useImperativeHandle`.

Comentários:

Queremos um *hook* que irá executar uma série de instruções a cada renderização, antes do visual da página ser gerado. Para isso, o uso indicado é do `useLayoutEffect`. Ele é uma versão do `useEffect` destinada a realizar o "efeito" antes da implementação do layout.

Gabarito: Letra C

09. (FGV/TJ SE/2023) O analista José está implementando a interface gráfica da aplicação web JusticaSE, utilizando a biblioteca JavaScript React. A JusticaSE possui uma tela de dashboard, implementada pelo componente TelaD, que demanda alto custo computacional para ser renderizada. A fim de reduzir o custo de renderização da tela de dashboard, José encapsulou determinadas functions de TelaD em um Hook nativo da biblioteca React. O Hook utilizado por José memoriza a definição de uma function ao longo de várias renderizações de um componente. José utilizou o React Hook:

- a) `useState`;
- b) `useEffect`;
- c) `useMemo`;
- d) `useCallback`;
- e) `useContext`.

Comentários:

Queremos um hook que irá encapsular as funções, e memorizar sua definição ao longo de diversas renderizações. A função responsável por isso é a `useMemo`.

Gabarito: Letra C

10. (FCC/TRT 12/2023) Um Analista deseja realizar mudanças nos dados de um componente e atualizar a interface do usuário de acordo com essas mudanças. No React, a sintaxe correta do método usado para atualizar o estado de um componente é:

- a) `this.componentState( novo estado: novo valor );`
- b) `this.updateState(( novo estado: novo valor );`
- c) `this.setState({ novo estado: novo valor });`
- d) `this.changeState({ novo estado: novo valor });`
- e) `this.modifyState({ novo estado: novo valor });`

Comentários:

Questão tranquila. Para atualizarmos o estado de um componente, tratando-se de um componente de classe (e não um componente funcional, onde usaríamos o `setState`), usamos uma função de mesmo nome que o hook - a `setState()`. No caso, usamos `this` para referenciar o componente atual, ficando com a sintaxe `this.setState({ estado desejado })`.

Gabarito: Letra C

11. (FCC/PGE AM/2022) Considere que um desenvolvedor está criando um aplicativo usando React e React Native e deseja criar um elemento `h1` contendo o título Amazonas, aplicando a classe de estilo CSS de nome `tit` e armazenando em uma constante chamada `elemento`. Para realizar esta tarefa, ele terá que utilizar a instrução

- a) `let elemento = ReactNative.createElement('h1',{className:'tit'},'Amazonas');`
- b) `let const elemento = React.createElement('h1'.tit),'Amazonas');`
- c) `const elemento = ReactNative.createElement('h1',{className:'tit'},'Amazonas');`
- d) `const elemento = React.createElement('h1',{className:'tit'},'Amazonas');`
- e) `const elemento = React.createElement('h1'.tit,'Amazonas');`

Comentários:

A questão quer que criemos um elemento em JavaScript “tradicional”. Para isso, devemos definir uma `const` e utilizar o `React.createElement`, passando como parâmetro o tipo de elemento (`h1`), as `props` (`className="tit"`) e o conteúdo (`'Amazonas'`). Ficamos então com `const elemento = React.createElement('h1', {className: 'tit'}, 'Amazonas')`.

Gabarito: Letra D

12. (Legalle/CM POA/2022) Analise as seguintes afirmações sobre ReactJS:

- I. Trata-se de uma biblioteca JavaScript.
- II. Uso exclusivo para desenvolvimento de aplicações de back-end
- III. Não possibilita o desenvolvimento de aplicações single-page.

IV. O aptitude é um gerenciador que permite gerenciar as dependências de um projeto de pacotes do ReactJS.

Esta(ão) CORRETA(S):

- a) Apenas I.
- b) Apenas I e III.
- c) Apenas II e IV.
- d) Apenas I e IV.
- e) Apenas I, III e IV.

Comentários:

Vamos analisar cada item.

I. Certo. De fato, o React é uma biblioteca de JS.

II. Errado. O React faz interfaces de usuário - portanto, o *front-end*.

III. Errado. Pelo contrário, ele é voltado para aplicações single page.

IV. Errado. O npm que costuma fazer o gerenciamento de dependências. Aptitude é uma interface para sistemas Debian.

Portanto, somente o item I está correto.

Gabarito: Letra A

13. (IDECAN/UNILAB/2022) Sobre a biblioteca React, analise as afirmativas abaixo:

I. O React a quebra em pequenos pedaços, chamados de componentes, e os isola, sendo cada um deles independentes entre si perante a tecnologia.

II. Os componentes são reutilizáveis e, uma vez categorizados, podem ser reaproveitados em quaisquer páginas de um site ou aplicativo.

III. Dentro dos estados (states) estão as propriedades, ou props. Estas props são objetos com várias informações que podem ser de variados tipos, como função, número ou string.

IV. Assim como as propriedades, os states são imutáveis, como por exemplo variáveis declaradas dentro de uma função.

Assinale

- a) se apenas as afirmativas I e II estiverem corretas.
- b) se apenas as afirmativas II e III estiverem corretas.
- c) se apenas as afirmativas I e IV estiverem corretas.
- d) se apenas as afirmativas II e IV estiverem corretas.

Comentários:

Vamos analisar cada item.

I. Certo. Perfeita a definição de componentes.

II. Certo. Com categorizados, entenda "inseridos no DOM". E de fato, podem ser reaproveitados.

III. Errado. As *props* não fazem parte dos estados, e sim dos componentes.

IV. Errado. Pelo contrário, os states são mutáveis. As *props* que são imutáveis em componentes filhos.

Portanto, os itens I e II estão corretos.

Gabarito: Letra A

14. (FCC/TJ CE/2022) Para criar um componente React chamado App, uma Analista usou o código abaixo.

```
class App extends React.Component {  
  
}
```

O único método necessário dentro desta classe, que recebe os dados de entrada e retorna o que deve ser exibido, é o componente

- a) showApp()
- b) root()
- c) createApp()
- d) handler()
- e) render()

Comentários:

A sintaxe apresentada sugere a criação de um componente de classe no React. Quando criamos um componente de classe, um único método é obrigatório, responsável por retornar a interface que será gerada a partir desse componente - o método `render()`.

Gabarito: Letra E

15. (FCC/TRT 23/2022) Considerando que o `npx` e `Node.js` estão instalados, um Técnico digitou os comandos abaixo no prompt do Windows para criar e executar um aplicativo React chamado `website`.

```
npm install create-react-app  
npx create-react-app  
website  
cd website  
npm start
```

No navegador, para aparecer o aplicativo React criado, deve ser utilizado o endereço

- a) <https://localhost:8080/>
- b) <http://127.0.0.1:8080/>
- c) <http://localhost:3000/>
- d) <https://127.0.0.1:3000/>
- e) <http://localhost:80/>

Comentários:

O endereço para acessar o aplicativo, após iniciarmos o servidor local, é <http://localhost:3000/>.

Gabarito: Letra C

16. (AVANÇASP/CM SOROCABA/2022) Considere que foi criado um projeto padrão em react utilizando o create-react-app. Onde, por padrão, ficarão armazenadas todas as dependências de bibliotecas utilizadas?

- a) Na pasta src.
- b) No arquivo .gitignore
- c) No arquivo package.json
- d) No arquivo index.
- e) Na pasta node_modules.

Comentários:

A pasta que armazena as dependências é a node_modules.

Gabarito: Letra E

17. (CONSULPLAN/SEED PR/2022) React é uma biblioteca JavaScript que simplifica e agiliza a tarefa de desenvolvimento de interfaces de usuário interativas e de alto desempenho. Sobre tal biblioteca, assinale a afirmativa correta.

- a) Pelo fato de ser uma biblioteca robusta, React é indicado somente para projetos de grande porte.
- b) Em geral, para usar uma funcionalidade React em um script, não é necessário importá-la para dentro do script.
- c) Pelo fato do objeto props ser do tipo somente leitura, não é possível alterar os valores passados aos componentes filho com uso de props.
- d) Router e Route são componentes que possuem funcionalidades idênticas. Ambos fazem parte da biblioteca React Router e são usados em um projeto de rotas (links).

Comentários:

Vamos analisar cada alternativa.

- a) Errado. React é adaptável a qualquer tamanho de projeto - mas é mais indicado para aplicações de médio e pequeno porte.
- b) Errado. Precisamos importar o React para o programa.
- c) Certo. O *props* é somente leitura nos componentes filhos, não sendo possível alterá-lo.
- d) Errado. Router é a biblioteca e o componente que envolve todos os componentes que desejam acessar a biblioteca. Route é uma ferramenta para criar rotas.

Portanto, correta a letra C.

▪

Gabarito: Letra C

18. (FCC/TRT 17/2022) Considere o arquivo index.js abaixo, que faz parte de um site construído com React, em condições ideais.

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function VerTribunal(props) {
  return <p>TRIBUNAL REGIONAL DO TRABALHO DA {props.regiao}* REGIÃO</p>;
}

const root = ReactDOM.createRoot(document.getElementById('root'));
...I...
```

Para que seja exibido na página o parágrafo TRIBUNAL REGIONAL DO TRABALHO DA 17ª REGIÃO, a próxima linha do código, correspondente à lacuna I, deve ser preenchida corretamente por:

- a) `root.call(<VerTribunal regiao="17"/>);`
- b) `root.send(<VerTribunal(props{regiao="17"})/>);`
- c) `root.render(<VerTribunal regiao="17"/>);`

- d) root.call(<VerTribunal(props{"regiao":17})/>);
- e) root.add(<VerTribunal regiao="17"/>);

Comentários:

A questão aborda os métodos responsáveis pela criação de componentes do React. Para o componente criado pela função VerTribunal aparecer, ele precisa ser renderizado - utilizando a função root.render().

Gabarito: Letra C

19. (FCC/TRT 17/2022) Considere o código abaixo, do arquivo index.js que faz parte de uma aplicação web criada com React, em condições ideais.

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function GetData() {
  const GetTribunal = () => {
    document.write("TRT DA 17ª REGIÃO");
  }

  return (
    <button ... >Ver Tribunal</button>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<GetData />);
```

Para que, ao clicar no botão Ver Tribunal seja exibida a mensagem TRT DA 17ª REGIÃO, a lacuna I deve ser preenchida corretamente por:

- a) onClick={GetData}
- b) onClick="GetTribunal()"
- c) onClick="GetData() => GetTribunal()"
- d) onClick={GetTribunal}
- e) onClick="GetData()"

Comentários:

Precisamos fazer uma referência a um código JavaScript dentro do evento onClick. Para isso, usamos pares de chaves {}. A função que contém a mensagem que se deseja exibir é a GetTribunal - portanto, o comando será onClick={GetTribunal}.

20. (PRGP UNIFEI/UNIFEI/2022) Observe o código JSX abaixo. Os dados advindos da url serão requisitados quantas vezes?

```
import React, { useState, useEffect } from "react";

export default function Usuarios() {
  const [usuarios, setUsuarios] = useState([]);
  const url = "https://jsonplaceholder.typicode.com/users";

  useEffect(() => {
    fetch(url)
      .then((resp) => resp.json())
      .then((resp) => setUsuarios(resp))
      .catch((err) => console.log(err));
  }, [url]);

  return (
    <div>
      {usuarios.map((item) => {
        return (
          <div key={item.id}>
            <span>{item.name} -- </span>
            <span>{item.email} </span>
          </div>
        );
      })}
    </div>
  );
}
```

- a) Todas as vezes que o estado da variável usuarios mudar.
- b) Uma vez.
- c) O número de vezes que o map iterar sobre o array usuarios.
- d) Nenhuma vez.

Comentários:

Numa primeira análise, por termos useState e useEffect você até pode pensar “ah, vamos fazer um fetch toda vez que atualizarmos o estado da variável usuarios”, já que é ela que está sendo lida no useState. Porém o useEffect não utiliza nenhuma variável de estado atualizável - ela está

utilizando a constante url, que é definida no começo do código e nunca modificada. Por esse motivo, só iremos executar o useEffect uma única vez, ao carregarmos a página. Excelente questão.

Gabarito: Letra B

21. (CPCON UEPB/UEPB/2021) Considere o seguinte trecho de código Reactjs:

```
class HelloMessage extends React.Component {  
  render() {  
    return (  
      <div>  
        Hello {this.props.name}  
      </div>  
    );  
  }  
}  
  
ReactDOM.render(  
  <HelloMessage name="Taylor" />,  
  document.getElementById('hello-example')  
);
```

Componentes React implementam um método render() que recebe dados de entrada e retorna o que se quer mostrar. Baseado nisso, o resultado da execução do código acima é

- a) Taylor.
- b) Hello Taylor.
- c) name=Taylor.
- d) hello-example.
- e) Hello name.

Comentários:

A questão aborda alguns conceitos que serão vistos - mas o cerne da questão gira em torno das props. No componente, temos esse trecho de código:

```
<div>  
  Hello {this.props.name}  
</div>
```

E posteriormente, quando vamos renderizar o componente, passamos a seguinte estrutura:

```
<HelloMessage name="Taylor" />
```

Isso quer dizer que estamos associando o valor "Taylor" à propriedade `name`. Então, quando digitamos `Hello {this.props.name}`, estamos, na realidade, digitando `Hello Taylor`. Portanto, correta a letra B.

Gabarito: Letra B

22. (CPCON UEPB) Considere o seguinte trecho de código React:

```
01: class Square extends React.Component {  
02:   constructor(props) {  
03:       
04:     this.state = {  
05:       value: null,  
06:     };  
07:   }  
08:   render() {  
09:     return (  
10:       <button className="square" onClick={() => alert('click')}>  
11:         {this.props.value}  
12:       </button>  
13:     );  
14:   }  
15: }
```

O trecho de código da linha 03 foi intencionalmente omitido. Todas as classes componentes React que possuem um construtor devem começar com uma chamada que deveria estar na linha 03. Esta chamada é

- a) `render()`.
- b) `this(props)`.
- c) `return true`.
- d) `React.createElement(object)`.
- e) `super(props)`.

Comentários:

Como os componentes de classe no React nada mais são que uma forma de implementação de herança (extensão) de um componente pai com todas as dependências e formas que um

componente precisa para ser implementado, precisamos acessar essas propriedades do componente pai - e, para isso, usamos o comando `super(props)`.

Gabarito: Letra E

23. (CPCON UEPB/UEPB/2021) Sobre React, considere as afirmações abaixo:

- I- Utiliza DOM virtual ao invés de DOM real.
- II- Utiliza renderização do lado do servidor.
- III- É apenas uma biblioteca e não um framework avançado.

Está CORRETO o que se afirma em

- a) II e III, apenas.
- b) I e III, apenas.
- c) I, II e III
- d) I e II, apenas.
- e) II, apenas.

Comentários:

Vamos analisar cada item.

I. Certo. O React utiliza o VirtualDOM para fazer suas implementações.

II. Certo, com ressalvas. Ao utilizarmos a abordagem de *framework*, ou seja, usando um *framework* full-stack como o NPM, de fato temos uma renderização server-side. Porém se a criação fosse feita com o Vite, teríamos a renderização client-side - mas, como o Vite não estava disponível em 2021, o item foi considerado correto.

III. Certo. Não sei ao certo o que a banca quis dizer com isso, mas o React realmente é um biblioteca, não um "framework avançado", implementando elementos novos na linguagem JavaScript.

Portanto, todos os itens estão corretos.

Gabarito: Letra C

24. (Instituto AOCP/SANESUL/2021) Durante a criação de componentes com react.js, dos métodos a seguir, qual é o único obrigatório em um class-component?

- a) constructor ()
- b) componentDidMount()
- c) render ()

- d) `componentDidUpdate()`
- e) `componentWillUnmount()`

Comentários:

Questão tranquila. Quando estamos falando de class-component, só existe um método obrigatório: o `render()`.

Gabarito: Letra C

25. (CEBRASPE/MPE CE/2020) Julgue o item seguinte, a respeito de arquitetura de aplicações para ambiente web e arquitetura em camadas.

React é uma biblioteca JavaScript usada para a criação de interfaces gráficas com o usuário e cuja arquitetura é baseada em componentes. Essa biblioteca pode ser renderizada no servidor, utilizando-se Node.

Comentários:

Perfeito! O React é uma forma de implementarmos JavaScript a partir de componentes modulares, podendo ser renderizada server-side com o NodeJS.

Gabarito: Certo

26. (COMPERVE/TJ RN/2020) O React é uma biblioteca JavaScript de código aberto e, atualmente, é uma das ferramentas mais populares entre os desenvolvedores web. São características do React ser uma biblioteca

- a) declarativa, que não gerencia seu próprio virtual DOM e não permite a criação de aplicativos móveis.
- b) imperativa, que não gerencia seu próprio virtual DOM e não dá suporte a componentes reutilizáveis.
- c) declarativa, que gerencia seu próprio virtual DOM e permite a criação de aplicativos móveis.
- d) imperativa, que gerencia seu próprio virtual DOM e dá suporte a componentes reutilizáveis.

Comentários:

O React é uma biblioteca declarativa, isso é, ele permite aos desenvolvedores descrever o que a interface de usuário (UI) deve parecer em termos de estado final, em vez de detalhar passo a passo como atingir esse estado. Além disso, ele gerencia seu próprio DOM Virtual para fazer sua criação de componentes. Portanto, correta a letra C.

Gabarito: Letra C

27. (CEBRASPE/SLU DF/2019) Julgue o item, relativo à linguagem de programação JavaScript e às ferramentas Node e React.

O JSX (JavaScript Syntax Extension) é de uso obrigatório no React e permite inserir a interface do usuário no código JavaScript.

Comentários:

O JSX serve para auxiliar na criação dos componentes, implementando tags HTML dentro de um código JavaScript - mas é plenamente possível criar componentes só com JS, sem utilizarmos o JSX. Por isso, ela não é de uso obrigatório.

Gabarito: Errado

28. (CCV UFC/UFC/2019) Em React.Js, como são chamadas as entradas que são passadas na criação dos componentes React, usando uma convenção de nomenclatura semelhante aos atributos de tag HTML.

- a) Refs
- b) Props
- c) State
- d) Keys
- e) Elements

Comentários:

A questão aborda o conceito de props, uma forma de passarmos parâmetros de elementos pai para elementos filho - no formato somente leitura.

Gabarito: Letra B

29. (Instituto AOC/PRODEB/2018) O React é uma biblioteca utilizada para desenvolvimento de interfaces (frontend) que tem como base o princípio do desenvolvimento de componentes. O React utiliza-se de uma técnica de dividir as estruturas complexas em partes menores e desenvolver para cada uma delas um componente. Como é o nome dessa técnica?

- a) Component Driven Development.
- b) Component Development Structured.
- c) Driven Component Divided.
- d) Work Driven Structured.

e) Development React Structured.

Comentários:

Questão bem intuitiva. A abordagem da maioria dos SPAs, que cria componentes pequenos e os agrega para formar uma página, como um Lego, é chamada de Component Driven Development - já que focamos no desenvolvimento dos componentes, não do todo.

Gabarito: Letra A

30. (CEBRASPE/TRE ES/2015) Na escolha de um framework e bibliotecas para apoiar a utilização do JavaScript, uma empresa levou em consideração algumas afirmações apresentadas por sua equipe técnica. Com base nesse contexto, assinale a opção correta.

- a) O ReactJS não depende exclusivamente do DOM (document object model) do navegador, uma vez que mantém um DOM virtual próprio.
- b) O AngularJS está baseado na manipulação pelo desenvolvedor da sincronização entre a camada de visão, fornecida pelo código HTML, e o modelo, e vice-versa.
- c) Segundo os princípios adotados pelo AngularJS, o código declarativo é melhor para expressar a lógica do negócio.
- d) A forma primária de organizar as interfaces no Ember.js são os templates escritos em JavaScript que definem o seu comportamento.
- e) Os projetos Ember.js podem ser criados e gerenciados por uma ferramenta de linha de comando denominada Ember Loc.

Comentários:

Vamos analisar cada alternativa.

- a) Certo. De fato, o React usa o VirtualDOM, fazendo com que ele não tenha dependência do DOM real.
- b) Errado. O AngularJS utiliza o conceito de "two-way data binding" (vinculação bidirecional de dados), que sincroniza automaticamente o modelo (dados) e a visão (interface de usuário) sem a necessidade de intervenção manual do desenvolvedor.
- c) Errado. No AngularJS, o código declarativo é usado principalmente para definir a interface de usuário e configurações, enquanto a lógica de negócio é tipicamente escrita de forma imperativa em controladores e serviços.
- d) Errado. No Ember.js, as interfaces são organizadas primariamente através de templates escritos em HTML Handlebars, que descrevem a estrutura da interface de usuário. O comportamento e a lógica são definidos em componentes e controladores, mas não diretamente nos templates.

- e) Errado. A ferramenta de linha de comando para criar e gerenciar projetos Ember.js é chamada Ember CLI (Command Line Interface), não Ember Loc.

Portanto, a alternativa correta é a letra A.

Gabarito: Letra A

LISTA DE QUESTÕES

01. (FGV/CGE PB/2024) Considere o seguinte código em JavaScript com React:

```
import React from 'react';

function Number() {
  let x = 5, y = 5;
  const [n, setN] = React.useState(x);

  React.useEffect(() => {
    setN(x => x + n);
    setN(y => x + y);
  }, [])

  return n;
}
```

Ao se utilizar o componente funcional Number, o valor retornado por Number após a renderização final será:

- a) 5;
- b) 10;
- c) 15;
- d) 20;
- e) 55.

02. (FGV/ALEP/2024) Sobre a biblioteca de Javascript React, assinale a afirmativa **incorreta**.

- a) React permite que a interface do usuário seja construída por meio de componentes reutilizáveis
- b) React manipula os objetos DOM reais do navegador diretamente, melhorando o desempenho das atualizações na interface do usuário.
- c) React Props são como argumentos de função em JavaScript e atributos em HTML
- d) JSX facilita a escrita e a adição de HTML no React
- e) o React renderiza HTML para a página da web usando uma função chamada createRoot() e seu método render()

03. (CESGRANRIO/BB/2023) O React Native 0.59 introduziu o conceito de Hooks.

Entre os Hooks, tem-se o usestate, que permite

- a) calcular o estado de um CEP ou ZIP de acordo com o Locale.
- b) chamar estados específicos do engine React para alterar seu comportamento.
- c) declarar uma classe que segue o padrão de design state.
- d) criar uma enumeration que representa estados.
- e) manter um estado local em uma função de um componente funcional.

04. (AVANÇASP/PREF. AMERICANA/2023) Um arquivo React é conhecido por ser um aplicativo de página única é uma aplicação web ou site que consiste de uma única página web (só tem um html), essa característica é conhecida como:

- a) FDA.
- b) SPA.
- c) APE.
- d) PUN.
- e) UPG.

05. (AVANÇASP/PREF. AMERICANA/2023) Para criar um projeto padrão em react, chamado projetoteste, a linha de comando correta é

- a) npx-create-app-projetoteste
- b) projetoteste-npx-create
- c) npm create-projetoteste
- d) npx create-react-app projetoteste
- e) npx -app-projetoteste.

06. (AVANÇASP/PREF. AMERICANA/2023) No React um conceito de programação onde uma representação ideal, ou "virtual", da interface do usuário é mantida em memória e sincronizada com o DOM "real" por uma biblioteca como o ReactDOM é conhecido como:

- a) DOMReact.
- b) ReactDOM.
- c) DOM
- d) DOMPrincipal
- e) VirtualDOM

07. (CEBRASPE/SERPRO/2023) O React é compatível com outras bibliotecas e frameworks, como o Redux e o AngularJS, e pode ser integrado a essas tecnologias. Em relação aos procedimentos necessários para realizar uma integração do React com o AngularJS, julgue o item subsequente.

Para permitir a integração entre os frameworks React e AngularJS, um passo a ser executado é criar um componente React e renderizá-lo dentro desse elemento, usando-se o método `ReactDOM.render()`.

08. (FGV/TCE SP/2023) A analista Ana está desenvolvendo a interface gráfica da aplicação web TCEDash, utilizando React com componentes funcionais. Ana precisa que o componente `TelaUm`, do TCEDash, execute determinada sequência de instruções a cada nova renderização. Porém, a sequência de instruções deve ser garantidamente executada antes de o navegador web atualizar visualmente a tela.

Para garantir a execução antes da atualização visual da tela, Ana deve implementar a sequência de instruções dentro do hook do React:

- a) `useEffect`;
- b) `useTransition`;
- c) `useLayoutEffect`;
- d) `useDeferredValue`;
- e) `useImperativeHandle`.

09. (FGV/TJ SE/2023) O analista José está implementando a interface gráfica da aplicação web JusticaSE, utilizando a biblioteca JavaScript React. A JusticaSE possui uma tela de dashboard, implementada pelo componente `TelaD`, que demanda alto custo computacional para ser renderizada. A fim de reduzir o custo de renderização da tela de dashboard, José encapsulou determinadas functions de `TelaD` em um Hook nativo da biblioteca React. O Hook utilizado por José memoriza a definição de uma function ao longo de várias renderizações de um componente. José utilizou o React Hook:

- a) `useState`;
- b) `useEffect`;
- c) `useMemo`;
- d) `useCallback`;
- e) `useContext`.

10. (FCC/TRT 12/2023) Um Analista deseja realizar mudanças nos dados de um componente e atualizar a interface do usuário de acordo com essas mudanças. No React, a sintaxe correta do método usado para atualizar o estado de um componente é:

- a) `this.componentState( novo estado: novo valor );`
- b) `this.updateState(( novo estado: novo valor ));`
- c) `this.setState({ novo estado: novo valor });`
- d) `this.changeState({ novo estado: novo valor });`
- e) `this.modifyState({ novo estado: novo valor });`

11. (FCC/PGE AM/2022) Considere que um desenvolvedor está criando um aplicativo usando React e React Native e deseja criar um elemento h1 contendo o título Amazonas, aplicando a classe de estilo CSS de nome tit e armazenando em uma constante chamada elemento. Para realizar esta tarefa, ele terá que utilizar a instrução

- a) `let elemento = ReactNative.createElement('h1',{className:'tit'},'Amazonas');`
- b) `let const elemento = React.createElement('h1'.tit),'Amazonas');`
- c) `const elemento = ReactNative.createElement('h1',{className:'tit'},'Amazonas');`
- d) `const elemento = React.createElement('h1',{className:'tit'},'Amazonas');`
- e) `const elemento = React.createElement('h1'.tit,'Amazonas');`

12. (Legalle/CM POA/2022) Analise as seguintes afirmações sobre ReactJS:

- I. Trata-se de uma biblioteca JavaScript.
- II. Uso exclusivo para desenvolvimento de aplicações de back-end
- III. Não possibilita o desenvolvimento de aplicações single-page.
- IV. O aptitude é um gerenciador que permite gerenciar as dependências de um projeto de pacotes do ReactJS.

Esta(ão) CORRETA(S):

- a) Apenas I.
- b) Apenas I e III.
- c) Apenas II e IV.
- d) Apenas I e IV.
- e) Apenas I, III e IV.

13. (IDECAN/UNILAB/2022) Sobre a biblioteca React, analise as afirmativas abaixo:

- I. O React a quebra em pequenos pedaços, chamados de componentes, e os isola, sendo cada um deles independentes entre si perante a tecnologia.
- II. Os componentes são reutilizáveis e, uma vez categorizados, podem ser reaproveitados em quaisquer páginas de um site ou aplicativo.
- III. Dentro dos estados (states) estão as propriedades, ou props. Estas props são objetos com várias informações que podem ser de variados tipos, como função, número ou string.
- IV. Assim como as propriedades, os states são imutáveis, como por exemplo variáveis declaradas dentro de uma função.

Assinale

- a) se apenas as afirmativas I e II estiverem corretas.

- b) se apenas as afirmativas II e III estiverem corretas.
- c) se apenas as afirmativas I e IV estiverem corretas.
- d) se apenas as afirmativas II e IV estiverem corretas.

14. (FCC/TJ CE/2022) Para criar um componente React chamado App, uma Analista usou o código abaixo.

```
class App extends React.Component {  
  
}
```

O único método necessário dentro desta classe, que recebe os dados de entrada e retorna o que deve ser exibido, é o componente

- a) showApp()
- b) root()
- c) createApp()
- d) handler()
- e) render()

15. (FCC/TRT 23/2022) Considerando que o npx e Node.js estão instalados, um Técnico digitou os comandos abaixo no prompt do Windows para criar e executar um aplicativo React chamado website.

```
npm install create-react-app  
npx create-react-app  
website  
cd website  
npm start
```

No navegador, para aparecer o aplicativo React criado, deve ser utilizado o endereço

- a) https://localhost:8080/
- b) http://127.0.0.1:8080/
- c) http://localhost:3000/
- d) https://127.0.0.1:3000/
- e) http://localhost:80/

16. (AVANÇASP/CM SOROCABA/2022) Considere que foi criado um projeto padrão em react utilizando o create-react-app. Onde, por padrão, ficarão armazenadas todas as dependências de bibliotecas utilizadas?

- a) Na pasta src.
- b) No arquivo .gitignore
- c) No arquivo package.json
- d) No arquivo index.
- e) Na pasta node_modules.

17. (CONSULPLAN/SEED PR/2022) React é uma biblioteca JavaScript que simplifica e agiliza a tarefa de desenvolvimento de interfaces de usuário interativas e de alto desempenho. Sobre tal biblioteca, assinale a afirmativa correta.

- a) Pelo fato de ser uma biblioteca robusta, React é indicado somente para projetos de grande porte.
- b) Em geral, para usar uma funcionalidade React em um script, não é necessário importá-la para dentro do script.
- c) Pelo fato do objeto props ser do tipo somente leitura, não é possível alterar os valores passados aos componentes filho com uso de props.
- d) Router e Route são componentes que possuem funcionalidades idênticas. Ambos fazem parte da biblioteca React Router e são usados em um projeto de rotas (links).

18. (FCC/TRT 17/2022) Considere o arquivo index.js abaixo, que faz parte de um site construído com React, em condições ideais.

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function VerTribunal(props) {
  return <p>TRIBUNAL REGIONAL DO TRABALHO DA {props.regiao}* REGIÃO</p>;
}

const root = ReactDOM.createRoot(document.getElementById('root'));
...I...
```

Para que seja exibido na página o parágrafo TRIBUNAL REGIONAL DO TRABALHO DA 17ª REGIÃO, a próxima linha do código, correspondente à lacuna I, deve ser preenchida corretamente por:

- a) root.call(<VerTribunal regiao="17"/>);
- b) root.send(<VerTribunal(props{regiao="17"})/>);
- c) root.render(<VerTribunal regiao="17"/>);
- d) root.call(<VerTribunal(props{"regiao":17})/>);
- e) root.add(<VerTribunal regiao="17"/>);

19. (FCC/TRT 17/2022) Considere o código abaixo, do arquivo index.js que faz parte de uma aplicação web criada com React, em condições ideais.

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function GetData() {
  const GetTribunal = () => {
    document.write("TRT DA 17ª REGIÃO");
  }

  return (
    <button ... >Ver Tribunal</button>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<GetData />);
```

Para que, ao clicar no botão Ver Tribunal seja exibida a mensagem TRT DA 17ª REGIÃO, a lacuna I deve ser preenchida corretamente por:

- a) onClick={GetData}
- b) onClick="GetTribunal()"
- c) onClick="GetData() => GetTribunal()"
- d) onClick={GetTribunal}
- e) onClick="GetData()"

20. (PRGP UNIFEI/UNIFEI/2022) Observe o código JSX abaixo. Os dados advindos da url serão requisitados quantas vezes?

```
import React, { useState, useEffect } from "react";

export default function Usuarios() {
  const [usuarios, setUsuarios] = useState([]);
  const url = "https://jsonplaceholder.typicode.com/users";

  useEffect(() => {
    fetch(url)
      .then((resp) => resp.json())
      .then((resp) => setUsuarios(resp))
      .catch((err) => console.log(err));
  }, [url]);

  return (
    <div>
      {usuarios.map((item) => {
```

```
    return (  
      <div key={item.id}>  
        <span>{item.name} -- </span>  
        <span>{item.email} </span>  
      </div>  
    );  
  }  
</div>  
);  
}
```

- a) Todas as vezes que o estado da variável usuarios mudar.
- b) Uma vez.
- c) O número de vezes que o map iterar sobre o array usuarios.
- d) Nenhuma vez.

21. (CPCON UEPB/UEPB/2021) Considere o seguinte trecho de código Reactjs:

```
class HelloMessage extends React.Component {  
  render() {  
    return (  
      <div>  
        Hello {this.props.name}  
      </div>  
    );  
  }  
}  
ReactDOM.render(  
  <HelloMessage name="Taylor" />,  
  document.getElementById('hello-example')  
);
```

Componentes React implementam um método render() que recebe dados de entrada e retorna o que se quer mostrar. Baseado nisso, o resultado da execução do código acima é

- a) Taylor.
- b) Hello Taylor.
- c) name=Taylor.
- d) hello-example.
- e) Hello name.

22. (CPCON UEPB) Considere o seguinte trecho de código React:

```
01: class Square extends React.Component {  
02:   constructor(props) {  
03:       
04:     this.state = {  
05:       value: null,  
06:     };  
07:   }  
08:   render() {  
09:     return (  
10:       <button className="square" onClick={() => alert('click')}>  
11:         {this.props.value}  
12:       </button>  
13:     );  
14:   }  
15: }
```

O trecho de código da linha 03 foi intencionalmente omitido. Todas as classes componentes React que possuem um construtor devem começar com uma chamada que deveria estar na linha 03. Esta chamada é

- a) render().
- b) this(props).
- c) return true.
- d) React.createElement(object).
- e) super(props).

23. (CPCON UEPB/UEPB/2021) Sobre React, considere as afirmações abaixo:

- I- Utiliza DOM virtual ao invés de DOM real.
- II- Utiliza renderização do lado do servidor.
- III- É apenas uma biblioteca e não um framework avançado.

Está CORRETO o que se afirma em

- a) II e III, apenas.
- b) I e III, apenas.
- c) I, II e III
- d) I e II, apenas.
- e) II, apenas.

24. (Instituto AOCP/SANESUL/2021) Durante a criação de componentes com react.js, dos métodos a seguir, qual é o único obrigatório em um class-component?

- a) constructor ()
- b) componentDidMount()
- c) render ()
- d) componentDidUpdate()
- e) componentWillUnmount()

25. (CEBRASPE/MPE CE/2020) Julgue o item seguinte, a respeito de arquitetura de aplicações para ambiente web e arquitetura em camadas.

React é uma biblioteca JavaScript usada para a criação de interfaces gráficas com o usuário e cuja arquitetura é baseada em componentes. Essa biblioteca pode ser renderizada no servidor, utilizando-se Node.

26. (COMPERVE/TJ RN/2020) O React é uma biblioteca JavaScript de código aberto e, atualmente, é uma das ferramentas mais populares entre os desenvolvedores web. São características do React ser uma biblioteca

- a) declarativa, que não gerencia seu próprio virtual DOM e não permite a criação de aplicativos móveis.
- b) imperativa, que não gerencia seu próprio virtual DOM e não dá suporte a componentes reutilizáveis.
- c) declarativa, que gerencia seu próprio virtual DOM e permite a criação de aplicativos móveis.
- d) imperativa, que gerencia seu próprio virtual DOM e dá suporte a componentes reutilizáveis.

27. (CEBRASPE/SLU DF/2019) Julgue o item, relativo à linguagem de programação JavaScript e às ferramentas Node e React.

O JSX (JavaScript Syntax Extension) é de uso obrigatório no React e permite inserir a interface do usuário no código JavaScript.

28. (CCV UFC/UFC/2019) Em React.Js, como são chamadas as entradas que são passadas na criação dos componentes React, usando uma convenção de nomenclatura semelhante aos atributos de tag HTML.

- a) Refs
- b) Props
- c) State
- d) Keys
- e) Elements

29. (Instituto AOCP/PRODEB/2018) O React é uma biblioteca utilizada para desenvolvimento de interfaces (frontend) que tem como base o princípio do desenvolvimento de componentes. O React utiliza-se de uma técnica de dividir as estruturas complexas em partes menores e desenvolver para cada uma delas um componente. Como é o nome dessa técnica?

- a) Component Driven Development.
- b) Component Development Structured.
- c) Driven Component Divided.
- d) Work Driven Structured.
- e) Development React Structured.

30. (CEBRASPE/TRE ES/2015) Na escolha de um framework e bibliotecas para apoiar a utilização do JavaScript, uma empresa levou em consideração algumas afirmações apresentadas por sua equipe técnica. Com base nesse contexto, assinale a opção correta.

- a) O ReactJS não depende exclusivamente do DOM (document object model) do navegador, uma vez que mantém um DOM virtual próprio.
- b) O AngularJS está baseado na manipulação pelo desenvolvedor da sincronização entre a camada de visão, fornecida pelo código HTML, e o modelo, e vice-versa.
- c) Segundo os princípios adotados pelo AngularJS, o código declarativo é melhor para expressar a lógica do negócio.
- d) A forma primária de organizar as interfaces no Ember.js são os templates escritos em JavaScript que definem o seu comportamento.
- e) Os projetos Ember.js podem ser criados e gerenciados por uma ferramenta de linha de comando denominada Ember Loc.

GABARITO

GABARITO



1. Letra B
2. Letra B
3. Letra E
4. Letra B
5. Letra D
6. Letra E
7. Certo
8. Letra C
9. Letra C
10. Letra C

11. Letra D
12. Letra A
13. Letra A
14. Letra E
15. Letra C
16. Letra E
17. Letra C
18. Letra C
19. Letra D
20. Letra B

21. Letra B
22. Letra E
23. Letra C
24. Letra C
25. Certo
26. Letra C
27. Errado
28. Letra B
29. Letra A
30. Letra A

VUE.JS

Conceitos Gerais



O **Vue.js** é um **framework progressivo de JavaScript**, destinado a criar **SPA** (Single Page Apps) e **interfaces de usuário** (UI). Criado por Evan You, o Vue.js é projetado para ser adotado gradualmente, o que significa que pode ser utilizado para construir componentes de UIs interativas em projetos existentes ou para desenvolver aplicativos completos.

O *framework* se baseia em três pilares:

- **Componentes:** tudo no Vue.js é um componente, que são trechos reutilizáveis de código que encapsulam HTML, CSS e JavaScript. Cada componente pode conter sua própria lógica, estilo e estrutura, permitindo que desenvolvedores construam interfaces complexas de forma modular e organizada.
- **Reatividade:** o Vue.js utiliza um sistema de reatividade que monitora alterações nos dados e automaticamente atualiza a interface do usuário quando esses dados mudam. Isso simplifica muito o gerenciamento do estado da aplicação, eliminando a necessidade de manipular o DOM diretamente.
- **Renderização declarativa:** em vez de programar diretamente a manipulação do DOM, os desenvolvedores descrevem como a interface deve parecer usando uma sintaxe semelhante ao HTML (templates) ou diretamente em JavaScript com a render function. O Vue cuida de atualizar o DOM de forma eficiente com base nessas declarações.

O principal objetivo do Vue.js é facilitar o desenvolvimento de interfaces de usuário interativas e dinâmicas, fornecendo uma experiência de desenvolvimento que é ao mesmo tempo produtiva e agradável. Ele busca atingir isso através de simplicidade, flexibilidade e alta performance.

(Inédita/Prof. Felipe Mathias) Acerca dos conhecimentos sobre *frameworks* de apps web, julgue o item subsequente.

O Vue.js é um framework que combina HTML, CSS e JavaScript para criar componentes ou aplicações de página única (SPA). Por utilizar tecnologias de JavaScript, podemos dizer que é um framework baseado na renderização *client-side* de aplicativos web.

Comentários:

Perfeita a definição do Vue.js. Ele combina a stack padrão da web para criar aplicativos web de página única. Além disso, como é inerente ao JavaScript, temos uma renderização que é feita no navegador do cliente - portanto, *client-side*. (Gabarito: Certo)

createApp

Podemos analisar o Vue em dois pontos - sua inserção no HTML, e sua instanciação no JavaScript. Vamos primeiramente analisar como ele funciona no arquivo de *script*, onde criamos uma instância do Vue para o projeto. Para criar um aplicativo Vue, usamos o comando **createApp(nome)**. Esse comando pode aparecer simplesmente assim, ou acompanhado do prefixo Vue anteriormente (se ele não houver sido importado de forma isolada) - como `Vue.createApp(nome)`.

O `createApp()` nada mais é que uma função, disponibilizada pela biblioteca do VueJS. Ao invocá-la, você cria uma nova aplicação Vue. Essa aplicação pode então ser montada em um elemento do DOM, configurada com componentes, plugins e outras opções.

A sintaxe básica para criar uma aplicação Vue é a seguinte:

JavaScript

```
import { createApp } from 'vue';  
import myApp from './myApp.vue';  
  
const app = createApp(myApp); // ou vue.createApp(myApp)
```

Passamos como parâmetro para a função o `rootComponent` - componente raiz da aplicação. Normalmente é um componente Vue definido em um arquivo `.vue`, caracterizando o que é chamado de "Single-File Component". Veremos mais sobre adiante na aula.

Podemos interagir com o aplicativo de duas formas distintas: adicionando elementos aos poucos, conforme eles forem surgindo, a partir da manipulação de propriedades, ou definindo as propriedades diretamente no corpo da função inicial. O primeiro caso, acessando as propriedades do objeto, pode ser feito da seguinte forma:

JavaScript

```
// Cria a aplicação Vue  
const app = createApp(App);  
// Usa o roteador e a store  
app.use(router);  
app.use(store);  
// Registra um componente globalmente  
app.component('my-component', MyComponent);  
// Registra uma diretiva globalmente  
app.directive('my-directive', MyDirective);  
// Fornece um valor que pode ser injetado  
app.provide('globalValue', 12345);
```

A segunda forma é um pouco mais “didática”. Um aplicativo compartilha as mesmas propriedades que um componente - afinal, ele não deixa de ser um componente também. Temos as seguintes propriedades que podem ser manipuladas:

- **data** → retorna um objeto contendo o estado reativo inicial do componente, basicamente uma coletânea de dados
- **methods** → um conjunto de métodos/funções que podem ser chamadas no aplicativo
- **computed** → um conjunto de objetos, onde cada chave é uma propriedade computada e o valor é uma função ou um objeto com get ou set
- **watch** → um objeto onde cada chave é uma expressão ou propriedade a ser observada (útil para variáveis que são atualizadas constantemente), e o seu valor é uma função de callback
- **components** → registra os componentes locais que podem ser usados dentro deste componente
- **directives** → registra as diretivas em objetos, onde a chave é a diretiva, e o valor é sua definição
- **provide** → fornece valores que podem ser injetados em componentes filho usando injeções
- **props** → declaração das propriedades aceitas pelo componente

Veja um exemplo onde definimos **data**, **methods** e **props** para o App.

JavaScript

```
const app = createApp({
  data() {
    return {
      message: 'Hello, Vue!',
      count: 0
    };
  },
  methods: {
    increment() {
      this.count++;
    }
  },
  props: {
    myProp: {
      type: String,
      required: true
    }
  }
});
```

Como as propriedades são importantes e volta e meia caem em prova (e temos muitas delas), vou trazer uma tabela-resumo para você com todas as propriedades encontradas na API do Vue, e sua descrição.

Propriedade	Descrição
data	Define o estado reativo inicial do componente.
methods	Define métodos que podem ser chamados no template ou no componente.
computed	Define propriedades computadas baseadas em outras propriedades reativas.
watch	Observa mudanças em propriedades e executa callbacks.
components	Registra componentes locais disponíveis para uso no template.
directives	Registra diretivas personalizadas locais disponíveis para uso no template.
mixins	Reutiliza funcionalidades através de múltiplos componentes.
extends	Estende um componente base existente.
provide	Fornecer valores injetáveis para componentes descendentes.
inject	Receber valores fornecidos por um componente ascendente.
props	Define propriedades aceitas pelo componente, permitindo a passagem de propriedades entre componentes pai e filho.
emits	Declara eventos que o componente pode emitir.
template	Define o template HTML do componente como uma string.
render	Define uma função de renderização para o componente.
beforeCreate	Hook de ciclo de vida chamado antes da criação do componente.
created	Hook de ciclo de vida chamado após a criação do componente.
beforeMount	Hook de ciclo de vida chamado antes do componente ser montado no DOM.
mounted	Hook de ciclo de vida chamado após o componente ser montado no DOM.
beforeUpdate	Hook de ciclo de vida chamado antes do componente ser atualizado.
updated	Hook de ciclo de vida chamado após o componente ser atualizado.
beforeUnmount	Hook de ciclo de vida chamado antes do componente ser desmontado.
unmounted	Hook de ciclo de vida chamado após o componente ser desmontado.
setup	Função especial usada na Composition API para configurar componentes.
name	Define um nome para o componente que pode ser usado para depuração ou em templates.
inheritAttrs	Controla se os atributos não reconhecidos são aplicados ao elemento raiz do componente.

<code>model</code>	Define um mapeamento personalizado para a diretiva v-model.
<code>propsData</code>	Define valores iniciais das propriedades (usado principalmente em testes).

QUESTÃO DE PROVA



(FUNDATEC/CIGA SC/2023) Qual alternativa descreve melhor a utilidade dos atributos props em VueJS?

- a) Proteger variáveis locais de acessos por componentes externos.
- b) Uma forma de aplicar automaticamente classes CSS a elementos.
- c) Uma forma de passar dados de um componente pai para um componente filho.
- d) Estabelecem relação de herança entre componentes reativos.
- e) Eventos que são disparados quando um elemento é alterado.

Comentários:

A propriedade *props* é passada no objeto de configuração, e declara as propriedades que um componente pode receber. Seu objetivo é facilitar a passagem de dados entre componente pai e componente filho, fazendo uma comunicação vertical de componentes. No caso em análise, os atributos *props* são as propriedades específicas definidas, que são passadas entre componentes. Portanto, a alternativa que corretamente descreve o *props* é a letra C. Quanto às demais alternativas, elas se referem a:

- a) Data
- b) Class e Style *bindings* (feito a partir de diretivas)
- c) Props
- d) Extends
- e) Watch

Sendo assim, ficamos com a letra C como nosso gabarito. (Gabarito: Letra C)

Um outro elemento importante aqui é a função `mount()`. Ia é usada para montar uma instância da aplicação Vue em um elemento DOM existente. A função `mount()` pega a instância da aplicação, criada com `createApp()`, e a conecta a um elemento específico no DOM, fazendo com que a aplicação comece a renderizar e funcionar.

Diretivas

As **diretivas do Vue** são elementos especiais **inseridos dentro dos atributos do HTML**. Você consegue identificá-los pela presença do prefixo **v-** dentro dos atributos HTML. Essas diretivas são usadas para conectar o Vue à instância de desejo e criar interfaces dinâmicas e interativas. Com isso, temos uma facilitação na criação de páginas responsivas, com muito menos códigos - ao menos se comparado aos métodos tradicionais no JavaScript.

Veja um exemplo de atribuição de diretiva:

```
HTML

<script
  src="https://unpkg.com/vue@3/dist/vue.global.js">
</script>

<div id="myApp">
  <div v-bind:class="VueClass"> Conteúdo da DIV </div>
</div>
```

Veja, primeiramente temos a importação da biblioteca do Vue para o arquivo. Como a estruturação do Vue é feita em um arquivo JavaScript, importamo-lo como um *script* dentro do nosso projeto. É importante saber o link, já que volta e meia as bancas cobram (o que eu acho uma aberração): <https://unpkg.com/vue@3/dist/vue.global.js>.

Em seguida, temos duas *divs*. A primeira pouco nos importa, apenas demarca o aplicativo - o ponto focal está na segunda. Estamos atribuindo a diretiva **v-bind**, que conecta um atributo em uma tag HTML a uma variável de dados dentro da instância do Vue. Aqui é interessante sabermos quais são as principais diretivas e como elas funcionam - vamos “explorar” essa área agora.

v-bind

A diretiva **v-bind** é destinada a fazer um *binding*, uma **vinculação entre dados do Vue e valores na visualização**. O objetivo é facilitar a atualização dinâmica de valores, sem termos a necessidade de recarregar a página. Pegue essa tag HTML como exemplo:

```

```

Aqui temos a vinculação entre o atributo **src**, que é responsável por trazer o endereço de origem da imagem, com um valor chamado de **url**. Esse valor **url** pode ser definido como uma variável dentro do ambiente Vue e ter seu conteúdo manipulado por diversas interações, como atualizações dinâmicas de endereço.

Esse *binding* pode ser feito também como uma forma de fazer estilos CSS in-line, isso é, diretamente no HTML. Veja esse exemplo abaixo:

```
HTML

<div v-bind:style="{ fontSize: size }">
  Conteúdo da DIV
</div>
```

Acima, definimos em `{ fontSize: size }` o parâmetro para tamanho de fontes `fontSize` e uma variável que irá comportar o tamanho de fato da fonte, o `size`. Ou, se quisermos atribuir o valor diretamente, sem basear seu conteúdo numa variável, também podemos:

```
HTML

<div v-bind:style="{
  backgroundColor: 'lightpink',
  'font-weight':
  'bolder' }">
  Conteúdo da DIV
</div>
```

Perceba que o Vue aceita a passagem de estilos CSS de duas formas:

- **Camel Case** → é a forma recomendada. É o que vemos em `backgroundColor`.
- **Kebab Case** → embora aceito, não é recomendado. É o caso do `'font-weight'`.

Então pense que estamos trabalhando com uma forma de vincular um ATRIBUTO DA TAG a uma VARIÁVEL JAVASCRIPT, através desse *bind* do Vue. Podemos fazer isso com qualquer atributo: `src`, `class`, `width`, `height`, `style`, entre outros.

O `v-bind` também pode aparecer em sua forma abreviada. Ela é composta simplesmente de dois pontos `:`. Veja o primeiro exemplo dessa seção escrito na forma abreviada:

```
HTML

<div :style="{ fontSize: size }">
  Conteúdo da DIV
</div>
```

v-if

A diretiva **v-if**, que também compreende o **v-else-if** e **v-else**, são responsáveis por inserir **renderizações condicionais** no Vue.JS. Uma renderização condicional significa que determinado elemento só irá ser renderizado, isso é, ser interpretado e aparecer graficamente na tela do usuário (no seu navegador) se ele atender a determinada condição.

Observe o seguinte exemplo:

```
HTML
<p v-if="produtoEmEstoque">
  Disponível em estoque
</p>

<p v-else>
  Estoque esgotado
</p>
```

No exemplo acima, se o valor para a variável `produtoEmEstoque` for `true`, renderizamos o primeiro parágrafo - ou seja, será mostrado "Disponível em estoque". Se o valor for `false`, iremos renderizar o bloco residual, o `v-else`, com o texto "Estoque esgotado". Podemos inserir comparações lógicas, operadores, entre outros para deixar a sintaxe mais completa.

```
HTML
<p v-if="produtoEmEstoque > 0">
  Disponível em estoque
</p>

<p v-else>
  Estoque esgotado
</p>
```

Só um cuidado - mantenha todo o texto da comparação, seja usando operador de maior que `>`, menor que `<`, ou operadores lógicos como o `&&` e o `||`, dentro das aspas. Isso para não termos problemas com as delimitações da tag HTML.

v-show

A diretiva **v-show** objetiva **tornar um elemento visível ou tirá-lo de visão**. É muito parecido com o `v-if`, mas aqui o elemento sempre será renderizado - o que irá acontecer é a mudança da

propriedade `display`, do CSS, para o valor `none`, o que torna o elemento invisível. Isso ocorrerá sempre que a condição definida for falsa.

V-IF → SÓ CRIA (RENDERIZA) O ELEMENTO SE ATENDER À CONDIÇÃO
V-SHOW → SEMPRE CRIA O ELEMENTO, MAS PODE OCULTÁ-LO

v-for

A **v-for** é direcionada a fazer uma **renderização de lista**. Ele nada mais faz que aplicar uma estrutura de repetição `for` padrão em cima de um elemento de lista (uma array, por exemplo). É uma forma simplificada de lidarmos com a renderização de elementos baseados em array, já que no JavaScript tradicional isso exige algumas adaptações.

A sintaxe base do v-for é:

`v-for="X in Y"`

Aqui, X representa o elemento que queremos, e Y a *array* em que iremos fazer a varredura. O Vue irá percorrer toda a array e, encontrando uma correspondência de X em Y, ele irá fazer uma renderização. Imagine o seguinte exemplo:

```
HTML

<ul>
  <li v-for="item in items" :key="item.id">
    {{ item.text }}
  </li>
</ul>

<script>
  new Vue({
    el: '#app',
    data: {
      items: [
        { id: 1, text: 'Item 1' },
        { id: 2, text: 'Item 2' },
        { id: 3, text: 'Item 3' }
      ]
    }
  });
</script>
```

Nele, iremos percorrer cada elemento da lista criada no script (não se preocupe em entendê-la agora). Para cada elemento, procuramos a propriedade texto (`item.text`) e renderizamos seu valor dentro da lista `<li>`. Com isso, conseguimos criar uma lista `<li>` em HTML com todos os elementos elencados no conjunto de dados `items`.

v-on

A diretiva **v-on** permite realizar ações baseadas em eventos do DOM. Funciona basicamente como os *event handlers* do JavaScript - interagindo com eventos específicos, com `onClick`, `onMouseOver`, entre outros.

```
HTML

<div id="app">
  <input v-on:input="inpCount++">
  <p>{{ Quantidade de teclas apertadas: ' + inpCount }}</p>
</div>

<script>
  const app = Vue.createApp({
    data() {
      return {
        inpCount: 0
      }
    }
  })
  app.mount('#app')
</script>
```

No exemplo acima, temos o `v-on:input`, que realiza ações baseadas no evento de digitação de valores em um campo do `input`. A cada tecla digitada, um evento é disparado. Outros eventos podem ser associados ao `v-on`, como `click` e `mousemove`. Também podemos usar uma abreviação para o `v-on`, escrita como um arroba `@`. No nosso exemplo acima, teríamos `@:input` como sintaxe abreviada.

v-model

A diretiva **v-model** é direcionada a trabalhar com formulários. Esse sempre é um tema delicado no JavaScript, já que é um tema um pouco trabalhoso e complexo para se lidar. A diretiva cria um link entre o elemento de `input` e um valor de dado dentro da instância Vue. Quando o valor de `input` é modificado, o valor do dado é atualizado - porém, quando o valor do dado é atualizado, o valor do `input` também é, caracterizando o que chamamos de *two-way binding*, ou vinculação de duas mãos.

Vamos a um exemplo para deixar tudo mais claro.

```
HTML

<div id="app">
  <input type="text" v-model="inpText">
  <p> {{ inpText }} </p>
</div>

<script>
  const app = Vue.createApp({
    data() {
      return {
        inpText: 'Texto inicial'
      }
    }
  })
  app.mount('#app')
</script>
```

Temos um campo de *input* do tipo texto. Ele será inicializado com o valor padrão "Texto inicial" e qualquer alteração tanto no campo, quanto no valor *inpText* alocado no *script*, irão refletir um no outro.

(FUNDATEC/CIGA SC/2020) Analise o trecho de código apresentado na Figura 5 abaixo, retirado de uma aplicação VueJS:

```
<html>
<head>
<script
src="https://cdn.jsdelivr.net/npm/vue/dist/vue.js"></script>
</head>
<body>

<div id="app">
<input type="radio" id="A" value="A" v-model="escolhida">
<label for="one">A</label>
<br>
<input type="radio" id="B" value="B" v-model="escolhida">
<label for="B">B</label>
<br>
<input type="radio" id="C" value="C" v-model="escolhida">
<label for="C">C</label>
<br><br>
<span>Alternativa                                escolhida:
<strong>{{ escolhida }}</strong></span>
</div>

<script>
var app = new Vue({
  el: '#app',
  data: {
    escolhida: 'Nenhuma'
  }
})
</script>
</html>
```

Figura 5

É correto afirmar que:

- a) Para que uma mensagem apareça corretamente no navegador com o valor do "radio button" selecionado, é necessário implementar um "listener" vinculado ao evento "onchange".
- b) VueJS faz uma ligação unidirecional entre o valor e o banco de dados por meio da diretiva "v-model".
- c) VueJS faz uma ligação bidirecional entre o valor e a camada de acesso ao banco de dados por meio da diretiva "v-model".
- d) VueJS faz uma ligação bidirecional entre o valor dos elementos "input" e os dados, ao mesmo tempo em que mostra o valor na tela utilizando sintaxe de template.
- e) VueJS faz uma ligação bidirecional entre o valor e os dados e os elementos "input", mas é necessário utilizar a propriedade "computed" do componente "app" no lugar da propriedade "data".

Comentários:

Analisando o código, podemos perceber o uso da diretiva v-model - que é responsável por fazer um *two way binding*, ou uma vinculação bidirecional entre um valor de *input* e uma variável armazenada nos dados. Nesse caso, a variável armazenada é chamada de 'escolhida', e inicia com o valor 'Nenhuma'. Qualquer alteração em ambas as pontas, no *input* ou no dado, implicam alteração na outra ponta. Vamos analisar as alternativas.

- a) Errado. O v-model já faz esse trabalho, sem necessidade de implementar um listener.
- b) Errado. A ligação é bidirecional.
- c) Errado. A ligação é feita com o dado, e não com a camada de acesso ao banco de dados.
- d) Certo.
- e) Errado.

Single-File Components (SFC)

Os **Single File Components (SFCs)** são uma das características mais poderosas e distintivas do Vue.js. Eles permitem encapsular o código relacionado a um componente Vue em um único arquivo, geralmente com a extensão `.vue`.

Este arquivo contém três seções principais:

- `<template>` → onde alocamos o conteúdo HTML
- `<script>` → onde o código Vue está alocado
- `<style>` → onde os estilos CSS ficam alocados

Lembrando que podemos ter os arquivos HTML, Vue e CSS criados em diferentes arquivos separados, de forma modular, mas importamos todos para um mesmo arquivo HTML para fazer basicamente uma “montagem”.

(CEBRASPE/BANRISUL/2022) A respeito de tecnologias e protocolos web, julgue o item que se segue.

Em aplicações construídas com VueJS, os componentes HTML, CSS e JavaScript são consolidados em um único arquivo.

Comentários:

Perfeito! Esse arquivo é o que chamamos de Single File Component (SFC). (Gabarito: Certo)

Componentes

Até agora falamos muito em componentes - mas, afinal, o que é um componente?

No Vue.JS, os componentes são pequenas partes de código, que recebem uma parte específica da aplicação. Por exemplo, um botão, um menu ou um formulário. Com isso, conseguimos decompor a página em partes menores, trazendo mais facilidade na manutenção e criação de aplicações.

Com os componentes, conseguimos trabalhar sua lógica de maneira isolada dos outros, focando na sua funcionalidade e integridade. É como uma pequena parte do código, que é juntada a diversos outros componentes na “aplicação final”, quase como um brinquedo de Lego.

Como o SFC também é um componente, você não deve estranhar em saber que a estrutura de um componente qualquer também é composta das mesmas três seções - `template`, `script` e `style`.

Veja um exemplo de código criando um componente qualquer:

```
HTML

<!-- template html -->
<template>
  <button @click="handleClick" :class="buttonClass">
    {{ label }}
  </button>
</template>

<!-- script -->
<script>
export default {
  name: 'MeuBotao',
  props: {
    label: {
      type: String,
      default: 'Clique aqui'
    },
    type: {
      type: String,
      default: 'button'
    },
    buttonClass: {
      type: String,
      default: ''
    }
  },
  methods: {
    handleClick() {
      this.$emit('click');
    }
  }
}
</script>

<!-- estilos -->
<style>
</style>
```

Veja que usamos as abreviações @ para v-on, e : para v-bind. Com esse código, criamos um botão que pode ser replicado em várias partes do código, trazendo modularidade e facilidade na manutenção - já que podemos realizar a manutenção diretamente no código do botão.

(Inédita/Prof. Felipe Mathias) As diretivas do VueJS são formas de conectar elementos HTML com os dados do Vue. Duas das principais diretivas, a v-bind e v-on, podem ser também escritas de forma abreviada. Assinale a alternativa que indica a forma de abreviação para as duas diretivas, respectivamente.

- a) - e +
- b) : e @
- c) + e -
- d) @ e :
- e) :: e --

Comentários:

Questão tranquila. Para abreviarmos o v-bind, usamos dois pontos :. Para abreviarmos o v-on, usamos o arroba @. Portanto, correta a letra B. (Gabarito: Letra B)

QUESTÕES COMENTADAS

01. (FUNDATEC/CIGA SC/2023) Qual alternativa descreve melhor a utilidade dos atributos props em VueJS?

- a) Proteger variáveis locais de acessos por componentes externos.
- b) Uma forma de aplicar automaticamente classes CSS a elementos.
- c) Uma forma de passar dados de um componente pai para um componente filho.
- d) Estabelecem relação de herança entre componentes reativos.
- e) Eventos que são disparados quando um elemento é alterado.

Comentários:

As *props* são formas de passar dados (propriedades) de um componente pai para seus componentes filhos, numa comunicação vertical. A alternativa que melhor descreve isso é a letra C.

Gabarito: Letra C

02. (NOSSO RUMO/CRM PR/2023) No Vue.js, a parte que conterá o corpo da página como um todo e que é amplamente usada pelo Bootstrap para agrupar seus elementos de forma hierárquica é chamada de

- a) Tag head.
- b) Div container.
- c) Lista ul.
- d) Div input-output.
- e) Tag whole.

Comentários:

Em SPAs, como é o caso do Vue, compreendemos nossa aplicação em uma <div> especial. Ela é chamada de div container, pois compreende toda nossa aplicação - e ela a resposta dessa questão.

Gabarito: Letra B

03. (CETREDE/UFC/2022) Sobre a configuração de um ambiente de desenvolvimento baseado na linguagem JavaScript para a plataforma Windows, marque a alternativa correta.

- a) Apesar de ser útil para criar aplicativos web SPA, o React também pode ser usado para criação de aplicações Plataforma Universal do Windows (UWP).

- b) A configuração de um ambiente de desenvolvimento JavaScript no Windows tem como requisito obrigatório a configuração do Subsistema do Windows para Linux (WSL).
- c) O único gerenciador de pacotes disponível para desenvolvedores no framework Vue.js é o npm.
- d) O único gerenciador de pacotes disponível para desenvolvedores no framework Vue.js é o Yarn.

Comentários:

Vamos analisar cada item.

- a) Certo. A UWP é a plataforma do Windows para dispositivos móveis. Nesse caso, o React, mais especificamente o React Native, também pode ser usado para criar aplicações para dispositivos móveis numa abordagem híbrida.
- b) Errado. Não há essa exigência.
- c) e d) Errado. O Vue usa tanto Yarn quanto npm, dentre outros gerenciadores.

Portanto, correta a letra A.

Gabarito: Letra A

04. (FCC/TJ CE/2022) Um Analista foi requisitado para corrigir uma aplicação construída com Vue.js e, para realizar este trabalho, estudou os princípios e as funcionalidades deste framework, em que consta, dentre outros, que:

- a) Para renderizar condicionalmente um bloco, utilizam-se as diretivas `vueIf`, `vueElse` e `vueElseif`.
- b) Todo aplicativo Vue começa criando-se uma nova instância da aplicação com a função `createApp`.
- c) Uma instância da aplicação renderiza todo o aplicativo antes do método `.mount()` ser chamado.
- d) Para definir um manipulador de erros (`errorHandler`) utiliza-se um objeto `catchComponent`.
- e) Deve haver somente uma instância de aplicação na mesma página.

Comentários:

Vamos analisar cada item.

- a) Errado. As diretivas utilizadas no Vue.js para renderização condicional são `v-if`, `v-else-if` e `v-else`.
- b) Certo. No Vue.js 3, um aplicativo começa criando uma nova instância da aplicação com a função `createApp`.

- c) Errado. No Vue.js, o aplicativo é renderizado após o método `.mount()` ser chamado. O método `.mount()` é responsável por anexar a instância Vue ao elemento DOM especificado.
- d) Errado. Para definir um manipulador de erros global no Vue.js, utiliza-se a propriedade `errorHandler` na instância de aplicação. Não existe um objeto chamado `catchComponent` para esse propósito.
- e) Errado. Pode haver múltiplas instâncias de aplicação na mesma página, embora geralmente seja prática comum ter uma única instância de aplicação para simplicidade e melhor organização do código.

Portanto, correta a letra B.

Gabarito: Letra B

05. (CEBRASPE/BANRISUL/2022) A respeito de tecnologias e protocolos web, julgue o item que se segue.

Em aplicações construídas com VueJS, os componentes HTML, CSS e JavaScript são consolidados em um único arquivo.

Comentários:

Perfeito! São os Single-File Components, que consolidam todos os componentes num arquivo só.

Gabarito: Certo

06. (CEPUERJ/UERJ/2021) O trecho de código a seguir foi retirado de uma página web construída utilizando vue.js e seus componentes:

```
<script src="https://cdn.jsdelivr.net/npm/vue"></script>
<div id="app-1">
  <ol>
    <todo-item v-for="item in codeList" v-bind:todo="item" v-bind:key="item.id"></todo-item>
  </ol>
  <button v-on:click="incluir">Adicionar PHP</button>
  <button v-on:click="modificar(5)">Modificar Vue</button>
</div>
<script>
  Vue.component('todo-item', {
    props: ['todo'],
    template: '<li>[[{todo.id}]] [[{todo.text}]]</li>'
  })
  var appl = new Vue({
    el: '#app-1',
    data: {
      codeList: [
        {id: 0, text: 'JavaScript'},
        {id: 1, text: 'Vue'},
        {id: 2, text: 'CodeIgniter'}
      ]
    },
    methods: {
      incluir: function () {
        if (appl.codeList.findIndex(x => x.text === 'PHP') == -1) {
          appl.codeList.push({id: 3, text: 'PHP'})
        }
      },
      modificar: function (num) {
        Vue.set(appl.codeList, appl.codeList.findIndex(x => x.id === num), {id: 1, text: 'Vue.js'})
      }
    }
  })
</script>
```

O resultado da execução desse código é a figura a seguir:



Essa figura ilustra a tela apresentada ao usuário quando, após a renderização inicial, é escolhida a opção:

- a) "Modificar Vue"
- b) "Adicionar PHP"
- c) "Modificar Vue" e, em seguida: "Adicionar PHP"
- d) "Adicionar PHP" e, em seguida: "Modificar Vue"

Comentários:

Temos dois botões com a diretiva v-on, usada para interagir com eventos. O botão "Adicionar PHP", que está ligado ao método incluir, e o botão "Modificar Vue", que está ligado ao método modificar.

Se clicarmos no botão Adicionar PHP, a lista que será printada é acrescida do valor PHP, ficando:

[0] JavaScript
[1] Vue
[2] Codelgniter
[3] PHP

E não é esse o nosso resultado - estamos apenas com modificar. Nesse caso, significa que estamos apertando o botão "Modificar Vue", que tem como retorno a lista base - teoricamente, modificaríamos o valor do *index* 5, mas como ele não existe, não temos nenhuma alteração.

Gabarito: Letra A

07. (Instituto AOCP/SANESUL/2021) O Vue.js é considerado um framework de JavaScript Progressivo. Utilizando-se dele, qual é a função do método global `Vue.extend(options)`?

- a) Criar uma "subclasse" do construtor base do Vue. O argumento deve ser um objeto contendo opções do componente.
- b) Adia o callback para ser executado depois do próximo ciclo de atualização do DOM.
- c) Adiciona uma propriedade em um objeto reativo, garantindo que a nova propriedade também seja reativa, disparando atualizações na interface.
- d) Remove uma propriedade de um objeto.
- e) Registra ou recupera uma diretiva global.

Comentários:

A função do método global `Vue.extend(options)` no Vue.js é criar uma "subclasse" do construtor base do Vue, permitindo que você estenda as funcionalidades dos componentes Vue com novas opções. O argumento `options` deve ser um objeto que contenha as opções do componente que você deseja definir. Portanto, correta a letra A.

Gabarito: Letra A

08. (UFC/CCV/2019) Sobre o framework Vue.js, assinale a alternativa correta.

Alternativas

- a) Vue.js é um framework escrito em Java.
- b) Templates são instâncias reutilizáveis do Vue com um nome.

- c) Vue.js é considerado um framework de backend (server side).
- d) Vue.js possui estruturas que possibilitam utilizar condicionais, mas não possui estruturas de laço.
- e) Os templates do Vue.js são compostos por HTML válido que podem ser compilados por navegadores compatíveis com as especificações e também por compiladores HTML.

Comentários:

Vamos analisar cada item.

- a) Errado. Vue.js é um framework escrito em JavaScript, não em Java.
- b) Errado. Templates em Vue.js são blocos de código HTML que definem a estrutura visual de componentes, mas não são instâncias reutilizáveis do Vue com um nome.
- c) Errado. Vue.js é um framework de frontend (client side) usado para construir interfaces de usuário.
- d) Errado. Vue.js possui estruturas tanto para condicionais (v-if, v-else-if, v-else) quanto para laços (v-for).
- e) Certo. Os templates do Vue.js são compostos por HTML válido e podem ser compilados tanto por navegadores compatíveis quanto por compiladores HTML.

Portanto, correta a letra E.

Gabarito: Letra E

QUESTÕES COMENTADAS

01. (FUNDATEC/CIGA SC/2023) Qual alternativa descreve melhor a utilidade dos atributos props em VueJS?

- a) Proteger variáveis locais de acessos por componentes externos.
- b) Uma forma de aplicar automaticamente classes CSS a elementos.
- c) Uma forma de passar dados de um componente pai para um componente filho.
- d) Estabelecem relação de herança entre componentes reativos.
- e) Eventos que são disparados quando um elemento é alterado.

02. (NOSSO RUMO/CRM PR/2023) No Vue.js, a parte que conterá o corpo da página como um todo e que é amplamente usada pelo Bootstrap para agrupar seus elementos de forma hierárquica é chamada de

- a) Tag head.
- b) Div container.
- c) Lista ul.
- d) Div input-output.
- e) Tag whole.

03. (CETREDE/UFC/2022) Sobre a configuração de um ambiente de desenvolvimento baseado na linguagem JavaScript para a plataforma Windows, marque a alternativa correta.

- a) Apesar de ser útil para criar aplicativos web SPA, o React também pode ser usado para criação de aplicações Plataforma Universal do Windows (UWP).
- b) A configuração de um ambiente de desenvolvimento JavaScript no Windows tem como requisito obrigatório a configuração do Subsistema do Windows para Linux (WSL).
- c) O único gerenciador de pacotes disponível para desenvolvedores no framework Vue.js é o npm.
- d) O único gerenciador de pacotes disponível para desenvolvedores no framework Vue.js é o Yarn.

04. (FCC/TJ CE/2022) Um Analista foi requisitado para corrigir uma aplicação construída com Vue.js e, para realizar este trabalho, estudou os princípios e as funcionalidades deste framework, em que consta, dentre outros, que:

- a) Para renderizar condicionalmente um bloco, utilizam-se as diretivas vuela, vueElse e vueElseif.
- b) Todo aplicativo Vue começa criando-se uma nova instância da aplicação com a função createApp.

- c) Uma instância da aplicação renderiza todo o aplicativo antes do método .mount() ser chamado.
- d) Para definir um manipulador de erros (errorHandler) utiliza-se um objeto catchComponent.
- e) Deve haver somente uma instância de aplicação na mesma página.

05. (CEBRASPE/BANRISUL/2022) A respeito de tecnologias e protocolos web, julgue o item que se segue.

Em aplicações construídas com VueJS, os componentes HTML, CSS e JavaScript são consolidados em um único arquivo.

06. (CEPUERJ/UERJ/2021) O trecho de código a seguir foi retirado de uma página web construída utilizando vue.js e seus componentes:

```
<script src="https://cdn.jsdelivr.net/npm/vue"></script>
<div id="app-1">
  <ol>
    <todo-item v-for="item in codeList" v-bind:todo="item" v-bind:key="item.id"></todo-item>
  </ol>
  <button v-on:click="incluir">Adicionar PHP</button>
  <button v-on:click="modificar(5)">Modificar Vue</button>
</div>
<script>
  Vue.component('todo-item', {
    props: ['todo'],
    template: '<li>[[{todo.id}]] [[{todo.text}]]</li>'
  })
  var appl = new Vue({
    el: '#app-1',
    data: {
      codeList: [
        {id: 0, text: 'JavaScript'},
        {id: 1, text: 'Vue'},
        {id: 2, text: 'CodeIgniter'}
      ]
    },
    methods: {
      incluir: function () {
        if (appl.codeList.findIndex(x => x.text === 'PHP') == -1) {
          appl.codeList.push({id: 3, text: 'PHP'})
        }
      },
      modificar: function (num) {
        Vue.set(appl.codeList, appl.codeList.findIndex(x => x.id === num), {id: 1, text: 'Vue.js'})
      }
    }
  })
</script>
```

O resultado da execução desse código é a figura a seguir:

1. [0] JavaScript
2. [1] Vue
3. [2] CodeIgniter

Adicionar PHP Modificar Vue

Essa figura ilustra a tela apresentada ao usuário quando, após a renderização inicial, é escolhida a opção:

- a) "Modificar Vue"
- b) "Adicionar PHP"
- c) "Modificar Vue" e, em seguida: "Adicionar PHP"
- d) "Adicionar PHP" e, em seguida: "Modificar Vue"

07. (Instituto AOCP/SANESUL/2021) O Vue.js é considerado um framework de JavaScript Progressivo. Utilizando-se dele, qual é a função do método global `Vue.extend(options)`?

- a) Criar uma "subclasse" do construtor base do Vue. O argumento deve ser um objeto contendo opções do componente.
- b) Adia o callback para ser executado depois do próximo ciclo de atualização do DOM.
- c) Adiciona uma propriedade em um objeto reativo, garantindo que a nova propriedade também seja reativa, disparando atualizações na interface.
- d) Remove uma propriedade de um objeto.
- e) Registra ou recupera uma diretiva global.

08. (UFC/CCV/2019) Sobre o framework Vue.js, assinale a alternativa correta.

Alternativas

- a) Vue.js é um framework escrito em Java.
- b) Templates são instâncias reutilizáveis do Vue com um nome.
- c) Vue.js é considerado um framework de backend (server side).
- d) Vue.js possui estruturas que possibilitam utilizar condicionais, mas não possui estruturas de laço.
- e) Os templates do Vue.js são compostos por HTML válido que podem ser compilados por navegadores compatíveis com as especificações e também por compiladores HTML.

GABARITO

GABARITO

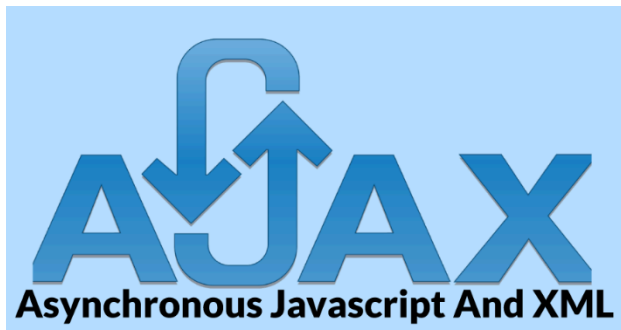


1. Letra C
2. Letra B
3. Letra A
4. Letra B

5. Certo
6. Letra A
7. Letra A
8. Letra E

AJAX

Conceitos Gerais



O **AJAX**, ou **Asynchronous Javascript and XML**, é um método de implementação de XML e JavaScript de forma assíncrona, de forma a termos páginas mais interativas. Seu uso **permite atualizar partes específicas de uma página sem a necessidade de recarregar a página inteira**. Essa abordagem possibilita uma interação mais dinâmica e responsiva entre o usuário e a aplicação web.

A principal característica do AJAX é a capacidade de realizar requisições assíncronas ao servidor, ou seja, o navegador pode enviar e receber dados do servidor em segundo plano, enquanto o usuário interage com a página. Isso é feito utilizando a **API XMLHttpRequest**, embora atualmente seja comum o uso de APIs mais modernas, como a Fetch API.

(VUNESP/DPE SP/2023) Sobre o uso do AJAX em uma página web, pode-se citar, como uma de suas vantagens,

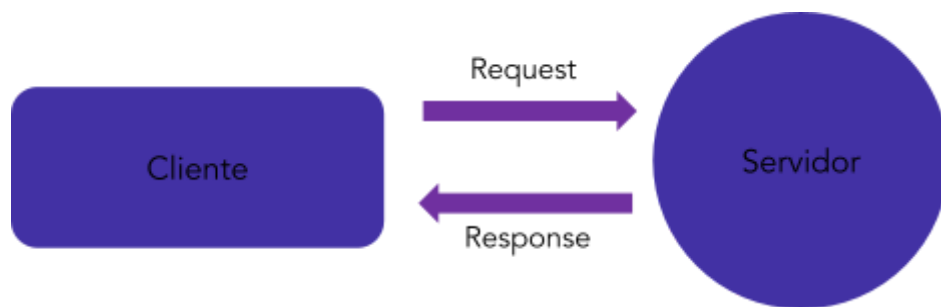
- a) que a comunicação entre cliente e servidor é criptografada com as credenciais do AJAX.
- b) que o conteúdo da página pode ser atualizado sem recarregar a página.
- c) a substituição do HTML + CSS pelo próprio AJAX.
- d) o uso de um algoritmo de tunelamento para maior velocidade de comunicação.
- e) a adoção do protocolo UDP na comunicação.

Comentários:

O AJAX é responsável por implementar uma comunicação assíncrona entre o cliente e o servidor, permitindo que sejam criadas páginas com interatividade, sem a necessidade de recarregar a página. (Gabarito: Letra B)

Antes de entendermos sobre o AJAX, precisamos entender dois tópicos. O primeiro deles é entender **o que são requisições HTTP**. HTTP é um protocolo de rede, atuando na camada de aplicação do modelo OSI, responsável por definir como as mensagens são formatadas e transmitidas entre os clientes e os servidores web. Essas requisições são a forma como os clientes solicitam recursos dos servidores, ou enviam recursos a eles.

Para realizar requisições HTTP, trabalhamos num sistema de **solicitação e resposta**, ou como é comumente referenciado pelo termo em inglês, **request/response**. O cliente faz uma requisição ao servidor, que pode ser o pedido de informações ou o envio delas, e o servidor responde essa requisição.



Essas interações são feitas a partir dos **métodos HTTP**. Dentre os diversos métodos existentes, temos 5 que se destacam quando estamos falando de programação Web:

- **GET:** Este método é utilizado para solicitar dados de um recurso específico no servidor. Por exemplo, quando você digita uma URL em seu navegador e pressiona Enter, o navegador envia uma requisição GET para o servidor solicitando a página web correspondente.
- **POST:** O método POST é utilizado para enviar dados ao servidor para processamento. É comumente usado em formulários web, onde os dados preenchidos pelo usuário são enviados para o servidor para serem processados. Ao contrário do método GET, os dados enviados via POST não são exibidos na URL, o que os torna mais adequados para o envio de dados sensíveis.
- **PUT:** Este método é usado para atualizar um recurso no servidor. Ele substitui completamente o recurso existente pelo novo que está sendo enviado na requisição PUT.
- **DELETE:** Como o nome sugere, este método é utilizado para remover um recurso do servidor.
- **PATCH:** O método PATCH é semelhante ao PUT, mas em vez de substituir completamente o recurso, ele é usado para aplicar modificações parciais ao recurso.

Métodos HTTP

GET

POST

DELETE

PUT

PATCH

O segundo passo é entender o que é uma **comunicação assíncrona**. No fluxo tradicional de comunicação, ao realizarmos uma comunicação com um elemento externo - no caso, uma requisição HTTP -, precisaríamos esperar todo o processamento da resposta chegar para dar seguimento ao programa. Isso resulta em aplicações de **baixa coesão** e **alto acoplamento**.

Para contornarmos esse "problema", surge a comunicação assíncrona: o cliente faz uma requisição ao servidor e, logo em seguida, sem a chegada da resposta, continua no

processamento do programa. O processo que necessita da resposta é levado a um segundo plano, e retomado após a chegada da resposta. Assim, temos **alta coesão** e **baixo acoplamento** - que é o ideal para aplicações.

Você lida com comunicações síncronas e assíncronas todo dia: suas ligações telefônicas são comunicações síncronas, enquanto os e-mails que você troca com seu chefe fazem parte da comunicação assíncrona. E é justamente na implementação da comunicação assíncrona que o AJAX trabalha.

QUESTÃO DE PROVA



(CEBRASPE/DPE RO/2022) A metodologia Ajax para desenvolvimento web engloba os componentes

- HTML, document object model e JavaScript.
- JavaScript, document object model e PHP.
- JavaScript, XML e PHP.
- PHP, HTML e document object model.
- XML, PHP e HTML.

Comentários:

O AJAX é uma tecnologia para implementação de comunicações assíncronas, baseado em JavaScript. Por isso, no contexto de uma página web, ao usarmos o AJAX, estamos usando o HTML, o DOM (Document Object Model), responsável por organizar os objetos do HTML, e o JavaScript, que irá fazer uma ponte entre o AJAX e o HTML, além de operacionalizar o próprio AJAX. (Gabarito: Letra A)

Formatos de Arquivos

Apesar do nome ser "Asynchronous Javascript And XML", esse não é único formato de arquivos envolvido nas operações. Na verdade, temos dois formatos de arquivos preponderantes na comunicação: os arquivos XML, e os arquivos JSON. Vamos entender um pouco mais sobre esses formatos.

XML, ou eXtensible Markup Language, é uma linguagem de marcação. Ela é responsável por criar arquivos com uma finalidade bem específica: armazenar e transportar dados. O XML por si só não faz nada - é só um arquivo de texto, composto por conjunto de dados envolto em tags (que se parecem com `<isso>` e `</isso>`).

A grande diferença do XML para outras linguagens de marcação, como o HTML, é que ele não conta com nenhuma estrutura pré-definida. As tags podem ser quaisquer que atenderem à necessidade daquele arquivo - e usualmente são definidas em um arquivo de estrutura, chamado de XML Schema.

Outro ponto importante para garantir a hierarquia correta de elementos num XML é o uso correto da **indentação**. Um exemplo de bloco de código XML seria esse retratado abaixo:

```
<curso>
  <nome>Desenvolvimento de sistemas</nome>
  <professor>Felipe Mathias</professor>
  <ID>123456</ID>
</curso>
```

Já o JSON, ou JavaScript Object Notation, é um formato de arquivo-padrão para troca de dados simples, voltado especificamente para a notação de objetos na linguagem JavaScript. Porém, apesar da notação estrutural ser a mesma, o JSON é um formato de dados independente da linguagem de implementação.

Nele, organizamos os valores da mesma forma que organizamos os objetos do JavaScript - dentro de chaves { } e em pares de chave: valor. Por exemplo, podemos ter informações acerca da minha pessoa ou dessa aula:

```
{
  "curso": "Desenvolvimento de sistemas",
  "professor": "Felipe Mathias",
  "ID": "123456"
}
```

Em tempos recentes, o JSON vem sendo preponderante nas relações da internet devido à sua **baixa verbosidade**. Verbo­sidade em programação refere-se à quantidade de código necessária

para expressar uma determinada informação ou realizar uma tarefa. Um código verboso é longo, repetitivo e contém muitos detalhes desnecessários, enquanto um código conciso é mais curto, direto e expressa a mesma informação de forma mais sucinta.

Quando dizemos que um formato de dados, como JSON, é "pouco verboso", significa que ele requer uma quantidade mínima de caracteres para representar uma determinada estrutura de dados. Em outras palavras, ele é compacto e expressa as informações de forma direta e eficiente.

JSON → POUCO VERBOSO

XML → MUITO VERBOSO

QUESTÃO DE PROVA



(CESGRANRIO/LIQUIGÁS/2015) A interoperabilidade entre sistemas pode ser obtida por meio de linguagens padronizadas para o intercâmbio de dados, tais como:

- a) JSON e XML
- b) JSON e XSL
- c) XML e XSL
- d) XML e XSLT
- e) XSL e XSLT

Comentários:

Para termos interoperabilidade entre sistemas, usamos linguagens de transporte de dados que são independentes da linguagem implementada nos sistemas - nesse caso, usamos XML e JSON. Quanto às demais apresentadas na alternativa:

XSL: XSL (Extensible Stylesheet Language) é uma linguagem de folhas de estilo usada para transformar documentos XML em outros formatos, como HTML e PDF

XSLT: É uma linguagem de transformação que permite definir regras para a conversão de documentos XML em outros formatos

A correta é, portanto, a letra A. (Gabarito: Letra A)

Operacionalizando o AJAX

Agora que você, Coruja, entendeu tudo que envolve os conhecimentos básicos para usarmos o AJAX, vamos ver como ele é implementado. Para que a requisição HTTP possa ser realizada, é necessário que tenhamos uma classe que forneça essa funcionalidade. É aí que entra o ponto central do AJAX: o `XMLHttpRequest`.

Essa classe permite que se façam requisições a um servidor, intermediando a saída da solicitação por parte do cliente, e a saída da resposta por parte do servidor. Em aplicações mais modernas, essa parte é usualmente implementada pela API Fetch, mas o `XMLHttpRequest` ainda fornece uma abordagem interessante, devido à sua compatibilidade com os navegadores e à **abordagem assíncrona**.

(CONSUPLAN/2022/MPE PA) O Ajax utiliza algumas tecnologias bastante conhecidas com o objetivo de melhorar a interatividade entre o usuário e o servidor. A tecnologia utilizada pelo Ajax que, na verdade, é um objeto JavaScript que torna possível a comunicação assíncrona com o servidor, sem a necessidade de recarregar a página por completo, é:

- a) CSS
- b) XML
- c) DOM
- d) `XMLHttpRequest`

Comentários:

O AJAX utiliza o `XMLHttpRequest`, uma API presente em navegadores responsável por receber as requisições e as encaminhar aos servidores. (Gabarito: Letra D)

Criando o objeto `XMLHttpRequest`

O primeiro passo para operacionalizarmos o AJAX, é **definir uma variável para receber o objeto `XMLHttpRequest`**. Isso é feito a partir de uma função construtora, veja:

```
JavaScript
var xhr = new XMLHttpRequest()
```

Sempre que você se deparar com `xhr` na aula de hoje, saiba que é o nome da variável que está recebendo o objeto `XMLHttpRequest`.

Em seguida, precisamos definir uma função de callback para ser executada sempre que recebermos uma resposta. Essa função de callback será alocada na propriedade `onload`, do objeto `XMLHttpRequest`. Veja um exemplo:

JavaScript

```
xhr.onload = function() {  
    console.log('Resposta recebida')  
}
```

A depender das interações de pedido e resposta, podemos ter diferentes estados, que são armazenados numa propriedade chamada de **readyState**. Para acompanharmos as mudanças, podemos criar uma função de callback que monitorará todas as mudanças - para isso, usamos o evento **onreadystatechange** (tudo em minúsculo mesmo, sem camel case).

Quando uma solicitação HTTP é feita usando o objeto XMLHttpRequest, o navegador inicia uma série de etapas para processar essa solicitação. Durante esse processo, o estado da solicitação muda várias vezes, e o evento **onreadystatechange** é acionado sempre que o estado da solicitação é alterado.

Veja um exemplo de código, onde atribuímos essa função que lidará com a mudança de estados:

JavaScript

```
xhr.onreadystatechange = function nomeDaFuncao() {}
```

Esse evento possui diferentes estados, que são representados por números inteiros - são os **readyStates**. São apenas 5 estados diferentes, e eu **preciso** que você decore todos - principalmente o estado de número 4.

ATENÇÃO, DECORE!



NÚMERO	DESCRIÇÃO
0	Requisição não inicializada
1	Conexão estabelecida

2	Requisição recebida
3	Processando requisição
4	Requisição finalizada, e resposta está pronta

(VUNESP/TCM SP/2023) Em uma requisição AJAX, o evento `onreadystatechange`, de um objeto `XMLHttpRequest`, é acionado no máximo

- a) 1 vez.
- b) 2 vezes.
- c) 3 vezes.
- d) 4 vezes.
- e) 5 vezes.

Comentários:

Temos 5 acionamentos possíveis do `onreadystatechange`, variando de 0 a 4. Apesar disso, a banca, erroneamente, apontou como gabarito a letra D, com 4 mudanças - provavelmente não contaram a mudança para o estado = 0. (Gabarito oficial: Letra D. Gabarito do Professor: Letra E)

Um exemplo de função que podemos implementar seria uma verificação de solicitação bem-sucedida. Veja:

```
JavaScript

xhr.onreadystatechange = function nomeDaFuncao() {
  if (xhr.readyState === 4 && xhr.status === 200){
    console.log('Requisição bem-sucedida!')
  }
}
```

Nesse caso, verificaremos se o `readyState` é igual a 4, caracterizando que a operação foi completada. Além disso, estamos verificando um outro objeto - o `status`. Esse `status` é o responsável por apontar o estado da solicitação, de acordo com o padrão notacional do HTTP. Veja algumas das respostas possíveis:

ESQUEMATIZANDO



CÓDIGO	FRASE	DESCRIÇÃO
INFORMATIVO		

100	Continue	Solicitação recebida, cliente pode prosseguir
101	Switching	Servidor está atendendo a solicitação para alterar os protocolos do cabeçalho
SUCESSO		
200	OK	Solicitação bem-sucedida
201	Created	URL nova criada
202	Accepted	Solicitação aceita, mas ainda não pode ser executada
204	No contente	Conteúdo inexistente para a solicitação
REDIRECIONAMENTO		
301	Moved permanently	URL solicitada não está mais em uso pelo servidor
302	Moved temporarily	URL solicitada foi temporariamente movida
304	Not modified	Documento não foi modificado
ERRO NO CLIENTE		
400	Bad request	Erro de sintaxe na solicitação
401	Unauthorized	Solicitação não tem autorização para ser executada
403	Forbidden	Serviço negado
404	Not found	Documento não foi encontrado
405	Method not allowed	Método solicitado não é suportado pelo URI
406	Not acceptable	Formato solicitado não é aceitável
ERRO NO SERVIDOR		
500	Internal server error	Erro interno, relacionado ao servidor

501	Not implemented	Ação solicitada não pode ser executada
503	Service unavailable	Serviço indisponível

Por fim, também é possível utilizar a propriedade `statusText`, que retornará o texto associado ao código de `status`, como "Not found" ou "Internal server error".

Métodos de Requisição

Após ter sido declarado o que acontecerá após a resposta, podemos definir a requisição de fato. Para isso, temos dois métodos distintos - o `open()` e o `send()`. O `open()` "abre" o pedido, com a seguinte sintaxe:

```
xhr.open(método, url, async)
```

Onde:

- **Método:** define o tipo de *request* que está sendo feita, podendo ser GET, POST, PUT etc.
- **URL:** define o caminho da requisição - seja para inserir ou para recuperar um dado
- **Async:** indica se a comunicação será assíncrona (recebendo o valor *true*), ou síncrona (recebendo o valor *false*).

QUESTÃO DE PROVA



(FCC/MPE PB/2023) Em uma aplicação web que utiliza AJAX, considere a existência de um objeto `http_obj` do tipo `XMLHttpRequest` inicializado e em condições ideais. Nestas condições, para enviar uma requisição para o arquivo `dados.jsp` no servidor, um analista utilizou as instruções abaixo.

```
http_obj.open(a, b, c);  
http_obj.send();
```

Considerando a sintaxe correta do método `open` e que `a`, `b` e `c` são variáveis que contêm valores válidos, a variável

- a) a deve conter dados.jsp.
- b) c deve conter dados.jsp.
- c) a deve conter GET ou POST.
- d) c deve conter GET ou POST.
- e) b deve conter true ou false.

Comentários:

Vamos analisar as alternativas.

- a) Errado. O campo *a* é destinado a receber o método.
- b) Errado. O campo *c* recebe valores true ou false, indicando se é uma comunicação assíncrona ou não.
- c) Certo. Vide letra A.
- d) Errado. Vide letra A.
- e) Errado. O campo *b* recebe a url, o caminho para o arquivo.

Portanto, a alternativa correta é a letra C. (Gabarito: Letra C)

Então, a abertura de uma requisição para GET tem a seguinte forma:

```
JavaScript
xhr.open('GET', 'https://api.exemplo.com', true)
```

O `open()` sinalizará o que iremos fazer. Para enviá-la, usamos a função `send()`. Porém, isso é um processo "genérico"... entre o `open()`, que indica a abertura, e o `send()`, que indica o envio, temos o conteúdo do método. Principalmente se tratando do uso de métodos POST, onde queremos armazenar algo, utilizamos o método `setRequestHeader`.

Ele é uma função pertencente ao XMLHttpRequest que **fornece informações adicionais** acerca da solicitação, e é usado para definir os valores dos cabeçalhos HTTP a serem enviados. Uma sintaxe genérica para a função é:

```
xhr.setRequestHeader(header, value)
```

Onde:

- **Header:** uma string representando o nome do cabeçalho HTTP que você deseja definir
- **Value:** o valor associado ao cabeçalho que você está definindo

De forma geral, combinando os três métodos, teremos algo parecido com isso:

JavaScript

```
xhr.open('POST', 'https://api.exemplo.com/submit', true);  
xhr.setRequestHeader('content-type', 'application/json');  
xhr.send();
```

(FGV/BANESTES/2021) Numa aplicação web, considere o caso em que um arquivo intitulado xpto.txt é obtido do servidor sem que a página inteira seja recarregada.

A estrutura básica correta do código para o uso da tecnologia Ajax nesse caso é:

- a) `const xhttp = new XMLHttpRequest();
xhttp.open("GET",
"xpto.txt"); xhttp.send();`
- b) `const xhttp = new HttpRequest();
xhttp.onload = function() { ... }
xhttp.open("GET", "
xpto.txt ");
xhttp.send();`
- c) `const xhttp = new XMLHttpRequest();
xhttp.open = function() { ... }
xhttp.send("GET", " xpto.txt ");
xhttp.onload();`
- d) `const xhttp = new XMLHttpRequest();
xhttp.onload = function(){ ... }
xhttp.open("GET", " xpto.txt ");
xhttp.send();`
- e) `const xhttp = new XMLHttpRequest();
xhttp.onload("GET", " xpto.txt ");
xhttp.send();`

Comentários:

O fluxo de uma requisição em AJAX é: (1) atribuir o XMLHttpRequest a uma variável; (2) criar uma função de callback para o onload; (3) abrir a requisição com o método open; (4) enviar a requisição com o método send. De forma geral, usando os nomes usados na questão, teremos:

```
const xhttp = new XMLHttpRequest; (1)  
xhttp.onload = function() {...} (2)  
xhttp.open("GET", "xpto.txt") (3)  
xhttp.send() (4)
```

A alternativa que corretamente descreve as etapas é a letra D. (Gabarito: Letra D)

Resposta

A resposta isso, é, a mensagem que sai do servidor em direção ao cliente após receber uma requisição, é processada com base na requisição. Podemos ter quatro formas diferentes de resposta, em relação ao formato do conteúdo carregado:

- **responseText**: retorna a resposta em uma *string* de texto
- **responseXML**: retorna a resposta com dados em formato XML
- **Blob**: retorna dados binários, como imagens, arquivos ou vídeos
- **FormData**: envolve dados relacionados a formulários
- **ArrayBuffer**: formato de dados binários, destinado a manipular dados binários brutos

As respostas, a depender da forma que são retornadas, necessitam de diferentes tratamentos para serem utilizadas. O formato em texto pode ser atribuído diretamente a algum objeto HTML, através de uma junção do AJAX com a manipulação do DOM via JavaScript. Veja um exemplo de código com essa ação:

```
JavaScript
//requisição GET
xhr.open('GET', 'https://api.exemplo.com/submit', true)
xhr.send()
//resposta
let novoTexto = xhr.responseText
document.getElementById('textoResposta').innerHTML = novoTexto
```

Com essa função, atribuímos, através do operador *innerHTML*, o conteúdo da resposta será alocado no elemento selecionado.

O conhecimento dos formatos de arquivo é essencial para o correto uso da resposta. Por exemplo, no uso do **responseXML**, podemos selecionar elementos conforme sua tag. Por exemplo, de um arquivo com um conjunto de artistas, podemos selecionar apenas a *tag* representando o nome do artista. Para isso, usamos o `getElementsByTagName("nomeDaTag")`;

Existem dois métodos de resposta pelo lado do servidor:

- **getResponseHeader**: retorna uma informação específica do cabeçalho do recurso
- **getAllResponseHeaders()**: retorna todas as informações de cabeçalho do recurso

jQuery e AJAX

Até agora, toda implementação que vimos foi a partir do JavaScript - que é a principal forma de uso. Porém, é possível implementar o AJAX de uma outra forma: com o jQuery. Ele é uma biblioteca de JavaScript que visa facilitar sua implementação, através de métodos e seletores simplificados.

No jQuery, usamos o cifrão \$ para indicar que estamos acessando suas funções. A sintaxe genérica do AJAX se dá da seguinte forma:

`$(seletor).método(parâmetros)`

Onde:

- **\$** - Indica que estamos acessando os métodos do jQuery
- **(seletor)** - indica qual elemento estamos trabalhando, a partir de seletores CSS
- **método** - define o método utilizado no elemento selecionado

O jQuery traz alguns métodos responsáveis pelas solicitações e respostas com o XMLHttpRequest. O primeiro deles é o **load()**. Ele é responsável por "baixar" dados de um servidor e aloca-los a um objeto, que é indicado pelo seletor. Sua sintaxe genérica é:

`$(seletor).load(URL, data, callback)`

Onde:

- **URL**: indica o caminho onde se encontra o dado
- **DATA**: aponta para quais dados do caminho serão retornados
- **Callback**: uma função de callback que será implementada ao fim do método load()

Veja a diferença da quantidade de linhas que temos de escrever para retornar determinado dado e aloca-lo em um elemento usando o JavaScript e o jQuery.

JavaScript

```
xmlDoc = new XMLHttpRequest()
xmlDoc.onload = function () {
    document.getElementById('p').innerHTML =.responseText
}
xmlDoc.open('GET', 'https://api.exemplo.com', true)
xmlDoc.send()
```

jQuery

```
$('#p').load('https://api.exemplo.com')
```

Vejam que a simplificação é “assustadora”, né? Podemos implementar três respostas diferentes nesse pedido:

- **responseTxt** - a resposta padrão, contém o conteúdo em formato de texto
- **statusTxt** - o status referente à requisição
- **xhr** - o objeto XMLHttpRequest

O jQuery também implementa diretamente os métodos GET e POST e, nesse caso, a sintaxe dos métodos é bem direta. Usamos **get()** para implementar as requisições GET, e **post()**, para as requisições de postagem de arquivo. Ambos recebem os mesmos parâmetros da função load - URL, data e callback.

Por exemplo, podemos usar a função post() para submeter um conjunto de dados de um formulário. A sintaxe funcionará da seguinte forma:

jQuery

```
$(document).ready(function(){  
    // Quando o botão for clicado  
    $('#button').click(function(){  
        // Realizar uma solicitação AJAX POST usando .post()  
        $.post('https://api.exemplo.com', {parametro1: 'valor1',  
parametro2: 'valor2'}, function(response){  
            // Callback executado em caso de sucesso  
            console.log('Requisição AJAX bem-sucedida!');  
            console.log('Resposta do servidor:', response);  
        });  
    });  
});
```

RESUMO

O QUE É O AJAX?

AJAX, ou Asynchronous Javascript And XML, é uma forma de comunicação entre cliente e servidor, implementada a partir do uso de JavaScript e da API de navegadores XMLHttpRequest. Com o AJAX, podemos ter **páginas com comunicação assíncrona**, permitindo mais interatividade **sem a necessidade de recarregar o conteúdo**.

O QUE É O XMLHttpRequest?

O XMLHttpRequest é uma API que fornece uma maneira de fazer solicitações HTTP assíncronas a um servidor web a partir de uma página web, permitindo que o conteúdo seja atualizado dinamicamente sem a necessidade de recarregar a página inteira. Ele é amplamente utilizado em desenvolvimento web para criar aplicativos web interativos e responsivos, permitindo a troca de dados entre o cliente e o servidor de forma eficiente e assíncrona, o que é essencial para a construção de aplicativos modernos.

QUAL O FLUXO DE CÓDIGO PARA UMA REQUISIÇÃO AJAX?

O fluxo de uma requisição AJAX, com JavaScript, é a seguinte: (1) Atribuir o objeto XMLHttpRequest a uma variável; (2) Definir uma função de callback para a resposta da requisição com o onload; (3) Abrir a requisição, definindo o método, URL e outros com o open(); (4) Enviar a requisição com o send(). Em código, teremos a seguinte estrutura:

```
const xhttp = new XMLHttpRequest; (1)
xhttp.onload = function() {...} (2)
xhttp.open("GET", "xpto.txt") (3)
xhttp.send() (4)
```

QUESTÕES COMENTADAS

01. (VUNESP/PREF. MARÍLIA/2023) AJAX utiliza o objeto XMLHttpRequest para fazer a requisição de dados a um servidor. O método open() deste objeto tem como um de seus parâmetros o tipo de requisição HTTP que será utilizada.

Nesse contexto, assinale a alternativa que apresenta requisições válidas.

- a) FIND e DELETE.
- b) MOVE e GREP.
- c) PUT e DEFINE.
- d) GET e GREP.
- e) GET e POST.

Comentários:

O método open(a, b, c) recebe 3 parâmetros - sendo o primeiro deles o tipo de requisição utilizado. Temos várias requisições baseadas nos métodos HTTP, mas as requisições no AJAX são feitas primariamente em dois tipos: GET, que recupera dados do servidor, e POST, que envia os dados ao servidor.

Gabarito: Letra E

02. (FCC/MPE PB/2023) Em uma aplicação web que utiliza AJAX, considere a existência de um objeto http_obj do tipo XMLHttpRequest inicializado e em condições ideais. Nestas condições, para enviar uma requisição para o arquivo dados.jsp no servidor, um analista utilizou as instruções abaixo.

```
http_obj.open(a, b, c);  
http_obj.send();
```

Considerando a sintaxe correta do método open e que a, b e c são variáveis que contêm valores válidos, a variável

- a) a deve conter dados.jsp.
- b) c deve conter dados.jsp.
- c) a deve conter GET ou POST.
- d) c deve conter GET ou POST.
- e) b deve conter true ou false.

Comentários:

O método open(a, b, c) recebe três parâmetros:

- a → o método utilizado (POST ou GET)
- b → a URL de destino

- $c \rightarrow$ se a comunicação é assíncrona (true) ou não (false)

Com isso em mente, vamos às alternativas.

- a) Errado. O endereço dados.jsp é a URL e deve ser inserida em *b*.
- b) Errado. Vide letra A.
- c) Certo. O elemento *a* contém os métodos.
- d) Errado. O método está no elemento *a*.
- e) Errado. O valor *true* e *false* refere-se ao tipo de comunicação - síncrono ou assíncrono, respectivamente. Esse valor é inserido no local do elemento *c*.

Gabarito: Letra C

03. (VUNESP/CM SBO/2023) O seguinte código de uma página HTML com JavaScript utiliza AJAX para a chamada de uma página JSP.

```
<html>
<body>

<div id="d">
<button type="button" onclick="loadDoc()">AJAX</button>
</div>

<script>
function loadDoc() {
    var xhttp = new XMLHttpRequest();
    xhttp.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
            document.getElementById("d").innerHTML = this.responseText;
        }
    };
    xhttp.open("GET", "ajax_info.jsp", true);
    xhttp.send();
}

</script>
</body>
</html>
```

A div identificada por d tem seu conteúdo alterado por meio da chamada à página ajax_info.jsp

- a) assim que a carga da página HTML no navegador é finalizada, apenas uma vez.
- b) durante a carga da página HTML no navegador, que já é renderizada pela primeira vez com o conteúdo retornado pela página ajax_info.jsp.
- c) assim que a carga da página HTML no navegador é finalizada, repetidas vezes, pois o código apresentado provoca um loop infinito que força a recarga da página HTML após cada requisição a ajax_info.jsp.

- d) sem que a página HTML seja recarregada, após o usuário clicar no botão "AJAX".
- e) após o usuário clicar no botão "AJAX", o que provoca a recarga completa da página HTML com o conteúdo atualizado.

Comentários:

O clique no botão aciona a função loadDoc(), que é responsável por fazer uma requisição AJAX do tipo GET. Com isso em mente, vamos às alternativas.

- a) Errado. O acionamento é feito ao clicarmos no botão.
- b) Errado. O acionamento é feito ao clicarmos no botão.
- c) Errado. O acionamento é feito ao clicarmos no botão.
- d) Certo. O AJAX implementa a comunicação assíncrona, que permite inserir elementos atualizáveis isoladamente, sem a necessidade de atualizar a página toda.
- e) Errado. Como estamos implementando o AJAX, apenas essa parte será atualizada, não a página toda.

Gabarito: Letra C

04. (VUNESP/DPE SP/2023) O seguinte trecho de código foi encontrado em uma página HTML

```
<script>
function loadDoc() {
  const xhttp = new XMLHttpRequest();
  xhttp.onload = function() {
    myFunction(this);
  }
  xhttp.open("GET", "arq.xml");
  xhttp.send();
}
...
</script>
```

sendo que a implementação da função myFunction foi propositalmente omitida.
Com base nas informações apresentadas, é correto afirmar que

- a) a função myFunction aplica alguma regra CSS a algum elemento da página.
- b) não é possível concluir se AJAX é utilizado sem conhecer a implementação de myFunction.
- c) é utilizado AJAX para efetuar uma requisição ao servidor web. A chamada xhttp.send() faz com que a página seja recarregada no navegador.
- d) é utilizado AJAX para efetuar uma requisição ao servidor web sem recarregar a página na chamada xhttp.send().
- e) o arquivo arq.xml contém regras de estilo CSS.

Comentários:

Vamos analisar as alternativas.

- a) Errado. A função myFunction é alocada ao *onload*, o que significa que ela é acionada como uma função de callback para a resposta à requisição.
- b) Errado. A função implementada não impede de visualizarmos o uso do AJAX - que é caracterizado pelo XMLHttpRequest().
- c) Errado. A alternativa está quase toda correta - mas o send() não faz com que a página seja recarregada, já que o AJAX permite atualizações sem recarregar o documento.
- d) Certo. Vide letra C.
- e) Errado. Regras de CSS estão em arquivos CSS, não XML.

Gabarito: Letra D

05. (VUNESP/DPE SP/2023) Sobre o uso do AJAX em uma página web, pode-se citar, como uma de suas vantagens,

- a) que a comunicação entre cliente e servidor é criptografada com as credenciais do AJAX.
- b) que o conteúdo da página pode ser atualizado sem recarregar a página.
- c) a substituição do HTML + CSS pelo próprio AJAX.
- d) o uso de um algoritmo de tunelamento para maior velocidade de comunicação.
- e) a adoção do protocolo UDP na comunicação.

Comentários:

Vamos às alternativas.

- a) Errado. O uso de AJAX não garante que teremos comunicações criptografadas.
- b) Certo. Essa é a principal vantagem do AJAX.
- c) Errado. AJAX faz requisições HTTP, não interferindo no HTML e CSS.
- d) Errado. Podemos ter tunelamento empregado, mas isso não advém do AJAX.
- e) Errado. O protocolo subjacente é o TCP, não o UDP.

Temos como correta, portanto, a letra B.

Gabarito: Letra B

06. (FGV/ALEMA/2023) O objeto XMLHttpRequest é essencial para o AJAX, sendo suportado por todos os navegadores modernos.

Os métodos válidos desse objeto são

- a) load() e reload().
- b) abort() e receive().
- c) open() e responseXML().
- d) getAllHeader() e setAllHeader().
- e) getAllResponseHeaders() e setRequestHeader().

Comentários:

O objeto XMLHttpRequest traz consigo alguns métodos:

- Métodos da requisição
 - setRequestHeader
 - open
 - send
 - abort
- Métodos da resposta
 - getResponseHeader
 - getAllResponseHeaders
 - load

Vamos analisar as alternativas, tachando os métodos que não pertencem ao objeto.

- a) Errado. load() e ~~reload()~~.
- b) Errado. abort() e ~~receive()~~.
- c) Errado. open() e ~~responseXML()~~.
- d) Errado. ~~getAllHeader()~~ e ~~setAllHeader()~~.
- e) Certo. getAllResponseHeaders() e setRequestHeader().

Gabarito: Letra E

07. (FGV/CÂMARA DOS DEPUTADOS/2023) Considere a requisição AJAX a seguir.

```
xhttp.open("GET", "teste.txt", true);  
xhttp.send();
```

O valor true, utilizado como o terceiro parâmetro do método, estabelece que

- a) a execução da requisição é síncrona.
- b) a execução da requisição é assíncrona.
- c) a execução da requisição inclui um call back.
- d) um evento erro na requisição deve ser tratado.
- e) em caso de sucesso, o valor é retornado pela requisição

Comentários:

O terceiro atributo do método open() é responsável por definir se a comunicação é assíncrona, com o atributo *true*, ou síncrona, com o atributo *false*. Nesse caso, como temos *true*, indica que se trata de uma requisição assíncrona.

Gabarito: Letra B

08. (CEBRASPE/TCE RJ/2022) Quanto ao desenvolvimento de sistemas web, julgue o item seguinte.

Um dos objetivos de usar Ajax é manter a página web atualizada, efetuando uma recarga completa para cada interação do usuário.

Comentários:

O AJAX realmente objetiva manter a página atualizada, mas isso é feito de forma assíncrona, resultando na desnecessidade de atualizar a página a cada interação.

Gabarito: Errado

09. (CEPS UFPA/UFPA/2022) As duas características de requisições AJAX são:

- a) Manipular o DOM e adicionar efeitos com animações nas páginas por uso de JS.
- b) Aplicar folha de estilos em elementos de HTML e realizar modificação de posições em tempo de execução.
- c) Transmitir dados de mídia em tempo real e executar dados em um Web Player.
- d) Fazer requisições para o servidor sem precisar carregar a página e receber e processar os dados do servidor.
- e) Gerenciar Web sockets no cliente e conectar diferentes agentes no servidor.

Comentários:

Vamos analisar as alternativas.

- a) Errado. O AJAX não manipula o DOM.
- b) Errado. A folha de estilos é aplicada pelo CSS, não pelo AJAX.
- c) Errado. A transmissão de dados em tempo real é feita por ferramentas específicas, e não envolvem o uso de AJAX.
- d) Certo. O AJAX é uma ferramenta de requisições assíncrona, que permite o uso de páginas interativas que não precisam ser recarregadas a cada interação.
- e) Errado. O AJAX lida com o XMLHttpRequest, não com Web Sockets.

Gabarito: Letra D

10. (VUNESP/PREF. PRES. PRUDENTE/2022) O principal objeto suportado pela maioria dos navegadores modernos e que é essencial para o funcionamento da tecnologia AJAX é o:

- a) XMLHttpRequest
- b) HTMLElement
- c) XMLHttpRequest
- d) XPathResult
- e) XMLSerializer

Comentários:

O principal objeto responsável pela operacionalização do AJAX, que faz a transmissão das ações feitas nos navegadores para os servidores é o XMLHttpRequest.

Gabarito: Letra C

11. (FUNDEP/UFJF/2022) Sobre a técnica AJAX, assinale com V as afirmativas verdadeiras e com F as falsas.

- () A técnica permite que os dados sejam carregados dinamicamente em uma página web, sem necessidade de recarregá-la por completo.
- () Os dados são recebidos sempre no formato JSON.
- () JSONP é uma técnica utilizada para requisitar dados no formato JSON.
- () A comunicação em AJAX deve ser feita de forma assíncrona.

Assinale a sequência correta.

- a) F V V F
- b) V F V F
- c) F V F V
- d) F F V V
- e) V F F V

Comentários:

Vamos analisar cada alternativa.

- () A técnica permite que os dados sejam carregados dinamicamente em uma página web, sem necessidade de recarregá-la por completo.

Verdadeiro. Esse é um dos principais objetivos do AJAX.

- () Os dados são recebidos sempre no formato JSON.

Falso. Podemos ter respostas em texto, HTML, JSON, Blob etc.

- () JSONP é uma técnica utilizada para requisitar dados no formato JSON.

Falso. JSONP é uma técnica destinada a delimitar políticas para o arquivo JSON.

- () A comunicação em AJAX deve ser feita de forma assíncrona.

Verdadeiro. Apesar de podermos ter requisições ocorrendo de forma síncrona, o AJAX deve sempre rodar de forma assíncrona.

Portanto, temos V F F V.

Gabarito: Letra E

12. (CEBRASPE/BANRISUL/2022) A respeito de tecnologias e protocolos web, julgue o item que se segue.

A tecnologia Ajax separa as interações com o usuário das interações com o servidor, de modo que ambas sejam executadas paralelamente.

Comentários:

Perfeito! É exatamente essa abordagem de separação de responsabilidades, separando requisição e resposta como objetos distintos, que permite a comunicação assíncrona.

13. (VUNESP/PREF. PIRACICABA/2022) AJAX (Asynchronous JavaScript and XML) utiliza o objeto XMLHttpRequest para realizar a comunicação com servidores. Uma vantagem de sua utilização é permitir fazer requisições para o servidor sem necessitar atualizar a página web. Sobre o método open do XMLHttpRequest, é correto afirmar que:

- a) a URL fornecida para o método deverá estar no formato DOMString.
- b) não é possível configurar a requisição para que ela ocorra de maneira síncrona.
- c) é possível utilizar o método sem fornecer uma URL, desde que ela tenha sido fornecida no momento de criação do objeto.
- d) é possível fornecer informação correspondente ao nome do usuário para o método fazer a requisição, porém não é possível fornecer a informação de senha do usuário, sendo necessário outro método para este propósito.
- e) caso a informação correspondente ao método de requisição HTTP não seja fornecida, é adotado por padrão a requisição do tipo GET.

Comentários:

Vamos às alternativas.

- a) Certo. A URL fornecida deve ser em formato de string, caracterizado pelas aspas ` `.
- b) Errado. Podemos definir a requisição de forma síncrona e assíncrona.
- c) Errado. A URL é passada unicamente junto do método.
- d) Errado. Podemos fornecer informações do usuário e forma completa, tanto usuário quanto senha.
- e) Errado. Não há funcionamento da sintaxe sem especificar o método.

14. (QUADRIX/PRODAM/2022) Assinale a alternativa que apresenta um dos princípios do AJAX.

- a) Os browsers são apenas terminais que possuem acesso ao sistema, portanto, cada vez que o usuário necessita requisitar uma informação nova, ele deve solicitar tal informação ao servidor.
- b) Os servidores não enviam dados, apenas os recebem.
- c) O AJAX utiliza somente o modo de atualização de dados síncrono.
- d) A utilização correta do AJAX diminui a performance dos aplicativos.
- e) Não há interação dos aplicativos com o usuário.

Comentários:

Vamos analisar as alternativas.

- a) Certo. Os navegadores, ou browsers, são apenas terminais de exibição gráfica do código. Cada solicitação deve ser feita diretamente ao servidor - e isso é intermediado pelo XMLHttpRequest.
- b) Errado. Eles enviam e recebem.

- c) Errado. AJAX utiliza o modo de atualização síncrono e assíncrono.
- d) Errado. Pelo contrário, o assíncronismo do AJAX melhora a performance.
- e) Errado. O usuário interage com o aplicativo através de botões, textos etc.

Portanto, gabarito letra A.

Gabarito: Letra A

15. (CEBRASPE/BANESE/2021) Acerca de desenvolvimento web em Java, julgue o próximo item.

A ferramenta Ajax permite o desenvolvimento de aplicações juntamente com as interações com o usuário e o servidor, o que faz as duas avançarem juntas.

Comentários:

Difícil entender o que o examinador quis dizer com essa frase. Provavelmente ele quis dizer que as aplicações se desenvolvem conjuntamente com os servidores no sentido da comunicação ser síncrona - o que é falso.

Gabarito: Errado

16. (FGV/BANESTES/2021) No contexto das aplicações web, Ajax é uma tecnologia que destina-se primordialmente a:

- a) criptografar dados que são transmitidos pela rede;
- b) enviar e receber dados do servidor sem precisar recarregar a página inteira;
- c) evitar ataques por meio de aplicativo web;
- d) implementar mecanismos de login de duas etapas;
- e) controlar aplicativos web que operam em diferentes plataformas.

Comentários:

O AJAX destina-se a implementar requisições e respostas HTTP, isso é, envio e recebimento de dados, por meio de comunicação assíncrona, permitindo que recarreguemos apenas o elemento que sofre essa interação, sem necessidade de recarregar toda a página.

Gabarito: Errado

17. (VUNESP/UFABC/2019) Uma das tecnologias utilizadas para o desenvolvimento de páginas web é o conjunto denominado AJAX, que apresenta a seguinte característica:

- a) a cada solicitação de nova informação pelo usuário, a página web é novamente carregada por inteiro.
- b) a comunicação do computador do usuário com o servidor requer autorização expressa desse usuário.
- c) a letra A do acrônimo AJAX significa Advanced.
- d) ao se solicitar uma página internet, o navegador recebe um programa escrito em javascript.

- e) utilizar o API XMLHttpRequest para criptografar a página em exibição (para a comunicação de dados entre cliente e servidor, utilizando o protocolo HTTP).

Comentários:

Vamos às alternativas.

- a) Errado. O AJAX é assíncrono, o que permite que interajamos com a página sem a necessidade de recarregá-la.
- b) Errado. A autorização expressa de comunicação pode, ou não, ser exigida.
- c) Errado. O A é referente a "Asynchronous".
- d) Certo. O navegador recebe um programa em JavaScript, que é "decodificado" pelo XMLHttpRequest e enviado para o servidor.
- e) Errado. A API faz a "ponte" da comunicação, e não a criptografia.

Gabarito: Letra D

18. (CEBRASPE/CGM JOÃO PESSOA/2018) Acerca de XMLHttpRequest e de linguagens de modelagem, julgue o item.

XMLHttpRequest é uma API utilizada pela tecnologia de apresentação Ajax e fornece a recuperação de dados de uma URL, em qualquer formato textual, sem que seja necessário atualizar completamente a página apresentada ao usuário.

Comentários:

Perfeito! A afirmativa descreve perfeitamente o funcionamento do XMLHttpRequest, API utilizada pelo AJAX para requisições HTTP.

Gabarito: Correto

19. (CEBRASPE/STM/2018 - adaptada) Julgue o item subsequente, a respeito de programação web.

Em HTML5, o valor igual a 2 para o atributo readyState do AJAX indica que a conexão foi fechada.

Comentários:

O readyState representa o estado da requisição HTTP. Temos 5 estados possíveis:

NÚMERO	DESCRIÇÃO
0	Requisição não inicializada

1	Conexão estabelecida
2	Requisição recebida
3	Processando requisição
4	Requisição finalizada, e resposta está pronta

Portanto o número 2 indica requisição recebida, e não conexão fechada.

Gabarito: Errado

20. (FGV/CM Salvador/2018) Para viabilizar a utilização do AJAX na Web, é importante que o browser ofereça suporte ao objeto denominado:

- a) AJAX_Http;
- b) AJAX_XML_Response;
- c) Http://AJAX;
- d) JSON_AJAX;
- e) XMLHttpRequest.

Comentários:

O objeto que deve ter suporte oferecido é o XMLHttpRequest.

Gabarito: Letra E

QUESTÕES COMENTADAS

01. (VUNESP/PREF. MARÍLIA/2023) AJAX utiliza o objeto XMLHttpRequest para fazer a requisição de dados a um servidor. O método open() deste objeto tem como um de seus parâmetros o tipo de requisição HTTP que será utilizada.

Nesse contexto, assinale a alternativa que apresenta requisições válidas.

- a) FIND e DELETE.
- b) MOVE e GREP.
- c) PUT e DEFINE.
- d) GET e GREP.
- e) GET e POST.

02. (FCC/MPE PB/2023) Em uma aplicação web que utiliza AJAX, considere a existência de um objeto http_obj do tipo XMLHttpRequest inicializado e em condições ideais. Nestas condições, para enviar uma requisição para o arquivo dados.jsp no servidor, um analista utilizou as instruções abaixo.

```
http_obj.open(a, b, c);  
http_obj.send();
```

Considerando a sintaxe correta do método open e que a, b e c são variáveis que contêm valores válidos, a variável

- a) a deve conter dados.jsp.
- b) c deve conter dados.jsp.
- c) a deve conter GET ou POST.
- d) c deve conter GET ou POST.
- e) b deve conter true ou false.

03. (VUNESP/CM SBO/2023) O seguinte código de uma página HTML com JavaScript utiliza AJAX para a chamada de uma página JSP.

```
<html>  
<body>  
  
<div id="d">  
<button type="button" onclick="loadDoc()">AJAX</button>  
</div>  
  
<script>  
function loadDoc() {  
    var xhttp = new XMLHttpRequest();  
    xhttp.onreadystatechange = function() {  
        if (this.readyState == 4 && this.status == 200) {
```

```
        document.getElementById("d").innerHTML = this.responseText;
    }
};
xhttp.open("GET", "ajax_info.jsp", true);
xhttp.send();
}

</script>
</body>
</html>
```

A div identificada por d tem seu conteúdo alterado por meio da chamada à página ajax_info.jsp

- a) assim que a carga da página HTML no navegador é finalizada, apenas uma vez.
- b) durante a carga da página HTML no navegador, que já é renderizada pela primeira vez com o conteúdo retornado pela página ajax_info.jsp.
- c) assim que a carga da página HTML no navegador é finalizada, repetidas vezes, pois o código apresentado provoca um loop infinito que força a recarga da página HTML após cada requisição a ajax_info.jsp.
- d) sem que a página HTML seja recarregada, após o usuário clicar no botão "AJAX".
- e) após o usuário clicar no botão "AJAX", o que provoca a recarga completa da página HTML com o conteúdo atualizado.

04. (VUNESP/DPE SP/2023) O seguinte trecho de código foi encontrado em uma página HTML

```
<script>
function loadDoc() {
    const xhttp = new XMLHttpRequest();
    xhttp.onload = function() {
        myFunction(this);
    }
    xhttp.open("GET", "arq.xml");
    xhttp.send();
}
...
</script>
```

sendo que a implementação da função myFunction foi propositalmente omitida.

Com base nas informações apresentadas, é correto afirmar que

- a) a função myFunction aplica alguma regra CSS a algum elemento da página.
- b) não é possível concluir se AJAX é utilizado sem conhecer a implementação de myFunction.
- c) é utilizado AJAX para efetuar uma requisição ao servidor web. A chamada xhttp.send() faz com que a página seja recarregada no navegador.
- d) é utilizado AJAX para efetuar uma requisição ao servidor web sem recarregar a página na chamada xhttp.send().

e) o arquivo `arq.xml` contém regras de estilo CSS.

05. (VUNESP/DPE SP/2023) Sobre o uso do AJAX em uma página web, pode-se citar, como uma de suas vantagens,

- a) que a comunicação entre cliente e servidor é criptografada com as credenciais do AJAX.
- b) que o conteúdo da página pode ser atualizado sem recarregar a página.
- c) a substituição do HTML + CSS pelo próprio AJAX.
- d) o uso de um algoritmo de tunelamento para maior velocidade de comunicação.
- e) a adoção do protocolo UDP na comunicação.

06. (FGV/ALEMA/2023) O objeto `XMLHttpRequest` é essencial para o AJAX, sendo suportado por todos os navegadores modernos.

Os métodos válidos desse objeto são

- a) `load()` e `reload()`.
- b) `abort()` e `receive()`.
- c) `open()` e `responseXML()`.
- d) `getAllHeader()` e `setAllHeader()`.
- e) `getAllResponseHeaders()` e `setRequestHeader()`.

07. (FGV/CÂMARA DOS DEPUTADOS/2023) Considere a requisição AJAX a seguir.

```
xhttp.open("GET", "teste.txt", true);  
xhttp.send();
```

O valor `true`, utilizado como o terceiro parâmetro do método, estabelece que

- a) a execução da requisição é síncrona.
- b) a execução da requisição é assíncrona.
- c) a execução da requisição inclui um call back.
- d) um evento erro na requisição deve ser tratado.
- e) em caso de sucesso, o valor é retornado pela requisição

08. (CEBRASPE/TCE RJ/2022) Quanto ao desenvolvimento de sistemas web, julgue o item seguinte.

Um dos objetivos de usar Ajax é manter a página web atualizada, efetuando uma recarga completa para cada interação do usuário.

09. (CEPS UFPA/UFPA/2022) As duas características de requisições AJAX são:

- a) Manipular o DOM e adicionar efeitos com animações nas páginas por uso de JS.
- b) Aplicar folha de estilos em elementos de HTML e realizar modificação de posições em tempo de execução.
- c) Transmitir dados de mídia em tempo real e executar dados em um Web Player.

- d) Fazer requisições para o servidor sem precisar carregar a página e receber e processar os dados do servidor.
- e) Gerenciar Web sockets no cliente e conectar diferentes agentes no servidor.

10. (VUNESP/PREF. PRES. PRUDENTE/2022) O principal objeto suportado pela maioria dos navegadores modernos e que é essencial para o funcionamento da tecnologia AJAX é o:

- a) XMLHttpRequest
- b) HTMLObjectElement
- c) XMLHttpRequest
- d) XPathResult
- e) XMLSerializer

11. (FUNDEP/UFJF/2022) Sobre a técnica AJAX, assinale com V as afirmativas verdadeiras e com F as falsas.

- () A técnica permite que os dados sejam carregados dinamicamente em uma página web, sem necessidade de recarregá-la por completo.
- () Os dados são recebidos sempre no formato JSON.
- () JSONP é uma técnica utilizada para requisitar dados no formato JSON.
- () A comunicação em AJAX deve ser feita de forma assíncrona.

Assinale a sequência correta.

- a) F V V F
- b) V F V F
- c) F V F V
- d) F F V V
- e) V F F V

12. (CEBRASPE/BANRISUL/2022) A respeito de tecnologias e protocolos web, julgue o item que se segue.

A tecnologia Ajax separa as interações com o usuário das interações com o servidor, de modo que ambas sejam executadas paralelamente.

13. (VUNESP/PREF. PIRACICABA/2022) AJAX (Asynchronous JavaScript and XML) utiliza o objeto XMLHttpRequest para realizar a comunicação com servidores. Uma vantagem de sua utilização é permitir fazer requisições para o servidor sem necessitar atualizar a página web. Sobre o método open do XMLHttpRequest, é correto afirmar que:

- a) a URL fornecida para o método deverá estar no formato DOMString.
- b) não é possível configurar a requisição para que ela ocorra de maneira síncrona.
- c) é possível utilizar o método sem fornecer uma URL, desde que ela tenha sido fornecida no momento de criação do objeto.
- d) é possível fornecer informação correspondente ao nome do usuário para o método fazer a requisição, porém não é possível fornecer a informação de senha do usuário, sendo necessário outro método para este propósito.

- e) caso a informação correspondente ao método de requisição HTTP não seja fornecida, é adotado por padrão a requisição do tipo GET.

14. (QUADRIX/PRODAM/2022) Assinale a alternativa que apresenta um dos princípios do AJAX.

- a) Os browsers são apenas terminais que possuem acesso ao sistema, portanto, cada vez que o usuário necessita requisitar uma informação nova, ele deve solicitar tal informação ao servidor.
- b) Os servidores não enviam dados, apenas os recebem.
- c) O AJAX utiliza somente o modo de atualização de dados síncrono.
- d) A utilização correta do AJAX diminui a performance dos aplicativos.
- e) Não há interação dos aplicativos com o usuário.

15. (CEBRASPE/BANESE/2021) Acerca de desenvolvimento web em Java, julgue o próximo item.

A ferramenta Ajax permite o desenvolvimento de aplicações juntamente com as interações com o usuário e o servidor, o que faz as duas avançarem juntas.

16. (FGV/BANESTES/2021) No contexto das aplicações web, Ajax é uma tecnologia que destina-se primordialmente a:

- a) criptografar dados que são transmitidos pela rede;
- b) enviar e receber dados do servidor sem precisar recarregar a página inteira;
- c) evitar ataques por meio de aplicativo web;
- d) implementar mecanismos de login de duas etapas;
- e) controlar aplicativos web que operam em diferentes plataformas.

17. (VUNESP/UFABC/2019) Uma das tecnologias utilizadas para o desenvolvimento de páginas web é o conjunto denominado AJAX, que apresenta a seguinte característica:

- a) a cada solicitação de nova informação pelo usuário, a página web é novamente carregada por inteiro.
- b) a comunicação do computador do usuário com o servidor requer autorização expressa desse usuário.
- c) a letra A do acrônimo AJAX significa Advanced.
- d) ao se solicitar uma página internet, o navegador recebe um programa escrito em javascript.
- e) utilizar o API XMLHttpRequest para criptografar a página em exibição (para a comunicação de dados entre cliente e servidor, utilizando o protocolo HTTP).

18. (CEBRASPE/CGM JOÃO PESSOA/2018) Acerca de XMLHttpRequest e de linguagens de modelagem, julgue o item.

XMLHttpRequest é uma API utilizada pela tecnologia de apresentação Ajax e fornece a recuperação de dados de uma URL, em qualquer formato textual, sem que seja necessário atualizar completamente a página apresentada ao usuário.

19. (CEBRASPE/STM/2018 - adaptada) Julgue o item subsequente, a respeito de programação web.

Em HTML5, o valor igual a 2 para o atributo readyState do AJAX indica que a conexão foi fechada.

20. (FGV/CM Salvador/2018) Para viabilizar a utilização do AJAX na Web, é importante que o browser ofereça suporte ao objeto denominado:

- a) AJAX_Http;
- b) AJAX_XML_Response;
- c) Http://AJAX;
- d) JSON_AJAX;
- e) XMLHttpRequest.

GABARITO

GABARITO



1. Letra E
2. Letra C
3. Letra C
4. Letra D
5. Letra B
6. Letra E
7. Letra B
8. Errado
9. Letra D
10. Letra C

11. Letra E
12. Correto
13. Letra A
14. Letra A
15. Errado
16. Errado
17. Letra D
18. Correto
19. Errado
20. Letra E

PWA (PROGRESSIVE WEB APPS)

Conceitos Básicos

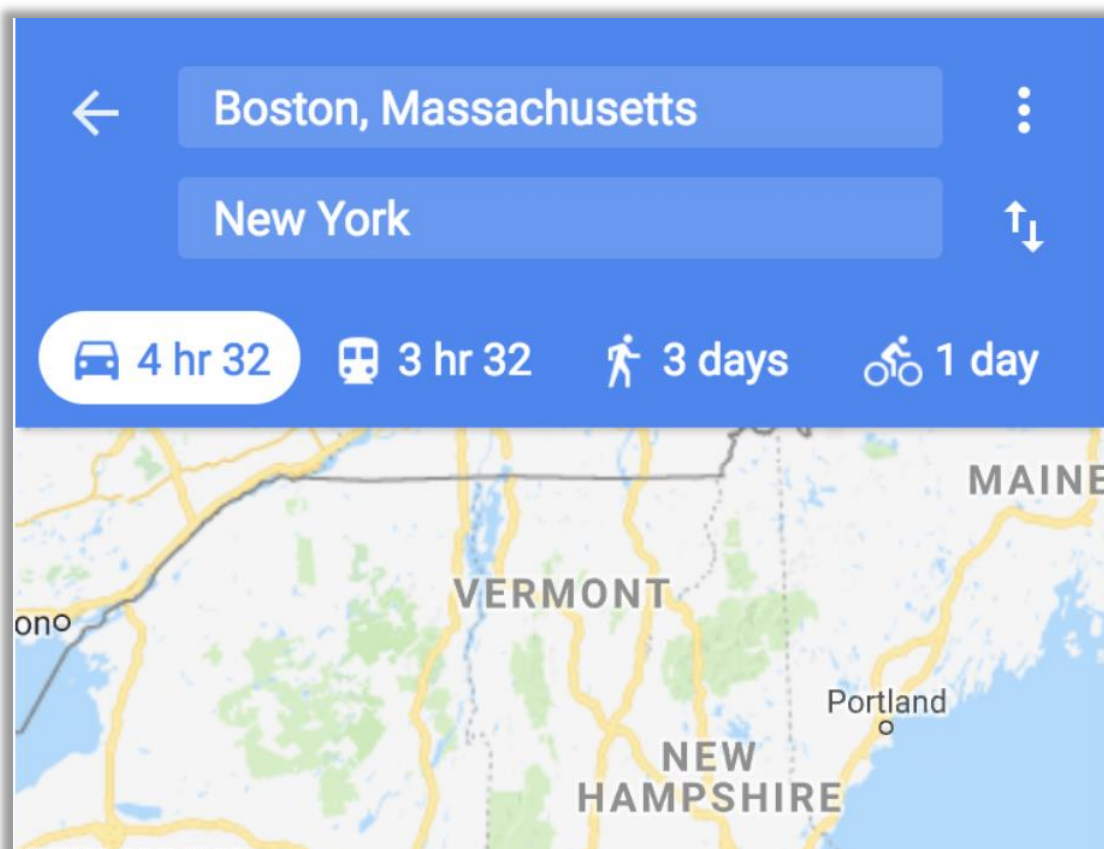
INCIDÊNCIA EM PROVA: MÉDIA

Galera esse assunto ainda está em evolução, como quase tudo na TI, mas esse em específico, dada a sua relevância no atual cenário das plataformas como serviço, se encontra literalmente em evolução. *Como assim, professor?*

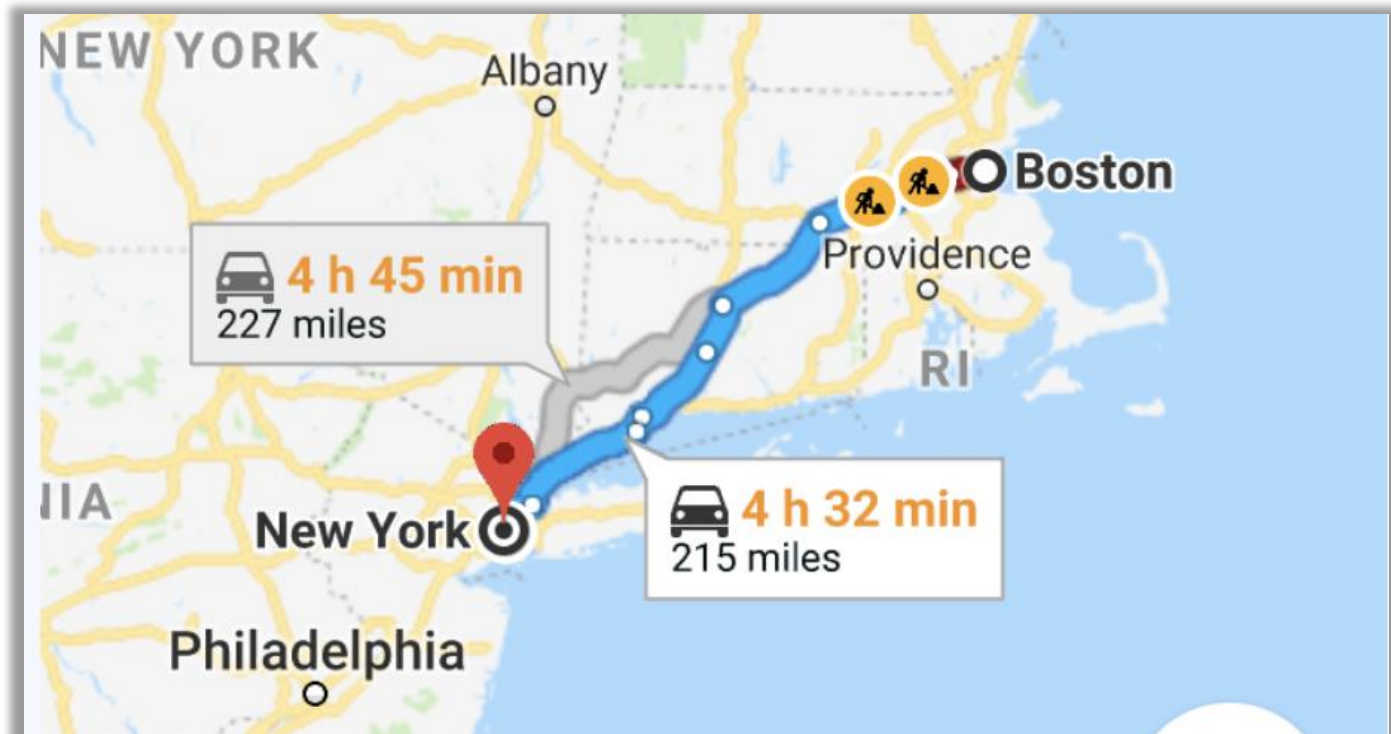
Os PWAs (Progressive Web Apps), caracterizam-se por serem progressivos, responsivos e semelhantes a aplicativos. São o novo horizonte na construção de plataformas, sendo uma das metodologias mais empregadas, quando o assunto é a criação de aplicações nativas, em especial de smartphones, web 3.0, etc.

A ideia é simples, a internet evolui e as plataformas também evoluem e a orientação a serviço é cada vez mais presente, nas nossas vidas. Os PWAs oferecem através de navegadores, design de aplicações (apps) independente de plataforma, ou seja, não importa se você está acessando da sua smartTV ou do seu smartphone, a experiência do usuário será semelhante.

Vamos explorar isso com os sites, para ficar mais aderente. Imagina que você acessou o Chrome ou Mozilla Firefox e quer acesso o Google Maps.



Onde entra o PWA? Simples: Se através de uma página web, você conseguir acessar uma função nativa de um app, teremos Progressive Web Apps em ação.



O que você acabou de ver é uma versão do Google Maps, que está disponível na versão PWA, assim como outros aplicativos comuns, como Twitter e Instagram. Logo, podemos derivar que um PWA é um meio para estender aplicações nativas de apps, para websites tradicionais.

Bom, agora imagino que esteja bem claro que todos nós utilizamos PWA diariamente, principalmente quando visitamos marketplaces. As funcionalidades disponíveis nos navegadores mais modernos, tornam os PWAs rivais de peso dos aplicativos comuns (apps). Exemplo:

RECURSOS PWA	DESCRIÇÃO
NOTIFICAÇÃO EM PUSH	os famosos alertas dos aplicativos podem ser executados a partir dos PWAs, mesmo após fechar o navegador;
SPLASH SCREEN	os PWAs atuais também podem exibir telas de abertura para apresentar a marca e sua identidade visual;
ÍCONE NA HOME DO SMARTPHONE	assim como os apps tradicionais, os PWAs podem guardar ícones na home do celular com link direto para o site ou uma área específica dele;
PROCESSOS EM BACKGROUND	também é possível rodar serviços e funções em segundo plano, sem interferir na experiência do usuário;
SUPORTE OFFLINE	como dito, os PWAs são capazes de manter funcionalidades disponíveis sem internet, utilizando o cache do navegador;
ACESSO À CÂMERA, ARQUIVOS, CONTATOS E GEOLOCALIZAÇÃO	As aplicações PWA podem acessar recursos do dispositivo para aprimorar a experiência no app.

QUESTÕES COMENTADAS – PWA

1. (CESPE / DPE-RO - 2022) Uma das vantagens do PWA em relação a outros *front-ends* é

- a) a utilização de NFC.
- b) a disponibilidade em lojas de aplicativos.
- c) o funcionamento *offline*.
- d) o suporte *cross-browser*.
- e) a utilização de *bluetooth*.

Comentários:

Galera, conforme vimos, funcionar offline é uma das vantagens dos PWAs. Só lembrar:

RECURSOS PWA	DESCRIÇÃO
NOTIFICAÇÃO EM PUSH	os famosos alertas dos aplicativos podem ser executados a partir dos PWAs, mesmo após fechar o navegador;
SPLASH SCREEN	os PWAs atuais também podem exibir telas de abertura para apresentar a marca e sua identidade visual;
ÍCONE NA HOME DO SMARTPHONE	assim como os apps tradicionais, os PWAs podem guardar ícones na home do celular com link direto para o site ou uma área específica dele;
PROCESSOS EM BACKGROUND	também é possível rodar serviços e funções em segundo plano, sem interferir na experiência do usuário;
SUPOORTE OFFLINE	como dito, os PWAs são capazes de manter funcionalidades disponíveis sem internet, utilizando o cache do navegador;
ACESSO À CÂMERA, ARQUIVOS, CONTATOS E GEOLOCALIZAÇÃO	As aplicações PWA podem acessar recursos do dispositivo para aprimorar a experiência no app.

Gabarito: Letra C

2. (IADES / BRB - 2019) O desenvolvimento de uma PWA pressupõe:

- a) Utilização de tecnologias comuns da Web, incluindo HTML, CSS e JavaScript.
- b) Codificação nativa na plataforma de destino (seja ela iOS ou Android).
- c) Desenvolvimento de uma aplicação considerada híbrida, pois será executada em uma Webview e terá acesso a recursos nativos do dispositivo via uma API JavaScript.
- d) Necessidade de disponibilidade ininterrupta de conectividade com a internet por se tratar de uma aplicação Web.
- e) Nenhum tipo de acesso aos recursos nativos do dispositivo, pois a aplicação será executada em um ambiente de navegador.

Comentários:

A questão afirma o óbvio: HTML, CSS e JS são fundamentais em conteúdo, aparência e interatividade de qualquer página web, logo pressupor isso para PWA é essencial.

Gabarito: Letra A

3. (COMPERVE / UFRN - 2019) Progressive Web Apps são experiências que combinam a web com os aplicativos. Eles são acessados por usuários por meio de um navegador sem exigir instalações e, conforme o usuário desenvolve uma relação com o aplicativo, ele se torna cada vez mais eficaz. Um progressive web app caracteriza-se por ser

- a) descobrível, nativo e independente de conectividade.
- b) progressivo, nativo e semelhante a aplicativos.
- c) descobrível, responsivo e acessível por lojas de aplicativos.
- d) progressivo, responsivo e semelhante a aplicativos.

Comentários:

Conforme estudamos, um PWA caracteriza-se por ser progressivo, responsivo e semelhante a aplicativos.

Gabarito: Letra D

4. (QUESTÃO INÉDITA) Os PWAs encerram um dos maiores desafios envolvidos na criação de apps: as limitações e regras definidas pelas grandes lojas, como o Google Play e a Apple Store, que podem tornar os projetos muito mais caros e complexos.

Comentários:

Perfeito! Os PWAs batem de frente com os apps quanto as limitações das Stores, como Google Play ou Apple Store, são mais simples e fáceis de elaboração.

Gabarito: Correto

5. (QUESTÃO INÉDITA) Em relação ao acesso do usuário, os PWAs são uma solução mais simples, afinal, todos serviços podem ser usados, simplesmente, entrando em um site e baixando a aplicação. Com os apps tradicionais, é necessário acessar a loja do sistema, baixar a aplicação, abrir e ainda conceder uma série de permissões para, finalmente, utilizá-los.

Comentários:

Basta acessar a web em um site que já seja compatível com PWA, não precisa baixar nenhuma aplicação, este é o erro da questão.

Gabarito: Errado

6. **(QUESTÃO INÉDITA)** Ao criar um PWA para sua empresa, seus desenvolvedores poderão fornecer updates normalmente. A diferença, aqui, é que o usuário não precisará realizar nenhum download de pacotes adicionais, pois todos os processos se mantêm vinculados ao site.

Comentários:

Exato! Ao criar um PWA para sua empresa, seus desenvolvedores poderão fornecer updates normalmente. A diferença, aqui, é que o usuário não precisará realizar nenhum download de pacotes adicionais, pois todos os processos se mantêm vinculados ao site.

Gabarito: Correto

7. **(QUESTÃO INÉDITA)** Os PWAs têm o potencial de ampliar significativamente os acessos a aplicações web, mas não oferecem redução de custos com desenvolvimento web.

Comentários:

Claramente oferecem redução de custos com desenvolvimento web, uma vez que a manutenção de apps exige diferentes equipes especializadas para cada tipo e geração de sistema operacional. Logo, PWAs são economicamente mais viáveis.

Gabarito: Errado

8. **(QUESTÃO INÉDITA)** As grandes redes sociais da atualidade oferecem PWAs em quase todas as funções dos seus apps oficiais.

Comentários:

Como dito, as redes sociais são sem dúvida um dos principais cases de utilização de PWAs.

Gabarito: Correto

9. **(QUESTÃO INÉDITA)** Os Progressive Web Apps podem favorecer blogs e portais de notícias gerando tráfego contínuo às suas páginas.

Comentários:

Além disso, os PWAs ainda podem incentivar a leitura de novos conteúdos nas redes sociais e newsletters, oferecendo acesso rápido pela home e por notificações, sem precisar convencer o seu público a baixar um app ou realizar qualquer tipo de cadastro.

Gabarito: Correto

10.(QUESTÃO INÉDITA) Os PWAs podem ser adicionados às homes dos smartphones, dispensando a necessidade de abrir o navegador e digitar uma URL.

Comentários:

Conforme vimos, os PWAs podem ser adicionados às homes dos smartphones, dispensando a necessidade de abrir o navegador e digitar uma URL.

Gabarito: Correto

LISTA DE QUESTÕES – PWA

1. (CESPE / DPE-RO - 2022) Uma das vantagens do PWA em relação a outros *front-ends* é
 - a) a utilização de NFC.
 - b) a disponibilidade em lojas de aplicativos.
 - c) o funcionamento *offline*.
 - d) o suporte *cross-browser*.
 - e) a utilização de *bluetooth*.
2. (IADES / BRB - 2019) O desenvolvimento de uma PWA pressupõe:
 - a) Utilização de tecnologias comuns da Web, incluindo HTML, CSS e JavaScript.
 - b) Codificação nativa na plataforma de destino (seja ela iOS ou Android).
 - c) Desenvolvimento de uma aplicação considerada híbrida, pois será executada em uma Webview e terá acesso a recursos nativos do dispositivo via uma API JavaScript.
 - d) Necessidade de disponibilidade ininterrupta de conectividade com a internet por se tratar de uma aplicação Web.
 - e) Nenhum tipo de acesso aos recursos nativos do dispositivo, pois a aplicação será executada em um ambiente de navegador.
3. (COMPERVE / UFRN - 2019) Progressive Web Apps são experiências que combinam a web com os aplicativos. Eles são acessados por usuários por meio de um navegador sem exigir instalações e, conforme o usuário desenvolve uma relação com o aplicativo, ele se torna cada vez mais eficaz. Um progressive web app caracteriza-se por ser
 - a) descobrível, nativo e independente de conectividade.
 - b) progressivo, nativo e semelhante a aplicativos.
 - c) descobrível, responsivo e acessível por lojas de aplicativos.
 - d) progressivo, responsivo e semelhante a aplicativos.
4. (QUESTÃO INÉDITA) Os PWAs encerram um dos maiores desafios envolvidos na criação de apps: as limitações e regras definidas pelas grandes lojas, como o Google Play e a Apple Store, que podem tornar os projetos muito mais caros e complexos.
5. (QUESTÃO INÉDITA) Em relação ao acesso do usuário, os PWAs são uma solução mais simples, afinal, todos serviços podem ser usados, simplesmente, entrando em um site e baixando a aplicação. Com os apps tradicionais, é necessário acessar a loja do sistema, baixar a aplicação, abrir e ainda conceder uma série de permissões para, finalmente, utilizá-los.

6. **(QUESTÃO INÉDITA)** Ao criar um PWA para sua empresa, seus desenvolvedores poderão fornecer updates normalmente. A diferença, aqui, é que o usuário não precisará realizar nenhum download de pacotes adicionais, pois todos os processos se mantêm vinculados ao site.
7. **(QUESTÃO INÉDITA)** Os PWAs têm o potencial de ampliar significativamente os acessos a aplicações web, mas não oferecem redução de custos com desenvolvimento web.
8. **(QUESTÃO INÉDITA)** As grandes redes sociais da atualidade oferecem PWAs em quase todas as funções dos seus apps oficiais.
9. **(QUESTÃO INÉDITA)** Os Progressive Web Apps podem favorecer blogs e portais de notícias gerando tráfego contínuo às suas páginas.
10. **(QUESTÃO INÉDITA)** Os PWAs podem ser adicionados às homes dos smartphones, dispensando a necessidade de abrir o navegador e digitar uma URL.

GABARITO – PWA

- | | | |
|------------|------------|-------------|
| 1. LETRA C | 5. ERRADO | 9. CORRETO |
| 2. LETRA A | 6. CORRETO | 10. CORRETO |
| 3. LETRA D | 7. ERRADO | |
| 4. CORRETO | 8. CORRETO | |

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.